SQL em ambiente Oracle

Índice

SQL em ambiente Oracle 1
Índice 2
I Introdução 5
Tabelas básicas a serem utilizadas no curso 5
Operações relacionais 7
Uma instalação Oracle típica 9
II SQL (Structured Query Language) 10
Introdução 10
Comandos DML SQL 11
Comando SELECT 12
1. Exibindo todos os dados de uma tabela. 13
2. Exibindo (projetando) o conteúdo de apenas algumas colunas da tabela. 14
3. Exibindo valores calculados. 15
4. Especificando um critério para seleção (filtragem) de linhas (cláusula `WHERE`). 17
5. Exibindo linhas com ou sem repetição de dados (modificadores `ALL` | `DISTINCT`). 22
6. Renomeando colunas no resultado da consulta (operador `AS`). 26
7. Projetando o resultado de uma função. 27
8. Exibindo os dados de uma tabela em uma determinada ordem (cláusula `ORDER BY`). 28
9. Fazendo totalizações simples (uso de funções de agregação SQL). 32
10. Produzindo totais com agrupamentos de linhas (cláusulas `GROUP BY` e `HAVING`). 35
11. Utilizando aliases (variáveis tupla) para tabelas. 38
12. Buscando dados em mais de uma tabela (junções). 39
13. Fazendo cálculos em agrupamentos com junção de várias tabelas. 48
14. Fazendo consultas com subconsultas. 50
15. Fazendo junção de uma tabela com ela mesma (auto-relacionamento). 56
16. Fazendo a operação de união de conjuntos (operador `UNION`). 57
17. Fazendo a operação de intersecção de conjuntos (operador `INTERSECT`). 58
18. Fazendo a operação de diferença de conjuntos (operador `MINUS`). 59
19. Utilizando condições ao projetar valores (expressões `CASE`). 60
20. Retendo apenas as n primeiras linhas de um resultado (pseudo-coluna `ROWNUM`). 62
Comando INSERT 63
21. Incluindo valores utilizando a cláusula `VALUES`. 64
22. Incluindo valores a partir de um `SELECT`. 66
Comando DELETE 67
23. Excluindo linhas de tabelas. 67
Comando UPDATE 69
24. Alterando várias colunas em um comando. 69
25. Alterando várias linhas em um comando. 70
Resumo dos principais operadores suportados pela SQL 71
Resumo das principais funções de agregação 72
Funções pré-definidas em SQL 72
ABS 73
ADD_MONTHS 73
ASCII 74
AVG 75
CEIL 77
CHR 77
COUNT 78
DECODE 80
FLOOR 80
GREATEST 81
INITCAP 81
INSTR 82
LAST_DAY 83
LEAST 83
LENGTH 84

LOWER....84
LPAD....85
LTRIM....86
MAX....87
MIN....88
MOD....89
MONTHS_BETWEEN....89
NEXT_DAY....90
NVL....91
POWER....92
REPLACE....93
ROUND – para números....94
ROUND – para datas....96
RPAD....98
RTRIM....100
SIGN....100
SOUNDEX....101
SQRT....101
STDDEV....102
SUBSTR....104
SUM....105
TO_CHAR – para números....107
TO_CHAR – para datas e horas....113
TO_DATE....117
TO_NUMBER....118
TRANSLATE....119
TRUNC – para números....120
TRUNC – para datas....122
UPPER....124
USERENV....125
VARIANCE....127
VSIZE....129
Comandos DDL SQL....130
Tipos de dados no Oracle....130
Regras para nomes no Oracle....131
Restrições (Constraints)....131
Comando CREATE TABLE....133
Comando ALTER TABLE....134
Comando DROP TABLE....135
Comando CREATE VIEW....136
Comando DROP VIEW....138
Comando CREATE INDEX....139
Comando DROP INDEX....140
Exercícios: Caso “Banco simplificado”....141
Comandos DCL SQL....143
Comando GRANT....143
Comando REVOKE....144
Comando CREATE USER....145
Comando DROP USER....145
III SQL *Plus....146
Introdução....146
Abertura de sessão SQL *Plus....147
Relação de comandos SQL *Plus....149
Comando LIST....150
Comando APPEND....151
Comando CHANGE....152
Comando DEL....153
Comando DESCRIBE....154
Comando EXIT....155
Comando INPUT....156

Instruções para consultas ao catálogo do SGBD....157
Como obter os nomes das tabelas disponíveis no banco de dados....157
Como ver a estrutura de uma tabela do banco de dados....157
Como ver os índices de uma tabela....158
Como obter os nomes das visões disponíveis no banco de dados....158
Como descobrir o SELECT de uma visão já criada no banco de dados....158
Como modificar a senha de um usuário do banco de dados....159

I Introdução

Tabelas básicas a serem utilizadas no curso

Para ilustrar os conceitos e recursos a serem trabalhados durante o curso, utilizaremos um conjunto de 3 tabelas bastante simples que implementam um pequeno controle de pessoal. As estruturas e os relacionamentos dessas tabelas são exibidos na figura 1.1 e para criá-las no Oracle você deverá executar o script `CriaPes_ora.sql`.

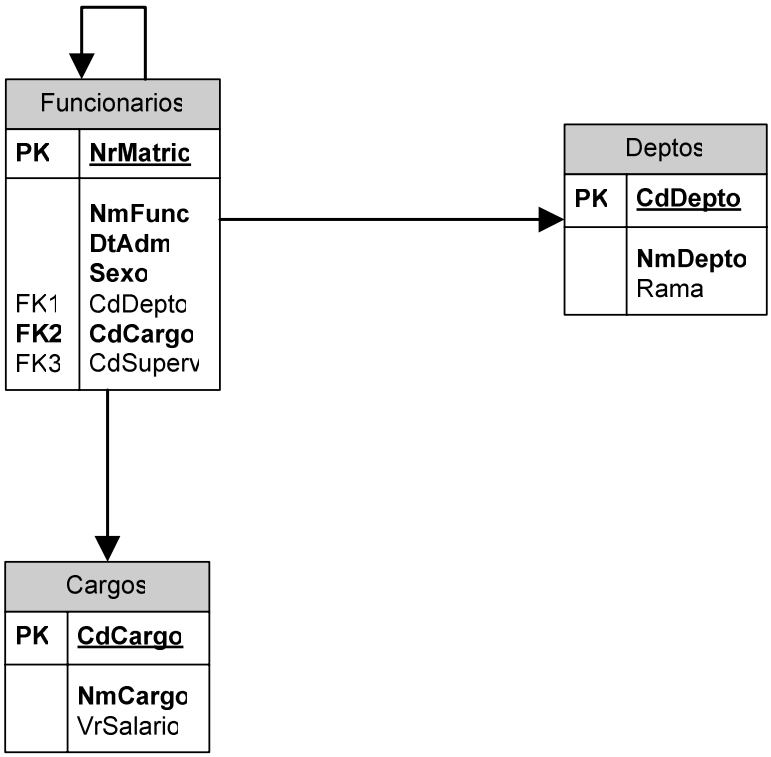


Figura 1.1: Esquema lógico do controle de funcionários.

A tabela Cargos contém a descrição dos cargos, por meio de 3 colunas: um código na coluna CdCargo, que é a sua chave primária; o nome do cargo, na coluna NmCargo, que é de preenchimento obrigatório e não aceita valores repetidos e o salário mensal referente ao cargo, que é armazenado na coluna numérica VrSalario. Na tabela Deptos está a descrição dos departamentos: um código, na coluna CdDepto, que é a chave primária da tabela; o nome do departamento, na coluna NmDepto, que é de preenchimento obrigatório e que não aceita valores repetidos e o número do ramal telefônico, armazenado na coluna Ramal. A tabela Funcionarios contém os dados dos funcionários cadastrados no banco de dados. NrMatric é o número de matrícula do funcionário e é a chave primária da tabela; NmFunc é o nome completo do funcionário, de preenchimento obrigatório; DtAdm indica a data em que o funcionário foi admitido na empresa; Sexo informa o gênero: 'M' para masculino e 'F' para feminino; CdDepto é a chave estrangeira de preenchimento opcional que associa um funcionário ao seu departamento; CdCargo é outra chave estrangeira, de preenchimento obrigatório, para indicar o cargo exercido pelo funcionário e, por fim, CdSuperv é o número de matrícula do supervisor do funcionário, que implementa uma auto-relacionamento opcional, já que um funcionário pode não ter supervisor.

Nas demonstrações vamos supor que cada tabela possui um pequeno número de linhas e que, por simplicidade, todos os dados alfabéticos armazenados nas tabelas estão em maiúsculas. O conteúdo inicial assumido para cada tabela é apresentado na figura 2.1 e, para incluir esses dados no banco de dados criado anteriormente no Oracle você deverá executar o script `DadosPes_ora.sql`.

Tabela FUNCIONARIOS

#	NrMatric	NmFunc	DtAdm	Sexo	CdCargo	CdDepto	CdSuperv
1	1001	JOAO SAMPAIO	10/08/1993	M	C002	D002	
2	1004	LUCIO TORRES	02/03/1994	M	C002	D002	1001
3	1034	ROBERTO PEREIRA	23/05/1992	M	C003	D001	1048
4	1021	JOSE NOGUEIRA	10/11/1994	M	C003	D001	1048
5	1029	RUTH DE SOUZA	05/11/1992	F	C001	D003	1048
6	1095	MARIA DA SILVA	03/09/1992	F	C004	D001	1048
7	1023	LUIZ DE ALMEIDA	12/01/1993	M	C002	D002	1001
8	1042	PEDRO PINHEIRO	29/07/1994	M	C004	D001	1048
9	1048	ANA SILVEIRA	01/06/1993	F	C005		
10	1015	PAULO RODRIGUES	17/08/1992	M	C002	D002	1001

Tabela CARGOS

#	CdCargo	NmCargo	VrSalario
1	C001	COZINHEIRA	350.00
2	C003	AUX. ESCRITÓRIO	450.00
3	C007	VIGIA	500.00
4	C002	MECANICO	750.00
5	C005	GERENTE	2300.00
6	C004	ESCRITURARIO	600.00

Tabela DEPTOS

#	CdDepto	NmDepto	Ramal
1	D001	ADMINISTRACAO	320
2	D002	OFICINA	310
3	D003	SERVICOS GERAIS	330
4	D004	VENDAS	300

Figura 2.1: Dados iniciais do controle de funcionários.

Observe que o funcionário 1048 não possui departamento e que, juntamente com o funcionário 1001, não possui supervisor. Isso significa que nas chaves estrangeiras correspondentes essas linhas possuem um valor Nulo. Todos os cargos possuem pelo menos um funcionário e o departamento D004 é o único que não possui funcionários cadastrados.

Operações relacionais

No modelo relacional, cada tabela (chamada relação) é tratada como um conjunto onde seus elementos são linhas (tuplas) e as manipulações sobre esses conjuntos é implementada através de operações especiais, chamadas operações relacionais, que são mostradas na tabela a seguir. O resultado de qualquer operação relacional é uma nova relação (tabela).

Operação	*Finalidade*
Seleção	Obter apenas as tuplas (linhas) desejadas em uma tabela
Projeção	Obter os componentes desejados (colunas) das linhas selecionadas
Junção	Obter uma tabela formada pela combinação das linhas de duas tabelas originais que possuem colunas com valores comuns
Produto Cartesiano	Obter todas as combinações possíveis dos elementos (linhas) de duas tabelas
União	Obter uma tabela cujas linhas foram obtidas a partir de duas tabelas originais diferentes (porém compatíveis)
Diferença	Obter as linhas de uma tabela que não possuem correspondência em outra tabela compatível
Renomeação	Atribuir apelidos lógicos a uma tabela dentro de uma operação
Intersecção	Obter os elementos (linhas) comuns a duas tabelas compatíveis
Divisão	Obter os elementos (linhas) de uma tabela que possuem correspondência com todos os elementos de um conjunto
Atribuição	Atribuir o resultado de uma seleção a uma variável para uso posterior

Algumas dessas operações são fundamentais e outras são suas derivadas. As operações que utilizamos com maior freqüência são a de Seleção, de Projeção e Junção. Exemplos de cada uma dessas operações com as tabelas que utilizaremos neste curso são mostrados a seguir.

Seleção: Obter todos os funcionários cuja coluna CdDepto possua o valor "D001".

Resultado:

NrMatric	NmFunc	DtAdm	Sexo	CdCargo	CdDepto
1034	ROBERTO PEREIRA	23/05/1992	M	C003	D001
1021	JOSE NOGUEIRA	10/11/1994	M	C003	D001
1095	MARIA DA SILVA	03/09/1992	F	C004	D001
1042	PEDRO PINHEIRO	29/07/1994	M	C004	D001

Projeção: Obter os valores referentes às colunas NmFunc e NrMatric de todos os funcionários cuja colunas CdCargo possua o valor "C002".

Resultado:

NmFunc	NrMatric
JOAO SAMPAIO	1001
LUCIO TORRES	1004
LUIZ DE ALMEIDA	1023
PAULO RODRIGUES	1015

Junção: Obter os valores referentes às colunas NrMatric, NmFunc e NmCargo de todas as linhas da tabela de funcionários em que a coluna CdDepto possua o valor "D001"

Resultado:

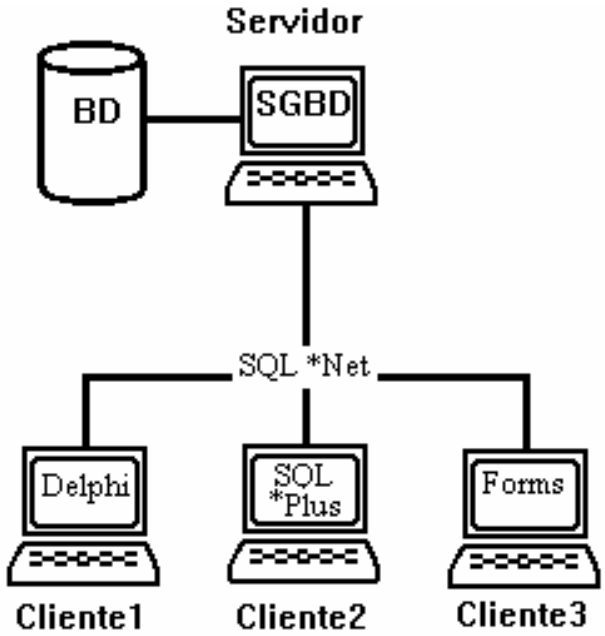
NrMatric	NmFunc	NmCargo
1034	ROBERTO PEREIRA	AUX. ESCRITÓRIO
1021	JOSE NOGUEIRA	AUX. ESCRITÓRIO
1095	MARIA DA SILVA	ESCRITURÁRIO
1042	PEDRO PINHEIRO	ESCRITURÁRIO

Observe que neste último caso precisamos combinar os dados de duas tabelas, uma vez que as colunas NrMatric e NmFunc a serem projetadas estão na tabela Funcionários e a coluna NmCargo encontra-se na tabela Cargos.

Uma instalação Oracle típica

Uma instalação Oracle típica é composta por um equipamento servidor onde está armazenado o banco de dados e onde roda o SGBD e por um conjunto de estações cliente que comunicam-se através de uma rede com o servidor. A rede pode ser, por exemplo, Novell NetWare, Linux ou MS-Windows NT e o protocolo, praticamente qualquer um, embora TCP/IP seja talvez o mais comum. Um produto de interconexão (*middleware*) da Oracle chamado SQL *Net (para a versão 7.x) ou Net8 (para a versão 8.x em diante) é instalado sobre o protocolo padrão da rede, viabilizando a comunicação do SGBD com os clientes Oracle.

A estação cliente pode rodar qualquer software que permita acessar o Servidor. Para acessos interativos ao SGBD existe um produto Oracle chamado SQL *Plus, que será utilizado neste curso.



Uma pessoa poderá utilizar os dados guardados pelo SGBD no banco de dados desde que tenha uma conta de usuário cadastrada. Sempre que quiser iniciar uma sessão de uso do banco de dados ela deverá primeiro conectar-se ao servidor de banco de dados (essa é uma outra denominação bastante comum para o SGBD) informando seu user_id, senha (se houver uma) e nome do banco de dados desejado (por ex. Vendas, Materiais, Pessoal, etc).

II SQL (Structured Query Language)

Introdução

Desenvolvida inicialmente nos laboratórios da IBM na primeira metade dos anos 70, a SQL é uma linguagem de acesso a bancos de dados. Baseia-se nos princípios lançados por E. F. Codd, tendo sido projetada para viabilizar a construção de um produto relacional pela IBM. Apesar dessa origem, constitui-se hoje num padrão internacional para os SGBDRs, com especificações formalizadas pela ANSI (o último padrão publicado é de 1992, chamado SQL-92) e adotadas em maior ou menor grau por vários fornecedores de bancos de dados, tendo sido implementada inclusive por produtos não exatamente relacionais.

Em suas definições, a SQL padrão prevê um reduzido conjunto de comandos, que podem ser classificados em 3 grupos:

Data Manipulation Language (DML): permite consultar, adicionar, alterar e excluir informações do banco de dados. Dentro da sintaxe de cada comando são permitidas algumas cláusulas adicionais que viabilizam operações relacionais tais como seleção, junção, união, etc e a utilização de algumas funções de uso mais comum, como de cálculo de média, soma, contagem de registros e determinação de máximo e mínimo entre valores.

Data Definition Language (DDL): permite criar e gerenciar a estrutura do banco de dados, como por exemplo criar ou eliminar tabelas e índices. É possível também a especificação de alguns itens de segurança adicionais, como por exemplo a definição de quais informações são obrigatórias dentro de uma dada tabela, que procedimentos devem ser associados a uma operação em particular sobre o banco de dados, etc.

Data Control Language (DCL): permite definir as contas dos usuários autorizados a acessar o banco de dados, bem como impor restrições de uso a esses usuários.

Uma visão um pouco mais detalhada dos principais comandos de cada grupo será vista mais adiante neste material.

Tecnicamente seria mais correto caracterizar a SQL padrão como uma sublinguagem, já que não proporciona comandos ou declarações próprios para o controle de repetições (loopings), de desvios (condicionais e incondicionais) ou de seqüências. Assim, não é possível a construção de aplicações com um mínimo de complexidade utilizando apenas a SQL padrão. Por esse motivo os principais gerenciadores do mercado desenvolveram extensões próprias (não padronizadas) à SQL de maneira a permitir a construção de rotinas para o SGBD. São exemplos dessas extensões a PL/SQL (Oracle) e Transact SQL (MS-SQL Server) entre outras.

Existem duas formas básicas em que a SQL é utilizada: a interativa (ou dinâmica) feita em nível de linha de comando, com os comandos sendo processados por um módulo interpretador, e a embutida ("*embedded* SQL"), onde o programa é codificado numa linguagem mais genérica (como o Cobol, C, Pascal ou PL/1) e faz chamadas a declarações SQL em algumas situações. Dizemos nesse último caso que a SQL está associada com uma linguagem "*host*" ou "hospedeira".

Mais recentemente, a difusão da SQL ganhou um significativo impulso com o advento da tecnologia Cliente / Servidor, tendo consolidado-se como o elemento chave na conexão entre SGBDs de fornecedores diferentes. É importante ressaltar que existem diferenças entre as implementações da sintaxe dos comandos conforme o fabricante, mas essas diferenças geralmente são secundárias e existe sempre uma sintaxe mínima que é suportada por todos os produtos.

Comandos DML SQL

Já vimos que os recursos SQL podem ser divididos em 3 grupos: DDL, DML e DCL. Veremos aqui a parte aparentemente mais evidente da SQL, aquela referente à manipulação de dados (DML), composta por um pequeno conjunto de comandos, uma variedade de operadores e inúmeras funções (algumas padronizadas para todos os SGBDRs, outras não). Um resumo dos comandos DML é apresentado na tabela a seguir:

Comando	***Descrição***
INSERT	Adiciona novos dados a uma tabela
DELETE	Remove dados de uma tabela
UPDATE	Modifica dados existentes em uma tabela
SELECT	Busca dados já cadastrados nas tabelas de um banco de dados

Veremos agora em detalhes cada um desses comandos, que serão apresentados inicialmente em sua sintaxe padrão e depois ilustrados por meio de diversos exemplos, onde os resultados esperados são apresentados e, sempre que necessário, comentados. É importante prestar a devida atenção a esses comentários, que discutem uma série de aspectos teóricos e práticos dos recursos utilizados.

Comando SELECT

Obtém um conjunto de linhas a partir de uma ou mais tabelas.

```
SELECT [ALL | DISTINCT] {* | <val1> [, <val2> ...]}
      FROM <tabela1> [, <tabela2> ...]
      [WHERE <condição_de_busca>]
      [GROUP BY <col1> [, <col2> ...]
      [HAVING <condição_de_busca>]
      [{UNION [ALL] | INTERSECT | MINUS } <select_expr>]
      [ORDER BY <lista_de_ordem> [ASC | DESC]] ;
```

Argumento	*Significado*
`SELECT [ALL | DISTINCT]`	Especifica os dados a recuperar. Utilizando `DISTINCT`, linhas duplicadas aparecem uma única vez. `ALL`, o default, recupera todas as linhas, sejam elas duplicadas ou não.
`* | <val1> [, <val2> ...]`	* retorna (faz a projeção) todas as colunas das tabelas especificadas. <val1> [, <val2> ...] retorna uma lista específica de colunas e valores.
`FROM <tabela1> [, <tabela2> ...]`	Lista de tabelas, *views* e *stored procedures* das quais os dados serão recuperados. Essa lista pode incluir junções, que podem ser aninhadas.
`WHERE <condição_de_busca>`	Especifica uma condição que limita as linhas a serem recuperadas.
`GROUP BY <col1> [, <col2> ...]`	Divide os resultados da consulta em grupos contendo todas as linhas com valor idêntico numa determinada coluna informada como argumento.
`HAVING <condição_de_busca>`	Utilizado em conjunto com `GROUP BY`, serve para especificar uma condição que deve ser satisfeita pelo conjunto de linhas de um agrupamento.
`UNION [ALL]`	Combina em forma de união de conjuntos os dados de duas consultas que possuem estrutura equivalente. Se a cláusula `ALL` for utilizada, linhas que existem em comum nas duas consultas são mantidas no resultado final, se `ALL` não for utilizado, as repetições são eliminadas.
`INTERSECT`	Combina em forma de intersecção de conjuntos os dados de duas consultas que possuem estrutura equivalente.
`MINUS`	Combina em forma de diferença de conjuntos os dados de duas consultas que possuem estrutura equivalente, retornando os elementos da primeira consulta que não existem na segunda.
`ORDER BY <lista_de_ordem> [ASC | DESC]`	Especifica o critério de ordenação das linhas a serem retornadas. `ASC` indica ordenação ascendente e `DESC`, ordenação descendente.

Exemplos:

1. Exibindo todos os dados de uma tabela.

✓ *Exemplo 1a:* Obtendo todo o conteúdo, ou seja, o valor de todas as colunas de todas as linhas, da tabela de funcionários.

```
SELECT *
      FROM Funcionarios ;
```

Resultado:

```
NRMATRIC  NMFUNC           DTADM       SEXO  CDCARGO  CDDEPTO  CDSUPERV
--------  ---------------  ----------  ----  -------  -------  --------
1048      ANA SILVEIRA     1993-06-01  F     C005
1001      JOAO SAMPAIO     1993-08-10  M     C002     D002     1048
1004      LUCIO TORRES     1994-03-02  M     C002     D002     1001
1042      PEDRO PINHEIRO   1994-07-29  M     C004     D001     1048
1034      ROBERTO PEREIRA  1992-05-23  M     C003     D001     1042
1021      JOSE NOGUEIRA    1994-11-10  M     C003     D001     1042
1029      RUTH DE SOUZA    1992-11-05  F     C001     D003     1042
1095      MARIA DA SILVA   1992-09-03  F     C004     D001     1042
1023      LUIZ DE ALMEIDA  1993-01-12  M     C002     D002     1001
1015      PAULO RODRIGUES  1992-08-17  M     C002     D002     1001
```

Comentários:

O asterisco colocado após a palavra `SELECT` significa "todas as colunas". Na cláusula `FROM` especificamos a tabela ou tabelas de onde os dados serão obtidos. A ordem em que as linhas do resultado são mostradas no seu computador pode ser diferente da indicada aqui. O formato da data de admissão também, pois depende de parâmetros de configuração do seu banco de dados.

✓ *Exemplo 1b:* Obtendo todo o conteúdo da tabela de cargos.

```
SELECT *
      FROM Cargos ;
```

Resultado:

```
CDCARGO   NMCARGO          VRSALARIO
--------  ---------------  ----------
C001      COZINHEIRA       350.00
C003      AUX. ESCRITORIO  450.00
C006      VIGIA            500.00
C002      MECANICO         750.00
C005      GERENTE          2300.00
C004      ESCRITURARIO     600.00
```

2. Exibindo (projetando) o conteúdo de apenas algumas colunas da tabela.

✓ *Exemplo 2a:* Listando, para cada funcionário, o seu nome, número de matrícula e data de admissão.

```
SELECT NmFunc, NrMatric, DtAdm
      FROM Funcionarios ;
```

Resultado:

```
NMFUNC           NRMATRIC  DTADM
---------------  --------  ----------
ANA SILVEIRA     1048      1993-06-01
JOAO SAMPAIO     1001      1993-08-10
LUCIO TORRES     1004      1994-03-02
PEDRO PINHEIRO   1042      1994-07-29
ROBERTO PEREIRA  1034      1992-05-23
JOSE NOGUEIRA    1021      1994-11-10
RUTH DE SOUZA    1029      1992-11-05
MARIA DA SILVA   1095      1992-09-03
LUIZ DE ALMEIDA  1023      1993-01-12
PAULO RODRIGUES  1015      1992-08-17
```

Comentários:

Observe que, em vez do asterisco, especificamos explicitamente quais as colunas que queremos ter no resultado.

✓ *Exemplo 2b:* Exibindo, para cada departamento, o seu nome e respectivo ramal telefônico.

```
SELECT NmDepto, Ramal
      FROM Deptos ;
```

Resultado:

```
NMDEPTO          RAMAL
---------------  -----
ADMINISTRACAO    320
OFICINA          310
SERVICOS GERAIS  330
VENDAS           300
```

3. Exibindo valores calculados.

✓ *Exemplo 3a:* Obtendo, para cada funcionário, o seu nome, data de admissão e a data que é 30 dias anterior à sua data de admissão.

```
SELECT NmFunc, DtAdm, DtAdm - 30
      FROM Funcionarios ;
```

Resultado:

```
NMFUNC           DTADM       DTADM-30
---------------  ----------  ----------
ANA SILVEIRA     1993-06-01  1993-05-02
JOAO SAMPAIO     1993-08-10  1993-07-11
LUCIO TORRES     1994-03-02  1994-01-31
PEDRO PINHEIRO   1994-07-29  1994-06-29
ROBERTO PEREIRA  1992-05-23  1992-04-23
JOSE NOGUEIRA    1994-11-10  1994-10-11
RUTH DE SOUZA    1992-11-05  1992-10-06
MARIA DA SILVA   1992-09-03  1992-08-04
LUIZ DE ALMEIDA  1993-01-12  1992-12-13
PAULO RODRIGUES  1992-08-17  1992-07-18
```

Comentários:

Um comando `SELECT` pode retornar valores de várias origens: colunas de tabelas, constantes, resultado de cálculos diversos, que podem envolver colunas ou não, e valores retornados por chamadas de função. No exemplo, para cada linha da tabela de funcionários foi projetado o valor das colunas `NmFunc` e `DtAdm`, além do resultado da subtração `DtAdm - 30`. Como a coluna `DtAdm` é uma data, o resultado é uma outra data, correspondente à data contida em `DtAdm` menos 30 dias.

✓ *Exemplo 3b:* Exibindo para cada cargo o seu código, nome e salário com 10% de aumento.

```
SELECT CdCargo, NmCargo, VrSalario * 1.1
      FROM Cargos ;
```

Resultado:

```
CDCARGO   NMCARGO          VRSALARIO*1.1
--------  ---------------  -------------
C001      COZINHEIRA       385.00
C003      AUX. ESCRITORIO  495.00
C006      VIGIA            550.00
C002      MECANICO         825.00
C005      GERENTE          2530.00
C004      ESCRITURARIO     660.00
```

Comentários:

É importante observar que o comando apresentado no exemplo não modifica os valores da coluna `VrSalario` da tabela, pois um comando `SELECT` apenas lê o conteúdo do banco de dados, mas não faz gravações. Caso quiséssemos realmente

modificar o conteúdo da coluna `VrSalario` deveríamos utilizar outro comando, o comando `UPDATE`, que será apresentado posteriormente.

4. Especificando um critério para seleção (filtragem) de linhas (cláusula `WHERE`).

✓ *Exemplo 4a:* Exibindo os funcionários do departamento D001.

```
SELECT *
      FROM Funcionarios
      WHERE CdDepto = 'D001' ;
```

Resultado:

```
NRMATRIC  NMFUNC           DTADM       SEXO  CDCARGO  CDDEPTO  CDSUPERV
--------  ---------------  ----------  ----  -------  -------  --------
1042      PEDRO PINHEIRO   1994-07-29  M     C004     D001     1048
1034      ROBERTO PEREIRA  1992-05-23  M     C003     D001     1042
1021      JOSE NOGUEIRA    1994-11-10  M     C003     D001     1042
1095      MARIA DA SILVA   1992-09-03  F     C004     D001     1042
```

Comentários:

Os códigos dos departamentos cadastrados possuem como inicial uma letra 'D' maiúscula. Na consulta do exemplo, se tivéssemos escrito `'d001'`, nenhuma linha seria apresentada no resultado, pois para cadeias de caracteres, a comparação considera que uma letra em minúsculas é diferente dessa mesma letra em maiúsculas. No jargão da computação dizemos que é uma comparação *case sensitive* (sensível a maiúsculas e minúsculas). Como regra geral, considere que qualquer conteúdo em forma de texto que apareça entre aspas simples em um comando SQL é *case sensitive*.

✓ *Exemplo 4b:* Mostrando o número de matrícula e nome de todas as funcionárias.

```
SELECT NrMatric, NmFunc
      FROM Funcionarios
      WHERE Sexo = 'F' ;
```

Resultado:

```
NRMATRIC  NMFUNC
--------  ---------------
1095      MARIA DA SILVA
1029      RUTH DE SOUZA
1048      ANA SILVEIRA
```

✓ *Exemplo 4c:* Listando os funcionários dos cargos C001, C003 e C005.

```
SELECT *
      FROM Funcionarios
      WHERE CdDepto = 'D002' OR CdCargo <> 'C001' AND Sexo = 'F' ;
```

Resultado:

```
NRMATRIC  NMFUNC           DTADM       SEXO  CDCARGO  CDDEPTO  CDSUPERV
--------  ---------------  ----------  ----  -------  -------  --------
1048      ANA SILVEIRA     1993-06-01  F     C005
1001      JOAO SAMPAIO     1993-08-10  M     C002     D002     1048
1004      LUCIO TORRES     1994-03-02  M     C002     D002     1001
1095      MARIA DA SILVA   1992-09-03  F     C004     D001     1042
1023      LUIZ DE ALMEIDA  1993-01-12  M     C002     D002     1001
1015      PAULO RODRIGUES  1992-08-17  M     C002     D002     1001
```

Comentários:

Podemos ter várias comparações na cláusula `WHERE`, que devem ser conectadas por meio dos operadores lógicos `AND` e `OR`, conforme a necessidade. Não é permitido especificar mais de uma cláusula `WHERE` por comando, nem separar as condições por vírgula, já que essa construção não produziria um valor lógico. Condições conectadas por meio de `AND` são avaliadas antes das condições conectadas com `OR`, o que significa que a consulta do exemplo é equivalente a

```
SELECT *
    FROM Funcionarios
    WHERE (CdDepto = 'D002') OR
          (CdCargo <> 'C001' AND Sexo = 'F');
```

✓ *Exemplo 4d:* Projetando a matrícula, o nome e o código do cargo para os funcionários que possuem a substring 'EIR' em qualquer parte do nome (operador `LIKE`).

```
SELECT NrMatric, NmFunc, CdCargo
      FROM Funcionarios
      WHERE NmFunc LIKE '%EIR%' ;
```

Resultado:

```
NRMATRIC  NMFUNC           CDCARGO
--------  ---------------  -------
1048      ANA SILVEIRA     C005
1042      PEDRO PINHEIRO   C004
1034      ROBERTO PEREIRA  C003
1021      JOSE NOGUEIRA    C003
```

Comentários:

O operador `LIKE` permite comparar uma string com um formato definido por meio de constantes e curingas. Existem 2 curingas: o %, que significa 'qualquer combinação de caracteres' e o *underscore* '_', que significa 'exatamente um caracter qualquer'. No exemplo apresentado, compusemos uma máscara '%EIR%', que significa 'qualquer combinação de caracteres, seguido da cadeia "EIR" e terminando com qualquer combinação de caracteres'.

✓ *Exemplo 4e:* Projetando os nomes dos funcionários que possuem uma letra 'A' como terceiro caracter do nome.

```
SELECT NmFunc
      FROM Funcionarios
      WHERE NmFunc LIKE '__A%' ;
```

Resultado:

```
NMFUNC
---------------
ANA SILVEIRA
JOAO SAMPAIO
```

Comentários:

A máscara que desejamos é 'qualquer caracter na primeira posição, qualquer caracter na segunda posição, uma letra A na terceira posição e depois, qualquer combinação de caracteres'.

✓ *Exemplo 4f:* Listando os cargos com salário entre $450 e $1000 (operador `BETWEEN`).

```
SELECT *
      FROM Cargos
      WHERE VrSalario BETWEEN 450 AND 1000 ;
```

Resultado:

```
CDCARGO   NMCARGO          VRSALARIO
--------  ---------------  ----------
C003      AUX. ESCRITORIO  450.00
C006      VIGIA            500.00
C002      MECANICO         750.00
C004      ESCRITURARIO     600.00
```

Comentários:

O operador `BETWEEN` permite verificar se um dados está dentro de um intervalo, sendo geralmente utilizado para o tratamento de números e datas. A estrutura do operador é `BETWEEN <limite inferior> AND <limite superior>`, e a expressão resulta em verdadeiro para valores que sejam iguais ou maiores que `<limite inferior>` e menores ou iguais a `<limite superior>`, o que significa que a consulta do exemplo é equivalente a

```
SELECT *
   FROM Cargos
   WHERE VrSalario >= 450 AND VrSalario <= 1000 ;
```

✓ *Exemplo 4g:* Listando os funcionários dos cargos C001, C003 e C005 (operador `IN`).

```
SELECT *
      FROM Funcionarios
      WHERE CdCargo IN ( 'C001', 'C003', 'C005' ) ;
```

Resultado:

```
NRMATRIC  NMFUNC           DTADM       SEXO  CDCARGO  CDDEPTO  CDSUPERV
--------  ---------------  ----------  ----  -------  -------  --------
1048      ANA SILVEIRA     1993-06-01  F     C005
1034      ROBERTO PEREIRA  1992-05-23  M     C003     D001     1042
1021      JOSE NOGUEIRA    1994-11-10  M     C003     D001     1042
1029      RUTH DE SOUZA    1992-11-05  F     C001     D003     1042
```

Comentários:

O operador `IN` permite verificar se um dado pertence a um conjunto de valores, e é uma alternativa mais prática e elegante para situações em que precisamos testar se um dado é igual a qualquer um de diversos valores, formando uma estrutura da forma `<dado> = <valor1> OR <dado> = <valor2> OR ... <dado> = <valorN>`. Isso significa que a consulta do exemplo é equivalente a

```
SELECT *
   FROM Funcionarios
   WHERE CdCargo = 'C001' OR CdCargo = 'C003'
         OR CdCargo = 'C005';
```

Uma outra situação em que o operador `IN` é importante ocorre na construção de subconsultas, como será discutido posteriormente.

5. Exibindo linhas com ou sem repetição de dados (modificadores `ALL` | `DISTINCT`).

✓ *Exemplo 5a:* Obtendo o código de cargo de cada funcionário.

```
SELECT ALL CdCargo
      FROM Funcionarios ;
```

ou

```
SELECT CdCargo
      FROM Funcionarios ;
```

Resultado:

```
CDCARGO
-------
C005
C002
C002
C004
C003
C003
C001
C004
C002
C002
```

Comentários:

`CdCargo` é uma coluna que possui valores repetidos, pois para alguns cargos existem dois ou mais funcionários cadastrados. Para preservar essas repetições no resultado, devemos utilizar a cláusula `ALL` após a palavra reservada `SELECT` ou, simplesmente, não indicar nem `ALL` nem `DISTINCT` pois, nesse caso, o gerenciador assumirá automaticamente o padrão `ALL`.

✓ *Exemplo 5b:* Obtendo os códigos dos cargos que possuem funcionários.

```
SELECT DISTINCT CdCargo
      FROM Funcionarios ;
```

Resultado:

```
CDCARGO
-------
C001
C002
C003
C004
C005
```

Comentários:

Essa consulta é bastante semelhante à anterior, exceto que não precisamos, nem desejamos, que cargos com mais de um funcionário apareçam repetidamente no resultado. Nesse caso, queremos descartar as repetições, de maneira que apenas uma linha para cada cargo com funcionários seja mostrada e, para isso, é necessário especificar a cláusula `DISTINCT` imediatamente após a palavra reservada `SELECT`. Observe também que os dados agora aparecem ordenados.

- ✓ *Exemplo 5c:* Listando os códigos de cargo e de departamento dos funcionários que não são do departamento D001, mantendo as possíveis repetições de valores no resultado.

```
SELECT CdCargo, CdDepto
      FROM Funcionarios
      WHERE CdDepto <> 'D001' ;
```

ou

```
SELECT ALL CdCargo, CdDepto
      FROM Funcionarios
      WHERE CdDepto <> 'D001' ;
```

Resultado:

```
CDCARGO  CDDEPTO
-------  -------
C002     D002
C002     D002
C001     D003
C002     D002
C002     D002
```

Comentários:

Em nosso banco de dados existe uma funcionária que não possui departamento, e para ela, a coluna `CdDepto` possui o valor nulo. Essa linha não aparece em nosso resultado, pois ao comparar (`NULL <> 'D001'`), o resultado não dá nem verdadeiro nem falso, dá nulo. Para que essa linha também aparecesse no resultado, teríamos que escrever a seguinte consulta:

```
SELECT CdCargo, CdDepto
  FROM Funcionarios
  WHERE CdDepto <> 'D001' OR CdDepto IS NULL ;
```

Resultado:

```
CDCARGO  CDDEPTO
-------  -------
C005
C002     D002
C002     D002
C001     D003
C002     D002
C002     D002
```

- ✓ *Exemplo 5d:* Listando os códigos de cargo e de departamento dos funcionários que não são do departamento D001, eliminando as possíveis repetições de valores no resultado.

```
SELECT DISTINCT CdCargo, CdDepto
      FROM Funcionarios
      WHERE CdDepto <> 'D001' ;
```

Resultado:

```
CDCARGO  CDDEPTO
-------  -------
C001     D003
C002     D002
```

Comentários:

Da mesma forma que ocorreu no exemplo anterior, temos a situação em que uma das linhas na tabela de funcionários possui um nulo na coluna `CdDepto`. Se quisermos que essa linha também seja considerada no resultado, precisamos complementar a cláusula `WHERE` do comando para prever essa possibilidade:

```
SELECT DISTINCT CdCargo, CdDepto
    FROM Funcionarios
    WHERE CdDepto <> 'D001' OR CdDepto IS NULL ;
```

Resultado:

```
CDCARGO  CDDEPTO
-------  -------
C001     D003
C002     D002
C005
```

6. Renomeando colunas no resultado da consulta (operador `AS`).

✓ *Exemplo 6a:* Exibindo para cada funcionário o seu nome, matrícula e data de admissão, modificando o nome das colunas no resultado final.

```
SELECT NmFunc AS Nome, NrMatric AS Matricula, DtAdm AS Admissao
      FROM Funcionarios ;
```

Resultado:

```
Nome             Matricula  Admissao
---------------  ---------  ----------
ANA SILVEIRA     1048       1993-06-01
JOAO SAMPAIO     1001       1993-08-10
LUCIO TORRES     1004       1994-03-02
PEDRO PINHEIRO   1042       1994-07-29
ROBERTO PEREIRA  1034       1992-05-23
JOSE NOGUEIRA    1021       1994-11-10
RUTH DE SOUZA    1029       1992-11-05
MARIA DA SILVA   1095       1992-09-03
LUIZ DE ALMEIDA  1023       1993-01-12
PAULO RODRIGUES  1015       1992-08-17
```

Comentários:

O cabeçalho das colunas no resultado final foi modificado, conforme o especificado na cláusula `AS` do comando `SELECT`. Esse recurso é interessante para melhorar a clareza na identificação das colunas em um relatório, ou para atribuir um nome significativo para uma coluna virtual (que foi calculada no comando) e pode ser especialmente importante em algumas situações envolvendo programação de aplicações. No Oracle existe uma alternativa para a cláusula AS, que consiste em colocar o novo nome da coluna entre aspas duplas, o que faria o comando do exemplo ser escrito da seguinte forma:

```
SELECT NmFunc "Nome", NrMatric "Matricula", DtAdm "Admissao"
  FROM Funcionarios ;
```

Esse recurso permite inclusive que o novo nome possua espaços em branco. Entretanto, como regra prática, desaconselha-se o seu uso, visto que não faz parte do padrão SQL.

7. Projetando o resultado de uma função.

✓ *Exemplo 7a:* Exibir, para cada funcionário, o seu nome, matrícula e data de admissão no formato <Dia>/<Iniciais do Mês>/<Ano com dois dígitos>.

```
SELECT NmFunc, NrMatric, TO_CHAR(DtAdm, 'DD/MON/YY')
      FROM Funcionarios ;
```

Resultado:

```
NMFUNC           NRMATRIC   TO_CHAR(DD
---------------  ---------  ----------
ANA SILVEIRA     1048       01/JUN/93
JOAO SAMPAIO     1001       10/AUG/93
LUCIO TORRES     1004       02/MAR/94
PEDRO PINHEIRO   1042       29/JUL/94
ROBERTO PEREIRA  1034       23/MAY/92
JOSE NOGUEIRA    1021       10/NOV/94
RUTH DE SOUZA    1029       05/NOV/92
MARIA DA SILVA   1095       03/SEP/92
LUIZ DE ALMEIDA  1023       12/JAN/93
PAULO RODRIGUES  1015       17/AUG/92
```

Comentários:

Existem várias funções pré-definidas para uso em comandos SQL com o Oracle e uma relação delas é apresentada posteriormente. Também é possível criar uma nova função, utilizando os recursos de programação do SGBD.

8. Exibindo os dados de uma tabela em uma determinada ordem (cláusula `ORDER BY`).

✓ *Exemplo 8a:* Obtendo as linhas da tabela de funcionários em ordem crescente de número de matrícula.

```
SELECT *
      FROM Funcionarios
      ORDER BY NrMatric ;
```

Resultado:

```
NRMATRIC  NMFUNC           DTADM       SEXO  CDCARGO  CDDEPTO  CDSUPERV
--------  ---------------  ----------  ----  -------  -------  --------
1001      JOAO SAMPAIO     1993-08-10  M     C002     D002
1004      LUCIO TORRES     1994-03-02  M     C002     D002     1001
1015      PAULO RODRIGUES  1992-08-17  M     C002     D002     1001
1021      JOSE NOGUEIRA    1994-11-10  M     C003     D001     1042
1023      LUIZ DE ALMEIDA  1993-01-12  M     C002     D002     1001
1029      RUTH DE SOUZA    1992-11-05  F     C001     D003     1042
1034      ROBERTO PEREIRA  1992-05-23  M     C003     D001     1042
1042      PEDRO PINHEIRO   1994-07-29  M     C004     D001     1048
1048      ANA SILVEIRA     1993-06-01  F     C005
1095      MARIA DA SILVA   1992-09-03  F     C004     D001     1048
```

Comentários:

Os dados apareceram ordenados conforme o número de matrícula, do menor para o maior, ou seja, em ordem crescente (ou ascendente). O efeito seria o mesmo se tivéssemos especificado explicitamente o sentido de ordenação com a cláusula opcional `ASC`, como mostra o comando a seguir. Se uma cláusula `ORDER BY` não tem para uma coluna a indicação de sentido, é assumida como padrão a ordem ascendente.

```
SELECT *
    FROM Funcionarios
    ORDER BY NrMatric ASC ;
```

✓ *Exemplo 8b:* Exibindo os dados de funcionários ordenados pelo valor da segunda coluna projetada pelo comando.

```
SELECT *
      FROM Funcionarios
      ORDER BY 2 ;
```

Resultado:

```
NRMATRIC  NMFUNC           DTADM       SEXO  CDCARGO  CDDEPTO  CDSUPERV
--------  ---------------  ----------  ----  -------  -------  --------
1048      ANA SILVEIRA     1993-06-01  F     C005
1001      JOAO SAMPAIO     1993-08-10  M     C002     D002     1048
1021      JOSE NOGUEIRA    1994-11-10  M     C003     D001     1042
1004      LUCIO TORRES     1994-03-02  M     C002     D002     1001
1023      LUIZ DE ALMEIDA  1993-01-12  M     C002     D002     1001
1095      MARIA DA SILVA   1992-09-03  F     C004     D001     1042
1015      PAULO RODRIGUES  1992-08-17  M     C002     D002     1001
1042      PEDRO PINHEIRO   1994-07-29  M     C004     D001     1048
1034      ROBERTO PEREIRA  1992-05-23  M     C003     D001     1042
1029      RUTH DE SOUZA    1992-11-05  F     C001     D003     1042
```

Comentários:

O resultado mostra os funcionários em ordem ascendente de nome, que é a segunda coluna da tabela. Este exemplo mostra que é possível indicar uma coluna como critério de ordenação não por seu nome, mas pela sua posição na lista de projeção que antecede à cláusula `FROM` do comando. Esse recurso pode ser útil para algumas colunas que são calculadas durante a consulta.

✓ *Exemplo 8c:* Exibindo os cargos com salário igual ou superior a $500, em ordem crescente de salário.

```
SELECT *
      FROM Cargos
      WHERE VrSalario >= 500
      ORDER BY VrSalario ;
```

Resultado:

```
CDCARGO   NMCARGO            VRSALARIO
--------  ---------------   ----------
C006      VIGIA                    500
C004      ESCRITURARIO             600
C002      MECANICO                 750
C005      GERENTE                 2300
```

Comentários:

Os dados que são selecionados pela cláusula `WHERE` do comando são depois ordenados conforme explicitado em `ORDER BY` para então serem projetados no resultado.

✓ *Exemplo 8d:* Exibindo o nome, data de admissão e número de matrícula de cada funcionário, em ordem decrescente de nome.

```
SELECT NmFunc, DtAdm, NrMatric
      FROM Funcionarios
      ORDER BY NmFunc DESC ;
```

Resultado:

```
NMFUNC          DTADM       NRMATRIC
--------------- ----------  --------
RUTH DE SOUZA   1992-11-05  1029
ROBERTO PEREIRA 1992-05-23  1034
PEDRO PINHEIRO  1994-07-29  1042
PAULO RODRIGUES 1992-08-17  1015
MARIA DA SILVA  1992-09-03  1095
LUIZ DE ALMEIDA 1993-01-12  1023
LUCIO TORRES    1994-03-02  1004
JOSE NOGUEIRA   1994-11-10  1021
JOAO SAMPAIO    1993-08-10  1001
ANA SILVEIRA    1993-06-01  1048
```

Comentários:

Para ordenações em sentido decrescente (descendente) de uma coluna, é obrigatório utilizar a cláusula `DESC` após a indicação da coluna na cláusula `ORDER BY`.

✓ *Exemplo 8e:* Listando os funcionários em ordem decrescente de cargo e, para aqueles funcionários de mesmo cargo, ordenando-os por nome.

```
SELECT *
      FROM Funcionarios
      ORDER BY CdCargo DESC, NmFunc ;
```

Resultado:

```
NRMATRIC  NMFUNC          DTADM       SEXO  CDCARGO  CDDEPTO  CDSUPERV
--------  --------------- ----------  ----  -------  -------  --------
1048      ANA SILVEIRA    1993-06-01  F     C005
1095      MARIA DA SILVA  1992-09-03  F     C004     D001     1042
1042      PEDRO PINHEIRO  1994-07-29  M     C004     D001     1048
1021      JOSE NOGUEIRA   1994-11-10  M     C003     D001     1042
1034      ROBERTO PEREIRA 1992-05-23  M     C003     D001     1042
1001      JOAO SAMPAIO    1993-08-10  M     C002     D002
1004      LUCIO TORRES    1994-03-02  M     C002     D002     1001
1023      LUIZ DE ALMEIDA 1993-01-12  M     C002     D002     1001
1015      PAULO RODRIGUES 1992-08-17  M     C002     D002     1001
1029      RUTH DE SOUZA   1992-11-05  F     C001     D003     1042
```

Comentários:

Uma consulta pode possuir diversos critérios de ordenação. Tipicamente essa situação ocorre quando o critério principal de ordenação é uma coluna que possui valores repetidos, como em nosso caso, a coluna `CdCargo`. Um critério secundário pode então ser fornecido no comando para determinar como organizar as linhas

com valor igual no primeiro critério. Vários critérios de ordenação são possíveis e cada um deles pode ser ou crescente ou descendente.

- ✓ *Exemplo 8f:* Listando os funcionários em ordem de departamento e, para aqueles funcionários do mesmo departamento, ordenando-os por cargo, e para aqueles de mesmo departamento e cargo, organizando-os então por nome.

```
SELECT *
      FROM Funcionarios
      ORDER BY CdDepto, CdCargo, NmFunc ;
```

Resultado:

```
NRMATRIC  NMFUNC           DTADM       SEXO  CDCARGO  CDDEPTO  CDSUPERV
--------  ---------------  ----------  ----  -------  -------  --------
1021      JOSE NOGUEIRA    1994-11-10  M     C003     D001     1042
1034      ROBERTO PEREIRA  1992-05-23  M     C003     D001     1042
1095      MARIA DA SILVA   1992-09-03  F     C004     D001     1042
1042      PEDRO PINHEIRO   1994-07-29  M     C004     D001     1048
1001      JOAO SAMPAIO     1993-08-10  M     C002     D002     1048
1004      LUCIO TORRES     1994-03-02  M     C002     D002     1001
1023      LUIZ DE ALMEIDA  1993-01-12  M     C002     D002     1001
1015      PAULO RODRIGUES  1992-08-17  M     C002     D002     1001
1029      RUTH DE SOUZA    1992-11-05  F     C001     D003     1042
1048      ANA SILVEIRA     1993-06-01  F     C005
```

Comentários:

A coluna `CdDepto` que é o critério principal de ordenação da consulta possui um nulo em uma das linhas. A forma como os nulos são tratados na ordenação pode variar entre dois SGBDs diferentes. No caso do Oracle e do PostgreSQL, por exemplo, os nulos são considerados como o maior valor da coluna, enquanto que no MS-Access, eles são considerados como o menor valor da coluna.

9. Fazendo totalizações simples (uso de funções de agregação SQL).

✓ *Exemplo 9a:* Determinando quantas linhas existem na tabela de cargos.

```
SELECT COUNT( * )
      FROM Cargos ;
```

Resultado:

```
   COUNT
--------
       6
```

Comentários:

A função `COUNT( * )` faz a contagem de linhas de um conjunto de dados.

✓ *Exemplo 9b:* Determinando quantas linhas existem na tabela de cargos com salário superior a $500.

```
SELECT COUNT( * )
      FROM Cargos
      WHERE VrSalario > 500 ;
```

Resultado:

```
   COUNT
--------
       3
```

Comentários:

Na consulta do exemplo, a cláusula `WHERE` é avaliada primeiro e a contagem é feita sobre o conjunto de linhas que atenderam à sua condição. Portanto, temos no resultado quantas linhas possuem um valor maior que 500 na coluna `VrSalario` da tabela de cargos.

✓ *Exemplo 9c:* Contando quantos funcionários possuem departamento.

```
SELECT COUNT( CdDepto )
      FROM Funcionarios ;
```

Resultado:

```
   COUNT
--------
       9
```

Comentários:

Caso informemos para a função `COUNT` um nome de coluna, ela contará quantas linhas possuem valor não nulo nessa coluna. Em nosso exemplo, temos 10 linhas na tabela de funcionários, mas como uma delas possui nulo na coluna `CdDepto`, o resultado de `COUNT( CdDepto )` retorna 9.

✓ *Exemplo 9d:* Contando quantos departamentos diferentes possuem funcionários.

```
SELECT COUNT( DISTINCT CdDepto )
      FROM Funcionarios ;
```

Resultado:

```
   COUNT
--------
       3
```

Comentários:

Especificar DISTINCT para uma coluna informada em COUNT leva à contagem de quantas linhas possuem valores distintos nessa coluna.

✓ *Exemplo 9e:* Descobrindo os valores do maior e do menor salários cadastrados.

```
SELECT MAX( VrSalario ) AS Maior, MIN( VrSalario ) As Menor
      FROM Cargos ;
```

Resultado:

```
   Maior     Menor
-------- --------
    2300       350
```

Comentários:

A função MAX determina o maior valor existente em uma coluna, e MIN determina o menor valor. Se a coluna utilizada com essas funções possuir nulos em algumas linhas, essas linhas serão ignoradas. Observe também que podemos projetar o resultado de mais de uma função de agregação para o mesmo conjunto de dados.

✓ *Exemplo 9f:* Produzindo a soma e a média dos salários cadastrados na tabela de cargos.

```
SELECT SUM( VrSalario ) AS Somatorio, AVG( VrSalario ) As Media
      FROM Cargos ;
```

Resultado:

```
 Somatorio    Media
---------- --------
      4950      825
```

Comentários:

A função SUM faz a soma do valor existente em uma coluna, e AVG calcula a média dos valores de coluna. Para obter a média, poderíamos obter o mesmo resultado de AVG utilizando as funções SUM e COUNT, como mostra o exemplo a seguir. Apesar dessa possibilidade, é melhor usar AVG.

```
SELECT SUM( VrSalario ) / COUNT( VrSalario ) As Media
   FROM Cargos ;
```

✓ *Exemplo 9g:* Calculando a média dos salários cadastrados na tabela de cargos, supondo um aumento de 10% nos valores.

```
SELECT AVG( VrSalario * 1.1 ) As Media
      FROM Cargos ;
```

Resultado:

```
     Media
--------
     907.5
```

Comentários:

A expressão utilizada como parâmetro para uma função de agregação pode ser o resultado de um cálculo. No exemplo, a função `AVG` vai fazer seu cálculo considerando o valor existente na coluna `VrSalario` de cada linha multiplicado por 1.1.

10. Produzindo totais com agrupamentos de linhas (cláusulas GROUP BY e HAVING).

✓ *Exemplo 10a:* Exibindo os códigos de cargo e a respectiva quantidade de funcionários.

```
SELECT CdCargo, COUNT(CdCargo) AS Qtde
      FROM Funcionarios
      GROUP BY CdCargo ;
```

Resultado:

```
CDCARGO  Qtde
-------  ----
C001        1
C002        4
C005        1
C003        2
C004        2
```

Comentários:

A cláusula GROUP BY especifica uma coluna que será usada como critério de agrupamento das linhas. Em nosso exemplo, as linhas da tabela de funcionários serão separadas em conjuntos conforme o valor da coluna CdCargo. Como temos funcionários de 5 cargos diferentes, serão formados 5 conjuntos de linhas, ou agrupamentos. Em um agrupamento estarão as linhas com CdCargo igual a 'C001', em outro agrupamento estarão as linhas de funcionários com CdCargo igual a 'C002', e assim por diante. Depois de formados os agrupamentos, projeta-se para cada um o valor da coluna CdCargo e o resultado obtido pela aplicação da função COUNT(CdCargo) sobre suas linhas.

✓ *Exemplo 10b:* Listando os códigos de departamento que possuem mulheres e quantas funcionárias cada um possui.

```
SELECT CdDepto, COUNT(*) AS Qtde
      FROM Funcionarios
      WHERE Sexo = 'F'
      GROUP BY CdDepto ;
```

Resultado:

```
CDDEPTO  Qtde
-------  ----
D001        1
D003        1
            1
```

Comentários:

A cláusula GROUP BY é executada após a avaliação da cláusula WHERE. Isso quer dizer que apenas as linhas que atendem à condição especificada na cláusula WHERE é que serão agrupadas.

- ✓ *Exemplo 10c:* Exibindo, para cada cargo com pelo menos 2 funcionários, o respectivo código e quantidade de funcionários.

```
SELECT CdCargo, COUNT(CdCargo)
      FROM Funcionarios
      GROUP BY CdCargo
      HAVING COUNT(CdCargo) >= 2;
```

Resultado:

```
CDCARGO  COUNT
-------  -----
C002         4
C003         2
C004         2
```

Comentários:

A cláusula `HAVING` funciona como um filtro que é aplicado sobre os agrupamentos, e por isso é executada após o `GROUP BY`. No exemplo apresentado, as linhas da tabela de funcionários são agrupadas conforme o valor da coluna `CdCargo` e, em seguida, para cada agrupamento é aplicada a condição especificada na cláusula `HAVING`. Se a condição resultar em verdadeiro para um grupo, ele será mantido no resultado final da consulta, se resultar em falso, ele será eliminado. No caso, os agrupamentos que tiverem menos de 2 linhas serão descartados. Não poderíamos fazer essa filtragem na cláusula `WHERE` (a menos que utilizássemos subconsultas) por que só podemos saber quantos funcionários existem para cada cargo após o agrupamento ter sido realizado. Como regra geral, assuma que sempre que uma condição precisar envolver a utilização de uma função de agregação (`COUNT`, `SUM`, `AVG`, `MAX`, etc), ela precisará ser definida na cláusula `HAVING` do comando, e não na cláusula `WHERE`.

✓ *Exemplo 10d:* Exibindo os códigos dos departamentos que possuem pelo menos 2 funcionários homens. Os dados devem ser apresentados em ordem alfabética.

```
SELECT CdDepto
      FROM Funcionarios
      WHERE Sexo = 'M'
      GROUP BY CdDepto
      HAVING COUNT( * ) > 1
      ORDER BY CdDepto ;
```

Resultado:

```
CDDEPTO
-------
D001
D002
```

Comentários:

Nesse exemplo devemos entender que após os dados originais serem lidos da tabela de funcionários, a condição da cláusula `WHERE` é aplicada, descartando as linhas que não tiverem na coluna `Sexo` o valor 'M'. Em seguida o agrupamento pelo código do departamento é realizado e então a condição especificada na cláusula `HAVING` é avaliada, descartando os agrupamentos que tiverem menos de 2 linhas. Por fim, o resultado é ordenado conforme o indicado em `ORDER BY` e só então os valores solicitados são projetados.

11. Utilizando aliases (variáveis tupla) para tabelas.

✓ *Exemplo 11a:* Exibindo a matrícula, o nome e a data de admissão dos funcionários que não são do departamento D002.

```
SELECT NrMatric, NmFunc, DtAdm
      FROM Funcionarios
      WHERE CdDepto <> 'D002 ' OR CdDepto IS NULL ;
```

Resultado:

```
NRMATRIC  NMFUNC              DTADM
--------  ------------------  ------------
1048      ANA SILVEIRA        1993-06-01
1042      PEDRO PINHEIRO      1994-07-29
1034      ROBERTO PEREIRA     1992-05-23
1021      JOSE NOGUEIRA       1994-11-10
1029      RUTH DE SOUZA       1992-11-05
1095      MARIA DA SILVA      1992-09-03
```

Comentários:

Em um comando SQL, podemos indicar uma coluna apenas pelo seu nome, ficando subentendida em qual tabela a coluna se encontra. Isso é, sem dúvida, válido para consultas simples que envolvem uma tabela só, mas, a rigor, deveríamos sempre indicar explicitamente a localização da coluna na tabela, na forma `<tabela>.<coluna>`, como é feito no comando a seguir:

```
SELECT Funcionarios.NrMatric, Funcionarios.NmFunc,
       Funcionarios.DtAdm
   FROM Funcionarios
   WHERE Funcionarios.CdDepto <> 'D002 ' ;
```

Esse tipo de detalhamento pode ser essencial em consultas que envolvem mais de uma tabela (junções), como veremos posteriormente. É possível atribuir um apelido para uma tabela dentro da consulta, recurso que pode tornar a escrita de alguns comandos mais curta e, em outros, tornar mais claro o papel de uma tabela. Em consultas que envolvem auto-relacionamentos, por exemplo, esse recurso é imprescindível. Para atribuirmos um apelido para uma tabela, basta indicá-lo após o nome desta, na cláusula `FROM` do comando `SELECT`. Esse apelido deverá então ser utilizado em todos os lugares em que a tabela precise ser referenciada no comando, como mostra o exemplo a seguir:

```
SELECT a.NrMatric, a.NmFunc, a.DtAdm
   FROM Funcionarios a
   WHERE a.CdDepto <> 'D002 ' ;
```

12. Buscando dados em mais de uma tabela (junções).

✓ *Exemplo 12a:* Listando os dados dos funcionários juntamente com os dados do respectivo cargo.

```
SELECT *
      FROM Funcionarios, Cargos
      WHERE Funcionarios.CdCargo = Cargos.CdCargo ;
```

Resultado:

```
NRMA NMFUNC          DTADM      SEXO CDCARGO CDDEPTO CDSUPERV CDCARGO NMCARGO         VRSALARIO
---- --------------- ---------- ---- ------- ------- -------- ------- --------------- ---------
1048 ANA SILVEIRA    1993-06-01 F    C005                     C005    GERENTE              2300
1001 JOAO SAMPAIO    1993-08-10 M    C002    D002    1048     C002    MECANICO              750
1004 LUCIO TORRES    1994-03-02 M    C002    D002    1001     C002    MECANICO              750
1042 PEDRO PINHEIRO  1994-07-29 M    C004    D001    1048     C004    ESCRITURARIO          600
1034 ROBERTO PEREIRA 1992-05-23 M    C003    D001    1042     C003    AUX. ESCRITORIO       450
1021 JOSE NOGUEIRA   1994-11-10 M    C003    D001    1042     C003    AUX. ESCRITORIO       450
1029 RUTH DE SOUZA   1992-11-05 F    C001    D003    1042     C001    COZINHEIRA            350
1095 MARIA DA SILVA  1992-09-03 F    C004    D001    1042     C004    ESCRITURARIO          600
1023 LUIZ DE ALMEIDA 1993-01-12 M    C002    D002    1001     C002    MECANICO              750
1015 PAULO RODRIGUES 1992-08-17 M    C002    D002    1001     C002    MECANICO              750
```

Comentários:

Para exibir os dados combinados de duas ou mais tabelas, devemos especificar os nomes dessas tabelas na cláusula `FROM` do comando. Com isso, serão geradas na memória todas as combinações possíveis entre as linhas dessas tabelas, chamada de produto cartesiano. Para manter apenas as combinações de linhas válidas a partir do produto cartesiano, é necessário especificar uma condição de filtragem de linhas. Essa condição é chamada de equação de junção, e pode ser especificada na cláusula `WHERE` do comando, como foi feito no exemplo, ou poderá ser inserida em um operador específico de junção, que será mostrado no exemplo *12b*. Antes, para ilustrar melhor alguns conceitos, podemos gerar o produto cartesiano das tabelas `Funcionarios` e `Cargos` através do comando

```
SELECT *
   FROM Funcionarios, Cargos ;
```

Veja que todas as combinações possíveis entre linhas de `Funcionarios` e de `Cargos` são produzidas por esse `SELECT`. Como existem 10 funcionários cadastrados e 6 linhas na tabela de cargos, o resultado é um conjunto de 60 linhas. A maioria dessas linhas combina um funcionário com um cargo que não é o seu e, para eliminar essas combinações indesejadas do resultado, seria necessário descartar as linhas onde o valor da coluna `CdCargo` vinda da tabela `Funcionarios` (que podemos escrever na forma `Funcionarios.CdCargo`) seja diferente do valor da coluna `CdCargo` originária da tabela `Cargos` (que indicamos escrevendo `Cargos.CdCargo`) como foi feito na cláusula `WHERE` do comando apresentado inicialmente neste exemplo. Em geral a forma de uma equação de junção é <chave estrangeira de uma tabela> = <chave primária da outra tabela>.

✓ *Exemplo 12b:* Listando os dados dos funcionários juntamente com os dados do respectivo cargo (utilizando operador de junção interna, `INNER JOIN`).

```
SELECT *
      FROM Funcionarios
         INNER JOIN Cargos ON Funcionarios.CdCargo = Cargos.CdCargo;
```

Resultado:

NRMA	NMFUNC	DTADM	SEXO	CDCARGO	CDDEPTO	CDSUPERV	CDCARGO	NMCARGO	VRSALARIO
1048	ANA SILVEIRA	1993-06-01	F	C005			C005	GERENTE	2300
1001	JOAO SAMPAIO	1993-08-10	M	C002	D002	1048	C002	MECANICO	750
1004	LUCIO TORRES	1994-03-02	M	C002	D002	1001	C002	MECANICO	750
1042	PEDRO PINHEIRO	1994-07-29	M	C004	D001	1048	C004	ESCRITURARIO	600
1034	ROBERTO PEREIRA	1992-05-23	M	C003	D001	1042	C003	AUX. ESCRITORIO	450
1021	JOSE NOGUEIRA	1994-11-10	M	C003	D001	1042	C003	AUX. ESCRITORIO	450
1029	RUTH DE SOUZA	1992-11-05	F	C001	D003	1042	C001	COZINHEIRA	350
1095	MARIA DA SILVA	1992-09-03	F	C004	D001	1042	C004	ESCRITURARIO	600
1023	LUIZ DE ALMEIDA	1993-01-12	M	C002	D002	1001	C002	MECANICO	750
1015	PAULO RODRIGUES	1992-08-17	M	C002	D002	1001	C002	MECANICO	750

Comentários:

O comando apresentado produz exatamente o mesmo resultado que o do exemplo *12a*. `INNER JOIN` é chamado de operador de junção interna, ou de junção equivalente (*equi-join*), e permite recuperar os dados relacionados entre duas tabelas. É aconselhável utiliza-lo em vez de tratar a junção na cláusula `WHERE`, como foi feito anteriormente, pois isso torna comandos `SELECT` extensos mais organizados pela separação formal entre as condições de filtragem comuns, colocadas em `WHERE`, das equações de junção, especificadas no operador `INNER JOIN`. O operador `INNER JOIN` está disponível no Oracle a partir de sua versão 9.2

✓ *Exemplo 12c:* Listando os dados dos funcionários juntamente com os dados do respectivo departamento.

```
SELECT *
      FROM Funcionarios
         INNER JOIN Deptos ON Funcionarios.CdDepto = Deptos.CdDepto;
```

ou

```
SELECT *
      FROM Funcionarios, Deptos
      WHERE Funcionarios.CdDepto = Deptos.CdDepto ;
```

Resultado:

NRMA	NMFUNC	DTADM	SEXO	CDCARGO	CDDEPTO	CDSUPERV	CDDEPTO	NMDEPTO	RAMAL
1001	JOAO SAMPAIO	1993-08-10	M	C002	D002		D002	OFICINA	310
1004	LUCIO TORRES	1994-03-02	M	C002	D002	1001	D002	OFICINA	310
1042	PEDRO PINHEIRO	1994-07-29	M	C004	D001	1048	D001	ADMINISTRACAO	320
1034	ROBERTO PEREIRA	1992-05-23	M	C003	D001	1042	D001	ADMINISTRACAO	320
1021	JOSE NOGUEIRA	1994-11-10	M	C003	D001	1042	D001	ADMINISTRACAO	320
1029	RUTH DE SOUZA	1992-11-05	F	C001	D003	1042	D003	SERVICOS GERAIS	330
1095	MARIA DA SILVA	1992-09-03	F	C004	D001	1042	D001	ADMINISTRACAO	320
1023	LUIZ DE ALMEIDA	1993-01-12	M	C002	D002	1001	D002	OFICINA	310
1015	PAULO RODRIGUES	1992-08-17	M	C002	D002	1001	D002	OFICINA	310

Comentários:

Observe que a junção interna traz apenas as linhas das tabelas envolvidas que possuem correspondência. No nosso caso, existe um funcionário que não possui departamento e, na junção interna de `Funcionarios` e `Deptos`, uma referência a essa linha não aparece. É por esse motivo que o resultado possui apenas 9 linhas, em vez das 10 que apareceram na junção do exemplo anterior.

✓ *Exemplo 12d:* Listando os dados dos funcionários juntamente com os dados do respectivo cargo (utilizando operador de junção natural, `NATURAL JOIN`).

```
SELECT *
      FROM Funcionarios NATURAL JOIN Cargos ;
```

Resultado:

CDCARGO	NRMA	NMFUNC	DTADM	SEXO	CDDEPTO	CDSUPERV	NMCARGO	VRSALARIO
C005	1048	ANA SILVEIRA	1993-06-01	F			GERENTE	2300
C002	1001	JOAO SAMPAIO	1993-08-10	M	D002	1048	MECANICO	750
C002	1004	LUCIO TORRES	1994-03-02	M	D002	1001	MECANICO	750
C004	1042	PEDRO PINHEIRO	1994-07-29	M	D001	1048	ESCRITURARIO	600
C003	1034	ROBERTO PEREIRA	1992-05-23	M	D001	1042	AUX. ESCRITORIO	450
C003	1021	JOSE NOGUEIRA	1994-11-10	M	D001	1042	AUX. ESCRITORIO	450
C001	1029	RUTH DE SOUZA	1992-11-05	F	D003	1042	COZINHEIRA	350
C004	1095	MARIA DA SILVA	1992-09-03	F	D001	1042	ESCRITURARIO	600
C002	1023	LUIZ DE ALMEIDA	1993-01-12	M	D002	1001	MECANICO	750
C002	1015	PAULO RODRIGUES	1992-08-17	M	D002	1001	MECANICO	750

Comentários:

A junção natural é uma junção interna em que o valor do atributo de relacionamento possui o mesmo nome nas duas tabelas (como ocorre nas tabelas de funcionários e de cargos), o valor do atributo de relacionamento aparece uma vez só no resultado caso `*` seja utilizado para indicar os valores a projetar e a equação de junção não é especificada no comando, ficando implícita. Junções naturais são uma construção teórica importante, mas de uso prático questionável, sendo mostrada aqui mais como uma curiosidade. Esse recurso está disponível no Oracle a partir de sua versão 9.2.

✓ *Exemplo 12e:* Obtendo para cada funcionário seu número de matrícula, nome, código e nome do respectivo cargo.

```
SELECT NrMatric, NmFunc, c.CdCargo, NmCargo
      FROM Funcionarios f
             INNER JOIN Cargos c ON f.CdCargo = c.CdCargo
      ORDER BY NmCargo ;
```

Resultado:

```
NRMATRIC   NMFUNC            CDCARGO   NMCARGO
--------   ---------------   -------   ---------------
1034       ROBERTO PEREIRA   C003      AUX. ESCRITORIO
1021       JOSE NOGUEIRA     C003      AUX. ESCRITORIO
1029       RUTH DE SOUZA     C001      COZINHEIRA
1095       MARIA DA SILVA    C004      ESCRITURARIO
1042       PEDRO PINHEIRO    C004      ESCRITURARIO
1048       ANA SILVEIRA      C005      GERENTE
1015       PAULO RODRIGUES   C002      MECANICO
1004       LUCIO TORRES      C002      MECANICO
1023       LUIZ DE ALMEIDA   C002      MECANICO
1001       JOAO SAMPAIO      C002      MECANICO
```

✓ *Exemplo 12f:* Exibindo, para cada funcionário com salário superior a $700, o seu número de matrícula e nome, juntamente com os nomes dos respectivos cargo e departamento (junção interna de 3 tabelas).

```
SELECT NrMatric, NmFunc, NmCargo, NmDepto
      FROM Funcionarios f
            INNER JOIN Cargos c ON f.CdCargo = c.CdCargo
            INNER JOIN Deptos d ON f.CdDepto = d.CdDepto
      WHERE VrSalario > 700 ;
```

Resultado:

```
NRMATRIC NMFUNC            NMCARGO          NMDEPTO
-------- ----------------- ---------------  --------------
1001     JOAO SAMPAIO      MECANICO         OFICINA
1004     LUCIO TORRES      MECANICO         OFICINA
1023     LUIZ DE ALMEIDA   MECANICO         OFICINA
1015     PAULO RODRIGUES   MECANICO         OFICINA
```

Comentários:

Uma junção de três tabelas precisa de duas equações de junção. Assim, indicamos a primeira junção e depois fazemos a junção desse resultado com a tabela restante. Para melhorar a clareza da estrutura e talvez tornar mais fácil seu entendimento, podemos utilizar parênteses, como no comando a seguir, que produz exatamente o mesmo resultado anterior:

```
SELECT NrMatric, NmFunc, NmCargo, NmDepto
    FROM ( Funcionarios f
             INNER JOIN Cargos c ON f.CdCargo = c.CdCargo )
             INNER JOIN Deptos d ON f.CdDepto = d.CdDepto
    WHERE VrSalario > 700 ;
```

- ✓ *Exemplo 12g:* Exibindo o nome, o salário atual e o salário com 10% de aumento para cada funcionário:

```
SELECT NmFunc, VrSalario, VrSalario * 1.1 AS Aumento
      FROM Funcionarios f
            INNER JOIN Cargos c ON f.CdCargo = c.CdCargo ;
```

Resultado:

NMFUNC	VRSALARIO	AUMENTO
ANA SILVEIRA	2300	2530
JOAO SAMPAIO	750	825
LUCIO TORRES	750	825
PEDRO PINHEIRO	600	660
ROBERTO PEREIRA	450	495
JOSE NOGUEIRA	450	495
RUTH DE SOUZA	350	385
MARIA DA SILVA	600	660
LUIZ DE ALMEIDA	750	825
PAULO RODRIGUES	750	825

✓ *Exemplo 12h:* Mostrando os dados de cada funcionário com os dados do respectivo departamento, inclusive para os funcionários que não possuem departamento (junção externa à esquerda).

```
SELECT *
      FROM Funcionarios f
         LEFT OUTER JOIN Deptos d ON f.CdDepto = d.CdDepto ;
```

ou

```
SELECT *
      FROM Funcionarios f
         LEFT JOIN Deptos d ON f.CdDepto = d.CdDepto ;
```

Resultado:

```
NRMA NMFUNC          DTADM      SEXO CDCARGO CDDEPTO CDSUPERV CDDEPTO NMDEPTO         RAMAL
---- --------------- ---------- ---- ------- ------- -------- ------- --------------- ---------
1095 MARIA DA SILVA  1992-09-03 F    C004    D001    1042     D001    ADMINISTRACAO   320
1021 JOSE NOGUEIRA   1994-11-10 M    C003    D001    1042     D001    ADMINISTRACAO   320
1034 ROBERTO PEREIRA 1992-05-23 M    C003    D001    1042     D001    ADMINISTRACAO   320
1042 PEDRO PINHEIRO  1994-07-29 M    C004    D001    1048     D001    ADMINISTRACAO   320
1015 PAULO RODRIGUES 1992-08-17 M    C002    D002    1001     D002    OFICINA         310
1023 LUIZ DE ALMEIDA 1993-01-12 M    C002    D002    1001     D002    OFICINA         310
1004 LUCIO TORRES    1994-03-02 M    C002    D002    1001     D002    OFICINA         310
1001 JOAO SAMPAIO    1993-08-10 M    C002    D002    1048     D002    OFICINA         310
1029 RUTH DE SOUZA   1992-11-05 F    C001    D003    1042     D003    SERVICOS GERAIS 330
1048 ANA SILVEIRA    1993-06-01 F    C005
```

Comentários:

Uma junção externa à esquerda traz os dados de um par de tabelas que estejam relacionados entre si, como faz a junção interna, acrescidos das linhas da primeira tabela da cláusula `FROM` do comando que não possuem correspondente na segunda. Em nosso exemplo, observe que agora todos os funcionários aparecem no resultado e, para o funcionário 1048, que não pertence a nenhum departamento, os dados do departamento aparecem com nulos. O operador `LEFT OUTER JOIN` está disponível no Oracle a partir da versão 9.2 e, nas versões anteriores a única forma de se fazer a junção externa a esquerda era utilizando o modificador `(+)`, que para o nosso exemplo produziria a seguinte consulta:

```
SELECT *
    FROM Funcionarios f, Deptos d
    WHERE f.CdDepto = d.CdDepto(+) ;
```

✓ *Exemplo 12i:* Mostrando os dados de cada funcionário com os dados do respectivo departamento. Exibir inclusive os departamentos que não possuem funcionários (junção externa à direita).

```
SELECT *
      FROM Funcionarios f
         RIGHT OUTER JOIN Deptos d ON f.CdDepto = d.CdDepto ;
```

ou

```
SELECT *
      FROM Funcionarios f
         RIGHT JOIN Deptos d ON f.CdDepto = d.CdDepto ;
```

Resultado:

```
NRMA NMFUNC          DTADM      SEXO CDCARGO CDDEPTO CDSUPERV CDDEPTO NMDEPTO         RAMAL
---- --------------- ---------- ---- ------- ------- -------- ------- --------------- ---------
1034 ROBERTO PEREIRA 1992-05-23 M    C003    D001    1042     D001    ADMINISTRACAO   320
1021 JOSE NOGUEIRA   1994-11-10 M    C003    D001    1042     D001    ADMINISTRACAO   320
1095 MARIA DA SILVA  1992-09-03 F    C004    D001    1042     D001    ADMINISTRACAO   320
1042 PEDRO PINHEIRO  1994-07-29 M    C004    D001    1048     D001    ADMINISTRACAO   320
1015 PAULO RODRIGUES 1992-08-17 M    C002    D002    1001     D002    OFICINA         310
1001 JOAO SAMPAIO    1993-08-10 M    C002    D002    1048     D002    OFICINA         310
1004 LUCIO TORRES    1994-03-02 M    C002    D002    1001     D002    OFICINA         310
1023 LUIZ DE ALMEIDA 1993-01-12 M    C002    D002    1001     D002    OFICINA         310
1029 RUTH DE SOUZA   1992-11-05 F    C001    D003    1042     D003    SERVICOS GERAIS 330
                                                                D004    VENDAS          300
```

Comentários:

Uma junção externa à direita traz os dados de um par de tabelas que estejam relacionados entre si, como faz a junção interna, acrescidos das linhas da segunda tabela da cláusula `FROM` do comando que não possuem correspondente na primeira. Em nosso exemplo, observe que agora todos os departamentos aparecem no resultado e, para o departamento D004, que não possui nenhum funcionário, os dados do funcionário aparecem com nulos. Repare também que a junção à esquerda das tabelas A e B é equivalente à junção à direita das tabelas B e A. O operador `RIGHT OUTER JOIN` está disponível no Oracle a partir da versão 9.2 e, nas versões anteriores a única forma de se fazer a junção externa a direita era utilizando o modificador `(+)`, que para o nosso exemplo produziria a seguinte consulta:

```
SELECT *
    FROM Funcionarios f, Deptos d
    WHERE f.CdDepto(+) = d.CdDepto ;
```

✓ *Exemplo 12j:* Mostrando os dados de cada funcionário com os dados do respectivo departamento. Exibir inclusive os funcionários sem departamento e os departamentos que não possuem funcionários (junção externa total).

```
SELECT *
      FROM Funcionarios f
         FULL OUTER JOIN Deptos d ON f.CdDepto = d.CdDepto ;
```

ou

```
SELECT *
      FROM Funcionarios f
         FULL JOIN Deptos d ON f.CdDepto = d.CdDepto ;
```

Resultado:

```
NRMA NMFUNC          DTADM      SEXO CDCARGO CDDEPTO CDSUPERV CDDEPTO NMDEPTO         RAMAL
---- --------------- ---------- ---- ------- ------- -------- ------- --------------- ---------
1034 ROBERTO PEREIRA 1992-05-23 M    C003    D001    1042     D001    ADMINISTRACAO   320
1021 JOSE NOGUEIRA   1994-11-10 M    C003    D001    1042     D001    ADMINISTRACAO   320
1095 MARIA DA SILVA  1992-09-03 F    C004    D001    1042     D001    ADMINISTRACAO   320
1042 PEDRO PINHEIRO  1994-07-29 M    C004    D001    1048     D001    ADMINISTRACAO   320
1015 PAULO RODRIGUES 1992-08-17 M    C002    D002    1001     D002    OFICINA         310
1001 JOAO SAMPAIO    1993-08-10 M    C002    D002    1048     D002    OFICINA         310
1004 LUCIO TORRES    1994-03-02 M    C002    D002    1001     D002    OFICINA         310
1023 LUIZ DE ALMEIDA 1993-01-12 M    C002    D002    1001     D002    OFICINA         310
1029 RUTH DE SOUZA   1992-11-05 F    C001    D003    1042     D003    SERVICOS GERAIS 330
1048 ANA SILVEIRA    1993-06-01 F    C005
                                                     D004    VENDAS          300
```

Comentários:

Uma junção externa total traz os dados de um par de tabelas que estejam relacionados entre si, como faz a junção interna, acrescidos das linhas que não possuem correspondência de cada uma das tabelas envolvidas. Em nosso exemplo, observe que agora todos os funcionários e departamentos aparecem no resultado e, para as linhas em que não há correspondência, os dados faltantes são preenchidos com nulos. O operador `FULL OUTER JOIN` está disponível no Oracle a partir da versão 9.2 e, nas versões anteriores a única forma de se fazer a junção externa total era por meio da união de uma junção externa a direita com a junção externa à esquerda, que para o nosso exemplo produziria a seguinte consulta:

```
SELECT *
    FROM Funcionarios f, Deptos d
    WHERE f.CdDepto(+) = d.CdDepto
UNION
SELECT *
    FROM Funcionarios f, Deptos d
    WHERE f.CdDepto = d.CdDepto(+) ;
```

O operador de união será visto posteriormente neste material.

13. Fazendo cálculos em agrupamentos com junção de várias tabelas.

✓ *Exemplo 13a:* Totalizando os salários por cargo.

```
SELECT f.CdCargo, SUM(VrSalario)
      FROM Funcionarios f
           INNER JOIN Cargos c ON f.CdCargo = c.CdCargo
      GROUP BY f.CdCargo ;
```

Resultado:

```
CDCARGO             SUM
--------   ------------
C001                350
C002               3000
C005               2300
C003                900
C004               1200
```

Comentários:

O fato de termos várias tabelas em uma consulta com agrupamento não acrescenta nada de significativo ao que já foi visto anteriormente. Apenas teremos a possibilidade de explorar algumas situações mais interessantes do ponto de vista prático.

✓ *Exemplo 13b:* Exibindo, para cada cargo cujo total de salários seja superior a $500, a quantidade de funcionários e o respectivo total de salários.

```
SELECT f.CdCargo, COUNT(f.CdCargo), SUM(VrSalario)
      FROM Funcionarios f
           INNER JOIN Cargos c ON f.CdCargo = c.CdCargo
      GROUP BY f.CdCargo
      HAVING SUM(VrSalario) >= 500 ;
```

Resultado:

```
CDCARGO        COUNT             SUM
--------    --------   ------------
C002               4           3000
C005               1           2300
C003               2            900
C004               2           1200
```

- ✓ *Exemplo 13c:* Exibindo, para cada departamento que possui funcionários, o seu código e a respectiva média de salários.

```
SELECT CdDepto, AVG(VrSalario)
      FROM Funcionarios f INNER JOIN Cargos c
             ON f.CdCargo = c.CdCargo
      GROUP BY CdDepto ;
```

Resultado:

```
CDDEPTO              AVG
--------   -------------
                    2300
D002                 750
D003                 350
D001                 525
```

- ✓ *Exemplo 13d:* Calculando a soma total de salários por departamento e cargo, exibindo então esse total acompanhado do nome do departamento e do nome do cargo.

```
SELECT NmDepto, NmCargo, SUM(VrSalario)
      FROM Funcionarios f
            INNER JOIN Cargos c ON f.CdCargo = c.CdCargo
            INNER JOIN Deptos d ON f.CdDepto = d.CdDepto
      GROUP BY NmDepto, NmCargo ;
```

Resultado:

```
NMDEPTO            NMCARGO                      SUM
----------------   ----------------     ------------
ADMINISTRACAO      AUX. ESCRITORIO               900
OFICINA            MECANICO                     3000
SERVICOS GERAIS    COZINHEIRA                    350
ADMINISTRACAO      ESCRITURARIO                 1200
```

- ✓ *Exemplo 13e:* Mostrando os nomes dos departamentos e cargo com a respectiva soma de salários, para as combinações de departamento e cargo em que a média de salário é maior que $500.

```
SELECT NmDepto, NmCargo, SUM(VrSalario)
      FROM Funcionarios f
            INNER JOIN Cargos c ON f.CdCargo = c.CdCargo
            INNER JOIN Deptos d ON f.CdDepto = d.CdDepto
      GROUP BY NmDepto, NmCargo
      HAVING AVG(VrSalario) > 500 ;
```

Resultado:

```
NMDEPTO            NMCARGO                      SUM
----------------   ----------------     ------------
ADMINISTRACAO      ESCRITURARIO                 1200
OFICINA            MECANICO                     3000
```

14. Fazendo consultas com subconsultas.

✓ *Exemplo 14a:* Exibindo o nome e a matrícula dos funcionários que ganham mais do que a média de salários da empresa, em ordem alfabética do nome (subconsulta simples, com valor escalar).

```
SELECT NmFunc, NrMatric
      FROM Funcionarios f
            INNER JOIN Cargos c ON f.CdCargo = c.CdCargo
      WHERE VrSalario >
            ( SELECT AVG( VrSalario )
                  FROM Funcionarios f
                        INNER JOIN Cargos c ON f.CdCargo = c.CdCargo)
      ORDER BY NmFunc ;
```

Resultado:

```
NMFUNC          NRMATRIC
--------------- --------
ANA SILVEIRA    1048
```

Comentários:

Inicialmente precisamos determinar qual é a média de salários da empresa, e isso é feito por meio do comando `SELECT` que se encontra entre parênteses, que chamamos de subconsulta. O valor que ele retorna é então utilizado pelo comando `SELECT` mais externo, que o compara com o salário de cada funcionário. Como a nossa subconsulta produz um escalar, ou seja, exatamente um valor, na forma de uma linha com uma única coluna, podemos vinculá-la com a consulta principal por meio de um operador de comparação simples, como `>` (maior que) ou `=` (igual a) ou `<>` (diferente de), por exemplo. Uma sugestão para quem está começando a estudar subconsultas é testar os comandos separadamente, para visualizar melhor os valores que estão sendo produzidos a cada etapa do processamento.

✓ *Exemplo 14b:* Exibindo, para os cargos que possuem mais de um funcionário, todos os dados de seus funcionários (subconsulta com operador `IN`).

```
SELECT *
      FROM Funcionarios
      WHERE CdCargo IN
            ( SELECT CdCargo
                  FROM Funcionarios
                  GROUP BY CdCargo
                  HAVING COUNT(*) > 1 ) ;
```

Resultado:

```
NRMATRIC  NMFUNC           DTADM       SEXO  CDCARGO  CDDEPTO  CDSUPERV
--------  ---------------  ----------  ----  -------  -------  --------
1001      JOAO SAMPAIO     1993-08-10  M     C002     D002     1048
1004      LUCIO TORRES     1994-03-02  M     C002     D002     1001
1015      PAULO RODRIGUES  1992-08-17  M     C002     D002     1001
1023      LUIZ DE ALMEIDA  1993-01-12  M     C002     D002     1001
1034      ROBERTO PEREIRA  1992-05-23  M     C003     D001     1042
1021      JOSE NOGUEIRA    1994-11-10  M     C003     D001     1042
1042      PEDRO PINHEIRO   1994-07-29  M     C004     D001     1048
1095      MARIA DA SILVA   1992-09-03  F     C004     D001     1042
```

Comentários:

Essa é uma consulta aparentemente simples mas que, pelas características da operação de agrupamento, requer para sua resolução uma complexidade maior do que inicialmente se espera. Uma primeira solução, aparentemente óbvia, mas que não funciona, seria o comando

```
SELECT *
   FROM Funcionarios
   GROUP BY CdCargo
   HAVING COUNT(*) > 1 ;
```

Como regra geral, em um comando com a cláusula `GROUP BY`, só podemos projetar totais produzidos pelas funções de agregação (`COUNT`, `SUM`, etc), colunas que aparecem como critérios de agrupamento e também constantes. Isso ocorre porque, para cada agrupamento, apenas uma linha é colocada no resultado final, e isso significa que só podemos projetar valores que sejam comuns a todos os elementos de um agrupamento. Se colocarmos asterisco para indicar o que deverá ser projetado pelo comando, estaremos solicitando para que sejam retornados todos os dados de cada linha e, obviamente, podem existir diversas linhas distintas dentro de um agrupamento. Como para um agrupamento só podemos ter uma linha no resultado, a consulta falhará. Dessa forma, para resolver a consulta proposta para o exemplo, devemos primeiro descobrir quais os cargos que possuem pelo menos 2 funcionários e, então, percorrer a tabela `Funcionarios` buscando apenas as linhas referentes a esses cargos. Precisamos então fazer uma primeira consulta, que produzirá a lista de cargos que nos interessam, e essa consulta é dada por

```
SELECT CdCargo
   FROM Funcionarios
   GROUP BY CdCargo
   HAVING COUNT(*) > 1 ;
```

Resultado:

```
CDCARGO
-------
C002
C003
C004
```

Esse resultado deverá ser utilizado por uma segunda consulta, que comparará o valor da coluna `CdCargo` de cada linha da tabela de funcionários com essa lista, projetando todas as colunas das linhas para as quais a comparação for verdadeira. Como a comparação consiste em verificar se um código existe dentro de uma lista de códigos, utilizamos o operador `IN`, que verifica se um elemento pertence ou não a um conjunto. Dizemos que a primeira consulta realizada é uma subconsulta do `SELECT` mais externo.

✓ *Exemplo 14c:* Exibindo os nomes dos departamentos em que há funcionários contratados tanto em 1993 como em 1994 (subconsulta com operador `EXISTS`).

```
SELECT NmDepto
      FROM Deptos d
      WHERE EXISTS
            ( SELECT *
                  FROM Funcionarios f
                  WHERE f.CdDepto = d.CdDepto
                        AND TO_CHAR( DtAdm, 'YYYY' ) = '1993' )
            AND EXISTS
            ( SELECT *
                  FROM Funcionarios f
                  WHERE f.CdDepto = d.CdDepto
                        AND TO_CHAR( DtAdm, 'YYYY' ) = '1994' ) ;
```

Resultado:

```
NMDEPTO
---------------
OFICINA
```

Comentários:

Os departamentos que devem aparecer no resultado precisam possuir algum funcionário com data de admissão em 1993 e também algum outro funcionário que foi admitido em 1994. Então a consulta que soluciona o problema deve, para cada linha da tabela `Deptos`, verificar se há alguma linha na tabela de funcionários com valor na coluna `CdDepto` igual ao código desse departamento e data de admissão no ano de 1993. Também é necessário que haja, para esse departamento, alguma linha correspondente na tabela `Funcionarios` com data de admissão em 1994. Então, estruturamos a consulta com um comando `SELECT` principal, que percorre linha a linha a tabela `Deptos` e, para cada uma dessas linhas, executa uma subconsulta que verifica se há linha relacionada na tabela `Funcionarios` e ano de admissão igual a 1993 e também executa uma outra subconsulta parecida, que busca em `Funcionarios` alguma linha relacionada com ano de admissão em 1994. Para associar a consulta principal com suas subsconsultas utilizamos o operador `EXISTS`, que retorna verdadeiro se a subconsulta encontra pelo menos alguma linha que atenda ao predicado definido na cláusula `WHERE` do `SELECT` secundário.

✓ *Exemplo 14d:* Exibindo os nomes dos departamentos e cargos que possuem a mesma quantidade de funcionários (subconsulta na cláusula `FROM` de um `SELECT`).

```
SELECT NmDepto, NmCargo, Dep.Qtde
      FROM  ( SELECT NmDepto, COUNT( * ) AS Qtde
                FROM Deptos d
                   INNER JOIN Funcionarios f ON d.CdDepto = f.CdDepto
                GROUP BY NmDepto ) Dep,
            ( SELECT NmCargo, COUNT( * ) AS Qtde
                 FROM Cargos c
                   INNER JOIN Funcionarios f ON c.CdCargo = f.CdCargo
                 GROUP BY NmCargo ) Car
      WHERE Dep.Qtde = Car.Qtde ;
```

Resultado:

```
NMDEPTO           NMCARGO              QTDE
---------------  ---------------   --------
OFICINA          MECANICO                 4
ADMINISTRACAO    MECANICO                 4
SERVICOS GERAIS  COZINHEIRA               1
SERVICOS GERAIS  GERENTE                  1
```

Comentários:

Para resolver a consulta precisamos determinar a quantidade de funcionários por departamento e também por cargo. São duas subconsultas, que são renomeadas respectivamente para 'Dep' e para 'Car', posicionadas na cláusula `FROM` do `SELECT` principal. Fazemos a junção entre ambas por meio da quantidade de linhas, obtendo então as combinações desejadas, restando apenas projetar as colunas desejadas. Muitas vezes precisaremos desse recurso, em situações em que os dados que queremos processar são tabelas com linhas totalmente virtuais, como em nosso exemplo.

✓ *Exemplo 14e:* Exibindo os nomes de cada departamentos e o percentual de funcionários que possui em relação ao total da empresa (subconsulta utilizada na cláusula de projeção de um `SELECT`).

```
SELECT NmCargo,
       ROUND( COUNT( * ) * 100 /
         ( SELECT COUNT( * ) FROM Funcionarios ), 2 ) AS Percentual
      FROM Funcionarios f
            INNER JOIN Cargos c ON f.CdCargo = c.CdCargo
      GROUP BY NmCargo ;
```

Resultado:

```
NMDEPTO           PERCENTUAL
---------------   ----------
MECANICO                  40
AUX. ESCRITORIO           20
GERENTE                   10
ESCRITURARIO              20
COZINHEIRA                10
```

Comentários:

Para calcular o percentual precisamos saber a quantidade de funcionários de cada cargo e dividi-lo pela quantidade total de funcionários da empresa. Fazemos então o agrupamento conforme o cargo e somamos para cada grupo a sua quantidade de linhas. Em cada grupo essa contagem é dividida pelo resultado produzido pela subconsulta que retorna a contagem de linhas da tabela `Funcionarios`. Utilizamos a função de arredondamento `ROUND`, com duas casas decimais, para melhorar a clareza do resultado final.

15. Fazendo junção de uma tabela com ela mesma (auto-relacionamento).

✓ *Exemplo 15a:* Exibir a matrícula e o nome de todos os funcionários, juntamente com o nome do respectivo supervisor (consulta com auto-relacionamento).

```
SELECT f.NrMatric, f.NmFunc, s.NmFunc
      FROM Funcionarios f, Funcionarios s
      WHERE f.CdSuperv = s.NrMatric ;
```

ou

```
SELECT f.NrMatric, f.NmFunc, s.NmFunc
      FROM Funcionarios f
         INNER JOIN Funcionarios s ON f.CdSuperv = s.NrMatric ;
```

Resultado:

```
NRMATRIC  NMFUNC            NMFUNC
--------  ----------------  ----------------
1001      JOAO SAMPAIO      ANA SILVEIRA
1004      LUCIO TORRES      JOAO SAMPAIO
1042      PEDRO PINHEIRO    ANA SILVEIRA
1034      ROBERTO PEREIRA   PEDRO PINHEIRO
1021      JOSE NOGUEIRA     PEDRO PINHEIRO
1029      RUTH DE SOUZA     PEDRO PINHEIRO
1095      MARIA DA SILVA    PEDRO PINHEIRO
1023      LUIZ DE ALMEIDA   JOAO SAMPAIO
1015      PAULO RODRIGUES   JOAO SAMPAIO
```

Comentários:

Uma consulta com auto-relacionamento é aquela em que é preciso combinar linhas de uma tabela com outras linhas dessa mesma tabela. Isso significa que a tabela deve ser indicada duas vezes (ou mais, em algumas situações) na cláusula `FROM`. Nesse caso, é essencial atribuir apelidos à tabela no comando, para eliminar a possibilidade de ambigüidades na referência a algumas colunas, como foi feito no exemplo apresentado. No exemplo dado, precisamos combinar a linha de cada funcionário com a linha de seu respectivo supervisor (que também está na tabela de funcionários), para que seja possível projetar as colunas solicitadas.

16. Fazendo a operação de união de conjuntos (operador UNION).

✓ *Exemplo 16a:* Exibindo os códigos e nomes dos cargos e dos departamentos em ordem alfabética do nome.

```
SELECT CdCargo AS Codigo, NmCargo AS Nome, 'Cargo ' AS Tipo
      FROM Cargos
UNION
SELECT CdDepto AS Codigo, NmDepto AS Nome, 'Depto ' AS Tipo
      FROM Deptos
ORDER BY 2 ;
```

Resultado:

```
CODIGO   NOME             TIPO
-------- ---------------- ---------
D001     ADMINISTRACAO    Depto
C003     AUX. ESCRITORIO  Cargo
C001     COZINHEIRA       Cargo
C004     ESCRITURARIO     Cargo
C005     GERENTE          Cargo
C002     MECANICO         Cargo
D002     OFICINA          Depto
D003     SERVICOS GERAIS  Depto
D004     VENDAS           Depto
C006     VIGIA            Cargo
```

Comentários:

O operador UNION permite acrescentar o resultado do segundo SELECT ao resultado do primeiro. As duas consultas envolvidas devem ser união-compatíveis, ou seja, projetar a mesma quantidade de colunas, com os mesmos tipos de dados na mesma ordem (se a primeira coluna de uma consulta é numérica, a primeira coluna da outra consulta também deve ser numérica). Caso existam linhas em comum nos resultados dos dois SELECT's envolvidos, apenas uma ocorrência de cada uma dessas linhas é colocada no resultado final, ou seja, as repetições são descartadas. Se for necessário manter as repetições, em vez de especificarmos UNION, devemos especificar UNION ALL. Se, como ocorreu em nosso exemplo, for necessário ordenar o resultado, a cláusula ORDER BY deve aparecer apenas após o segundo SELECT.

17. Fazendo a operação de intersecção de conjuntos (operador `INTERSECT`).

✓ *Exemplo 17a:* Exibindo os códigos que existem tanto na tabela de cargos como na de departamentos.

```
SELECT CdCargo AS Codigo
      FROM Cargos
INTERSECT
SELECT CdDepto AS Codigo
      FROM Deptos ;
```

Resultado:

```
não há linhas selecionadas
```

Comentários:

O operador `INTERSECT` permite obter linhas que existem tanto no resultado do primeiro `SELECT` como no do segundo. As duas consultas envolvidas deverão ser união-compatíveis, ou seja, projetar a mesma quantidade de colunas, com os mesmos tipos de dados na mesma ordem (se a primeira coluna de uma consulta é numérica, a primeira coluna da outra consulta também deve ser numérica, e assim por diante). Em nosso exemplo, não há qualquer código de departamento que seja igual a algum código de cargo, portanto a consulta não retorna linhas.

18. Fazendo a operação de diferença de conjuntos (operador `MINUS`).

✓ *Exemplo 18a:* Exibindo os nomes dos cargos que não são usados para o departamento D001.

```
SELECT NmCargo AS Cargo
      FROM Cargos
MINUS
SELECT NmCargo AS Cargo
      FROM Funcionarios f
            INNER JOIN Cargos c ON f.CdCargo = c.CdCargo
      WHERE CdDepto = 'D001' ;
```

Resultado:

```
CARGO
----------
COZINHEIRA
GERENTE
MECANICO
VIGIA
```

Comentários:

O operador `MINUS` permite obter linhas que existem no resultado do primeiro `SELECT` e não existem no resultado do segundo. As duas consultas envolvidas devem ser união-compatíveis, ou seja, projetar a mesma quantidade de colunas, com os mesmos tipos de dados na mesma ordem (se a primeira coluna de uma consulta é numérica, a primeira coluna da outra consulta também deve ser numérica). No exemplo, o primeiro `SELECT` obtém os nomes de todos os cargos e o segundo, os nomes dos cargos encontrados no departamento D001. A diferença entre esses dois conjuntos de dados é exatamente os cargos que existem na tabela de cargos mas não estão associados com o departamento D001.

19. Utilizando condições ao projetar valores (expressões CASE).

✓ *Exemplo 19a:* Exibindo os funcionários com uma definição de faixas de salário.

```
SELECT NmFunc, VrSalario,
      ( CASE
          WHEN VrSalario > 1000 THEN 'ALTA'
          WHEN VrSalario < 500  THEN 'BAIXA'
          ELSE 'INTERMEDIARIA'
        END ) AS Faixa
      FROM Funcionarios f
           INNER JOIN Cargos c ON f.CdCargo = c.CdCargo ;
```

Resultado:

```
NMFUNC               VRSALARIO FAIXA
-----------------    --------- -------------
ANA SILVEIRA              2300 ALTA
JOAO SAMPAIO               750 INTERMEDIARIA
LUCIO TORRES               750 INTERMEDIARIA
PEDRO PINHEIRO             600 INTERMEDIARIA
ROBERTO PEREIRA            450 BAIXA
JOSE NOGUEIRA              450 BAIXA
RUTH DE SOUZA              350 BAIXA
MARIA DA SILVA             600 INTERMEDIARIA
LUIZ DE ALMEIDA            750 INTERMEDIARIA
PAULO RODRIGUES            750 INTERMEDIARIA
```

Comentários:

Uma expressão CASE é uma estrutura que permite definir uma série de condições que devem ser testadas para cada linha, de maneira a definir o valor a ser projetado como uma coluna do resultado. No exemplo, para cada linha da junção dos dados dos funcionários com o respectivo cargo, o valor do salário é comparado com valores específicos que determinarão o que deverá ser exibido na terceira coluna.

✓ *Exemplo 19b:* Exibindo os funcionários com uma definição de categoria que considera os departamentos e os sexos.

```
SELECT NmFunc,
      ( CASE
           WHEN CdDepto IS NULL OR CdDepto = 'D001' THEN
                ( CASE
                     WHEN Sexo = 'M' THEN 'Administrativo'
                     WHEN Sexo = 'F' THEN 'Administrativa'
                  END )
           ELSE
                ( CASE
                     WHEN Sexo = 'M' THEN 'Técnico'
                     WHEN Sexo = 'F' THEN 'Técnica'
                  END )
        END ) AS Categoria
      FROM Funcionarios ;
```

Resultado:

```
NMFUNC              CATEGORIA
-----------------   -------------
ANA SILVEIRA        Administrativa
JOAO SAMPAIO        Técnico
LUCIO TORRES        Técnico
PEDRO PINHEIRO      Administrativo
ROBERTO PEREIRA     Administrativo
JOSE NOGUEIRA       Administrativo
RUTH DE SOUZA       Técnica
MARIA DA SILVA      Administrativa
LUIZ DE ALMEIDA     Técnico
PAULO RODRIGUES     Técnico
```

Comentários:

É possível estabelecer condições mais complexas, utilizando os conectivos lógicos `OR` e `AND`, bem como aninhar estruturas `CASE`, como mostra o exemplo.

20. Retendo apenas as *n* primeiras linhas de um resultado (pseudo-coluna `ROWNUM`).

✓ *Exemplo 20a:* Exibindo os dados das 5 primeiras linhas da tabela de funcionários.

```
SELECT *
      FROM Funcionarios
      WHERE ROWNUM <= 5 ;
```

Resultado:

```
NRMATRIC  NMFUNC           DTADM       SEXO  CDCARGO  CDDEPTO  CDSUPERV
--------  ---------------  ----------  ----  -------  -------  --------
1048      ANA SILVEIRA     1993-06-01  F     C005
1001      JOAO SAMPAIO     1993-08-10  M     C002     D002     1048
1004      LUCIO TORRES     1994-03-02  M     C002     D002     1001
1042      PEDRO PINHEIRO   1994-07-29  M     C004     D001     1048
1034      ROBERTO PEREIRA  1992-05-23  M     C003     D001     1042
```

Comentários:

`ROWNUM` é uma coluna virtual cujo valor é definido durante a fase de avaliação da cláusula `WHERE` do comando. A primeira linha que atende ao predicado de seleção recebe como `ROWNUM` o valor 1, a segunda recebe o valor 2 e assim sucessivamente. Esse recurso é útil para selecionar apenas uma parte do total de linhas que atende a um determinado critério.

✓ *Exemplo 20b:* Exibindo os dados das 3 últimas linhas da tabela de funcionários.

```
SELECT *
      FROM ( SELECT f.*, ROWNUM AS NrLinha FROM Funcionarios f )
      WHERE NrLinha > 7 ;
```

Resultado:

```
NRMATRIC  NMFUNC           DTADM       SEXO  CDCARGO  CDDEPTO  CDSUPERV
--------  ---------------  ----------  ----  -------  -------  --------
1095      MARIA DA SILVA   1992-09-03  F     C004     D001     1042
1023      LUIZ DE ALMEIDA  1993-01-12  M     C002     D002     1001
1015      PAULO RODRIGUES  1992-08-17  M     C002     D002     1001
```

Comentários:

Devido à forma como o valor de `ROWNUM` é atribuído, que é sempre após a linha ter a cláusula `WHERE` avaliada como verdadeira, qualquer consulta que tenha como condição `ROWNUM > <expN>` retornará zero linhas, já que no momento dessa comparação o valor de `ROWNUM` da linha não estará definido. Nessas situações é necessário fazer uma subconsulta inicial que atribuirá o valor a `ROWNUM` para que a filtragem desejada possa então ser realizada na consulta principal, como foi feito no exemplo.

Comando INSERT

Insere linhas numa tabela ou visão.

```
INSERT INTO {<tabela> | <visão>} [(<col1> [, <col2> ...])]
       {VALUES (<val1> [, <val2> ...]) | <select_expr>};
```

Argumento	*Significado*
`INTO {<tabela> | <visão>}`	Nome de uma tabela ou visão previamente existente na qual se deseja inserir dados.
`<col`$_n$`>`	Nome de uma coluna existente na tabela ou visão onde os valores serão armazenados. Esta cláusula é opcional
`VALUES (<val1> [, <val2> ...])`	Lista de valores a inserir na tabela ou visão. Os valores precisam ser apresentados na mesma ordem da lista de nomes das colunas indicada na cláusula anterior. Caso essa lista de colunas não tenha sido indicada no comando, os valores apresentados em `VALUES` precisam estar na mesma ordem das colunas da tabela de destino, e exatamente na mesma quantidade, ou seja, um valor para cada coluna.
`<select_expr>`	Consulta que retorna os valores a inserir nas colunas destino.

Exemplos:

21. Incluindo valores utilizando a cláusula `VALUES`.

✓ *Exemplo 21a:* Inserindo uma linha na tabela `Funcionarios`, omitindo a os nomes das colunas.

```
INSERT INTO Funcionarios VALUES
   ('1090', 'ROSANA GARCIA', '11-JAN-1997', 'F', 'C004', 'D001',
    '1042') ;
```

Resultado:

```
SELECT *
      FROM Funcionarios ;
```

NRMATRIC	NMFUNC	DTADM	SEXO	CDCARGO	CDDEPTO	CDSUPERV
1048	ANA SILVEIRA	1993-06-01	F	C005		
1001	JOAO SAMPAIO	1993-08-10	M	C002	D002	1048
1004	LUCIO TORRES	1994-03-02	M	C002	D002	1001
1042	PEDRO PINHEIRO	1994-07-29	M	C004	D001	1048
1034	ROBERTO PEREIRA	1992-05-23	M	C003	D001	1042
1021	JOSE NOGUEIRA	1994-11-10	M	C003	D001	1042
1029	RUTH DE SOUZA	1992-11-05	F	C001	D003	1042
1095	MARIA DA SILVA	1992-09-03	F	C004	D001	1042
1023	LUIZ DE ALMEIDA	1993-01-12	M	C002	D002	1001
1015	PAULO RODRIGUES	1992-08-17	M	C002	D002	1001
1090	ROSANA GARCIA	1997-01-11	F	C004	D001	1042

Comentários:

A lista com os nomes das colunas entre parênteses pode ser omitida sempre que o comando `INSERT` terá, na cláusula `VALUES`, exatamente um valor para cada coluna da tabela, na ordem correta em que as colunas aparecem (o primeiro valor para a primeira coluna, o segundo valor para a segunda coluna, e assim por diante).

✓ *Exemplo 21b:* Inserindo uma linha na tabela `Funcionarios`, indicando os nomes das colunas.

```
INSERT INTO Funcionarios (NmFunc, NrMatric, DtAdm, CdCargo, Sexo)
      VALUES ('CRISTINA MATOS', '1082', '23-FEV-1997', 'C003', 'F');
```

Resultado:

```
SELECT *
      FROM Funcionarios ;
```

NRMATRIC	NMFUNC	DTADM	SEXO	CDCARGO	CDDEPTO	CDSUPERV
1048	ANA SILVEIRA	1993-06-01	F	C005		
1001	JOAO SAMPAIO	1993-08-10	M	C002	D002	1048
1004	LUCIO TORRES	1994-03-02	M	C002	D002	1001
1042	PEDRO PINHEIRO	1994-07-29	M	C004	D001	1042
1034	ROBERTO PEREIRA	1992-05-23	M	C003	D001	1042
1021	JOSE NOGUEIRA	1994-11-10	M	C003	D001	1042
1029	RUTH DE SOUZA	1992-11-05	F	C001	D003	1042
1095	MARIA DA SILVA	1992-09-03	F	C004	D001	1042
1023	LUIZ DE ALMEIDA	1993-01-12	M	C002	D002	1001
1015	PAULO RODRIGUES	1992-08-17	M	C002	D002	1001
1090	ROSANA GARCIA	1997-01-11	F	C004	D001	1042
1082	CRISTINA MATOS	1997-02-23	F	C003		

Comentários:

Este comando `INSERT` não fornece valores para todas as colunas da tabela de funcionários. Nesse caso, as colunas `CdDepto` e `CdSuperv` da nova linha conterão o valor `NULL`, que significa 'informação desconhecida'. Caso a definição da tabela `Funcionarios` especificasse que essas colunas fossem de preenchimento obrigatório (restrição `NOT NULL`) e não houvesse uma cláusula `DEFAULT` para elas, uma mensagem de erro seria emitida pelo SGBD e a linha não seria incluída. Para comandos `INSERT` colocados dentro de programas, é interessante sempre especificar a lista de colunas imediatamente antes da cláusula `VALUES`, pois além de melhorar a clareza da instrução, também pode evitar algumas situações em que o comando deixa de funcionar devido a pequenas mudanças no *lay-out* da tabela.

22. Incluindo valores a partir de um `SELECT`.

✓ *Exemplo 22a:* Acrescentando dados em Cargos a partir do conteúdo de uma tabela auxiliar.

```
INSERT INTO Cargos
      SELECT Codigo, Nome, Valor FROM Funcoes ;
```

Resultado:

```
SELECT *
      FROM Cargos ;
```

CDCARGO	NMCARGO	VRSALARIO
C003	AUX. ESCRITORIO	450.00
C006	VIGIA	500.00
C002	MECANICO	750.00
C005	GERENTE	2300.00
C004	ESCRITURARIO	600.00
C001	COZINHEIRA	350.00
F1	PEDREIRO	700.00
F2	SERVENTE	250.00
F3	AJUDANTE GERAL	250.00
F4	JARDINEIRO	400.00

Comentários:

Podemos acrescentar a uma tabela linhas provenientes de um comando `SELECT`, desde que esta consulta retorne os valores na ordem correta esperada para as colunas da tabela de destino. Para que o comando do exemplo apresentado funcione devemos assumir que existe uma tabela chamada `Funcoes` onde temos 4 linhas, descrevendo as funções denominadas 'PEDREIRO', 'SERVENTE', 'AJUDANTE GERAL' e 'JARDINEIRO'.

Comando DELETE

Exclui linhas de uma tabela ou visão.

```
DELETE FROM {<tabela> | <visão>}
      [WHERE <condição_de_busca>];
```

Argumento	*Significado*
`FROM <tabela> | <view>`	Nome da tabela ou *view* da qual as linhas serão excluídas.
`WHERE <condição_de_busca>`	Condição de busca que especifica as linhas a excluir. Sem esta cláusula, o comando `DELETE` eliminará todas as linhas na tabela ou *view* especificada.

Exemplos:

23. Excluindo linhas de tabelas.

✓ *Exemplo 23a:* Eliminando algumas linhas da tabela de cargos (`DELETE` com cláusula `WHERE`).

```
DELETE FROM Cargos
      WHERE CdCargo LIKE 'F%' ;
```

Resultado:

```
SELECT *
      FROM Cargos ;
```

CDCARGO	NMCARGO	VRSALARIO
C003	AUX. ESCRITORIO	450.00
C006	VIGIA	500.00
C002	MECANICO	750.00
C005	GERENTE	2300.00
C004	ESCRITURARIO	600.00
C001	COZINHEIRA	350.00

Comentários:

No exemplo, eliminamos todos os cargos cujo código se iniciava com a letra 'F'. É possível que um comando `DELETE` corretamente escrito falhe, se sua ação vier a tornar o banco de dados inconsistente. Isso ocorreria, por exemplo, se tentássemos excluir um cargo que ainda possui funcionários pertencentes a ele. Nesse caso, um erro seria gerado pelo SGBD e nenhuma linha seria excluída.

- ✓ *Exemplo 23b:* Excluindo TODAS as linhas da tabela de funcionários (`DELETE` sem cláusula `WHERE`).

```
DELETE FROM Funcionarios ;
```

Resultado:

```
SELECT *
      FROM Funcionarios ;

                    não há linhas selecionadas
```

Comando UPDATE

Atualiza o conteúdo de uma tabela ou visão.

```
UPDATE {<tabela> | <view>}
       SET <col1> = <val1> [, <col2> = <val2> ...]
       [WHERE <condição_de_busca>] ;
```

Argumento	*Significado*
`<tabela> | <view>`	Nome da tabela ou visão a atualizar.
`SET <coln> = <valn>`	Especifica as colunas a atualizar e os valores a atribuir a essas colunas.
`WHERE <condição_de_busca>`	Especifica a condição para que uma linha possa ser atualizada.

Exemplos:

24. Alterando várias colunas em um comando.

✓ *Exemplo 24a:* Modificando o código do cargo e do supervisor para um funcionário específico.

```
UPDATE Funcionarios
      SET CdCargo = 'C004', CdSuperv = '1048'
      WHERE NrMatric = '1034' ;
```

Resultado:

```
SELECT *
      FROM Funcionario ;
```

NRMATRIC	NMFUNC	DTADM	SEXO	CDCARGO	CDDEPTO	CDSUPERV
1048	ANA SILVEIRA	1993-06-01	F	C005		
1001	JOAO SAMPAIO	1993-08-10	M	C002	D002	1048
1004	LUCIO TORRES	1994-03-02	M	C002	D002	1001
1042	PEDRO PINHEIRO	1994-07-29	M	C004	D001	1042
1034	ROBERTO PEREIRA	1992-05-23	M	C004	D001	1048
1021	JOSE NOGUEIRA	1994-11-10	M	C003	D001	1042
1029	RUTH DE SOUZA	1992-11-05	F	C001	D003	1042
1095	MARIA DA SILVA	1992-09-03	F	C004	D001	1042
1023	LUIZ DE ALMEIDA	1993-01-12	M	C002	D002	1001
1015	PAULO RODRIGUES	1992-08-17	M	C002	D002	1001

Comentários:

É possível modificar o valor de várias colunas em cada linha, bastando para isso especificar na cláusula `SET` as atribuições desejadas, separadas por vírgulas.

25. Alterando várias linhas em um comando.

✓ *Exemplo 25a:* Aplicando 10% de aumento no salário de alguns cargos.

```
UPDATE Cargos
      SET VrSalario = VrSalario * 1.1
      WHERE CdCargo IN ('C004', 'C003', 'C001' ) ;
```

Resultado:

```
SELECT *
      FROM Cargos ;
```

CDCARGO	NMCARGO	VRSALARIO	
C006	VIGIA	500.00	
C002	MECANICO	750.00	
C005	GERENTE	2300.00	
C003	AUX. ESCRITORIO	495.00	◄——
C004	ESCRITURARIO	660.00	◄——
C001	COZINHEIRA	385.00	◄——

Comentários:

No exemplo, para cada linha da tabela de cargos é verificado se o código do cargo pertence à lista indicada no operador `IN` e, caso seja, o valor do salário é multiplicado por 1.1 e o resultado é gravado na própria coluna correspondente ao salário.

Resumo dos principais operadores suportados pela SQL

Operador	*Significado*
+	Aritmético: Adição.
-	Aritmético: Subtração.
*	Aritmético: Multiplicação.
/	Aritmético: Divisão.
^	Aritmético: Exponenciação.
=	Comparação: Igualdade.
<	Comparação: Menor que ...
<=	Comparação: Menor ou igual a ...
<>	Comparação: Diferente de ...
>	Comparação: Maior que ...
>=	Comparação: Maior ou igual a ...
IN	Comparação: Verifica se um valor está contido em uma lista.
BETWEEN	Comparação: Verifica se um valor está contido numa faixa contínua de valores.
LIKE	Comparação: Verifica se o formato de um valor atende a um determinado padrão.
%	Curinga: Qualquer combinação de caracteres (usado com LIKE).
_	Curinga: Qualquer caracter na posição (usado com LIKE).
AND	Lógico: E (Conjunção).
OR	Lógico: OU não exclusivo.
NOT	Lógico: Negação.

Resumo das principais funções de agregação

A linguagem SQL oferece um conjunto de funções destinadas a calcular valores estatísticos a partir de grupos de linhas selecionadas através da cláusula GROUP BY prevista no comando SELECT. Conforme os atributos que especificamos nessa cláusula serão os agrupamentos de linhas que serão produzidos, uma vez que linhas com o mesmo valor nos atributos especificados são colocadas no mesmo grupo. As funções de agregação previstas pela SQL padrão são indicadas na tabela a seguir.

Função	*Finalidade*
`AVG`	Média de valores de uma expressão numérica.
`COUNT`	Contagem de valores.
`MAX`	Valor máximo de uma expressão.
`MIN`	Valor mínimo de uma expressão.
`STDDEV`	Desvio padrão dos valores de uma expressão numérica.
`SUM`	Soma total dos valores de uma expressão numérica.
`VARIANCE`	Variância dos valores de uma expressão numérica.

Funções pré-definidas em SQL

Algumas das principais funções pré-definidas (*built-in functions*) do dialeto SQL suportado pelo Oracle são apresentadas aqui, de maneira bastante resumida. Alguns exemplos fornecidos podem não produzir em outros computadores o mesmo resultado aqui apresentado pois certas operações são dependentes de detalhes de configuração do ambiente. Muitos exemplos são apresentados utilizando-se a tabela padrão `Dual`, que é destinada a consultas que não requerem nenhuma tabela específica para produzir seu resultado. Em alguns casos, porém, são usadas as tabelas `Funcionarios`, `Cargos` e `Deptos`, utilizadas em aula e criadas através dos scripts `CriPes.sql` e `DadosPes.sql`.

ABS

Sintaxe:

```
ABS( <expN> )
```

Descrição:

Retorna o valor absoluto, ou seja, o módulo ou valor sem sinal, de `expN`.

Exemplos:

```
SELECT ABS( -3 ) FROM Dual ;
```

Resultado:

```
                              ABS(-3)
                            ----------
                                     3
```

```
SELECT ABS( 8 ) FROM Dual ;
```

Resultado:

```
                               ABS(8)
                            ----------
                                     8
```

ADD_MONTHS

Sintaxe:

```
ADD_MONTHS( <expD>, <expN> )
```

Descrição:

Retorna a data correspondente a `expD` acrescida de `expN` meses.

Exemplos:

```
SELECT ADD_MONTHS( SYSDATE, 6 ) FROM Dual ;
```

Resultado:

```
ADD_MONTHS(SYSDATE,6)
---------------------
01/07/2000
```

Comentários:

No exemplo, estamos supondo que a data do sistema, contida em SYSDATE, seja 1º de Janeiro de 2000. O formato de exibição da data no resultado pode ser diferente do mostrado aqui, devido a configurações do seu ambiente.

ASCII

Sintaxe:

```
ASCII( <expC> )
```

Descrição:

Retorna o código ASCII do primeiro caracter de `expC`.

Exemplos:

```
SELECT ASCII( CdDepto ) FROM Deptos WHERE CdDepto = 'D001' ;
```

Resultado:

```
ASCII(CDDEPTO)
--------------
            68
```

Comentários:

A consulta filtra a linha da tabela de departamentos que possui o código 'D001' na coluna `CdDepto`. Esse valor é passado para a função `ASCII`, que retorna o código 68, que corresponde à letra 'D' na tabela ASCII.

AVG

Sintaxe:

```
AVG( [ DISTINCT | ALL] <expN> )
```

Descrição:

Função de agregação que retorna a média dos valores contidos em `expN`, desconsiderando seus eventuais valores nulos. Os modificadores `DISTINCT` e `ALL` indicam respectivamente se devem ser desprezadas ou não as repetições de valores contidos em `expN`.

Exemplos:

```
SELECT AVG( VrSalario ) FROM Cargos ;
```

Resultado:

```
AVG(VRSALARIO)
--------------
           825
```

Comentários:

A consulta produz a média dos salários de todos os cargos cadastrados na tabela `Cargos`.

```
SELECT AVG( VrSalario )
   FROM Cargos
   WHERE CdCargo IN ( SELECT CdCargo FROM Funcionarios ) ;
```

Resultado:

```
AVG(VRSALARIO)
--------------
           890
```

Comentários:

A consulta produz a média dos salários dos cargos cadastrados, mas considerando apenas os cargos que possuem funcionários.

```
SELECT AVG( VrSalario )
   FROM Funcionarios f
           INNER JOIN Cargos c ON f.CdCargo = c.CdCargo ;
```

Resultado:

```
AVG(VRSALARIO)
--------------
           775
```

Comentários:

A consulta produz o valor correspondente à média geral dos salários pagos pela empresa, considerando a quantidade de funcionários de cada cargo, já que é feita a junção das tabelas `Cargos` e `Funcionarios`.

```
SELECT AVG( DISTINCT VrSalario )
   FROM Funcionarios f
           INNER JOIN Cargos c ON f.CdCargo = c.CdCargo ;
```

Resultado:

```
AVG(DISTINCTVRSALARIO)
----------------------
                   890
```

Comentários:

Como é utilizado o modificador `DISTINCT` na função `AVG`, a consulta produz o valor correspondente à média dos salários pagos pela empresa, considerando os valores dos salários sem repetições, ou seja, um mesmo salário que é pago a dois funcionários distintos é considerado apenas uma vez no cálculo. Observe que este resultado é igual ao obtido no segundo exemplo, mostrado anteriormente, pois na tabela de cargos não há dois cargos distintos com o mesmo salário.

```
SELECT AVG( ALL VrSalario )
   FROM Funcionarios f
           INNER JOIN Cargos c ON f.CdCargo = c.CdCargo ;
```

Resultado:

```
AVG(ALLVRSALARIO)
-----------------
              775
```

Comentários:

Como é utilizado o modificador `ALL` na função `AVG`, a consulta produz o valor correspondente à média dos salários pagos pela empresa, considerando os valores dos salários preservando repetições, ou seja, um mesmo salário que é pago a dois funcionários distintos é considerado duas vezes no cálculo. Observe que este resultado é igual ao obtido no terceiro exemplo, mostrado anteriormente, pois quando não especificamos os modificadores `ALL` ou `DISTINCT`, `ALL` é assumido como padrão.

CEIL

Sintaxe:

```
CEIL( <expN> )
```

Descrição:

Retorna o menor inteiro que seja não menor que `expN`. Esse tipo de resultado é muitas vezes chamado de o teto de `expN`.

Exemplos:

```
SELECT CEIL( 12.33 ) FROM Dual ;
```

Resultado:

```
                          CEIL(12.33)
                          -----------
                                   13
```

CHR

Sintaxe:

```
CHR( <expN> )
```

Descrição:

Retorna o caracter da tabela ASCII correspondente ao valor informado no parâmetro.

Exemplos:

```
SELECT CHR( 75 ) FROM Dual ;
```

Resultado:

```
                           CHR(75)
                           -------
                           K
```

Comentários:

O código 75 corresponde à letra K maiúscula na tabela ASCII.

COUNT

Sintaxe:

```
COUNT( [ DISTINCT | ALL ] { * | <exp> } )
```

Descrição:

É uma função de agregação que retorna o número de linhas da expressão fornecida como parâmetro, desconsiderando seus eventuais valores nulos. Se for utilizado o asterisco em vez de uma expressão, a função retornará a quantidade de linhas do conjunto, independentemente de possuirem ou não valores nulos em alguma coluna. Os modificadores `DISTINCT` e `ALL` indicam respectivamente se devem ser desprezadas ou não as repetições de valores.

Exemplos:

```
SELECT COUNT( CdDepto ) FROM Funcionarios ;
```

Resultado:

```
COUNT(CDDEPTO)
--------------
             9
```

Comentários:

A chamada à função `COUNT` leva como parâmetro o nome da coluna `CdDepto`, significando que devem ser contadas quantas linhas da tabela `Funcionarios` possuem um valor não nulo nessa coluna. Como dos 10 funcionários cadastrados apenas um não possui um código de departamento, o resultado apresentado é 9.

```
SELECT COUNT( DISTINCT CdDepto ) FROM Funcionarios ;
```

Resultado:

```
COUNT(DISTINCTCDDEPTO)
----------------------
                     3
```

Comentários:

Como é utilizado o modificador `DISTINCT` na função `COUNT`, indicamos que desejamos saber quantos valores distintos e não nulos existem na coluna `CdDepto` da tabela `Funcionarios`.

```
SELECT COUNT( ALL CdDepto ) FROM Funcionarios ;
```

Resultado:

```
COUNT(ALLDDEPTO)
----------------
               9
```

Comentários:

Como é utilizado o modificador `ALL` na função `COUNT`, a consulta produz o valor correspondente à quantidade de valores não nulos existentes na coluna CdDepto da tabela Funcionarios, preservando eventuais valores repetidos, ou seja, um mesmo código de departamento atribuído a dois funcionários distintos é considerado duas vezes na contagem. Observe que este resultado é igual ao obtido no primeiro exemplo, mostrado anteriormente, pois quando não especificamos os modificadores `ALL` ou `DISTINCT`, `ALL` é assumido como padrão.

```
SELECT COUNT( * ) FROM Funcionarios ;
```

Resultado:

```
  COUNT(*)
----------
        10
```

Comentários:

Como não especificamos o nome de nenhuma coluna em particular, estamos interessados em saber quantas linhas existem no conjunto de dados correspondente à tabela `Funcionarios`, que possui 10 linhas.

DECODE

Sintaxe:

```
DECODE( <exp> , <exp1> , <exp2> [, <exp3> ] )
```

Descrição:

Esta função retorna `exp2` sempre que `exp` seja igual a `exp1` e, em caso contrário, retorna `exp3`. Quando um valor para `exp3` for omitido, a função retorna nulo se nenhuma condição for satisfeita. A utilização de expressões `CASE`, apresentadas anteriormente com o comando `SELECT`, oferece maiores recursos que a função `DECODE`.

Exemplos:

```
SELECT NmFunc, DECODE( CdDepto, 'D001', 'ADMIN.', 'Outros' )
   FROM Funcionarios WHERE Sexo = 'F' ;
```

Resultado:

```
NMFUNC              DECODE(CdDepto,'D001','ADMIN.','Outros')
------------------  ----------------------------------------
ANA SILVEIRA        Outros
RUTH DE SOUZA       Outros
MARIA DA SILVA      ADMIN.
```

FLOOR

Sintaxe:

```
FLOOR( <expN> )
```

Descrição:

Retorna o maior inteiro que seja não maior que `expN`. Na matemática, esse tipo de resultado é chamado de piso de `expN`.

Exemplos:

```
SELECT FLOOR( 25.128 ) FROM Dual ;
```

Resultado:

```
                        FLOOR(25.128)
                        -------------
                                   25
```

```
SELECT FLOOR( -1.25 ) FROM Dual ;
```

Resultado:

```
                         FLOOR(-1.25)
                        -------------
                                  -2
```

GREATEST

Sintaxe:

```
GREATEST( <exp>, ...)
```

Descrição:

Retorna o maior valor de uma lista de expressões fornecida como parâmetro.

Exemplos:

```
SELECT GREATEST( 'batata', 825, 'tomate', 'caju' ) FROM Dual ;
```

Resultado:

```
GREATEST('BATATA',825,'TOMATE','CAJU')
--------------------------------------
tomate
```

INITCAP

Sintaxe:

```
INITCAP( <expC> )
```

Descrição:

Retorna o conteúdo de `expC` com a primeira letra de cada palavra em maiúscula e o restante em minúsculas.

Exemplos:

```
SELECT INITCAP( NmFunc ) FROM Funcionarios
   WHERE NrMatric = '1004' ;
```

Resultado:

```
INITCAP(NMFUNC)
-------------------
Lucio Torres
```

INSTR

Sintaxe:

```
INSTR( <expC1>, <expC2> [, <expN1> [, <expN2> ] ] )
```

Descrição:

Retorna a posição da n-ésima ocorrência de `expC2` dentro de `expC1`, começando a pesquisa na posição `expN1` e qual a `expN2` ocorrência a ser encontrada. Caso `expN1` ou `expN2` não sejam informados, o valor default para esses parâmetros é 1.

Exemplos:

```
SELECT INSTR( 'Programa', 'r', 1, 2 ) FROM Dual ;
```

Resultado:

```
                    INSTR('PROGRAMA','R',1,2)
                    -------------------------
                                            5
```

```
SELECT INSTR( 'Programa', 'r', 1, 1 ) FROM Dual ;
```

Resultado:

```
                    INSTR('PROGRAMA','R',1,1)
                    -------------------------
                                            2
```

```
SELECT INSTR( 'Programa', 'a' ) FROM Dual ;
```

Resultado:

```
                      INSTR('PROGRAMA','A')
                      ---------------------
                                         6
```

```
SELECT INSTR( 'Programa', 'x', 1, 1 ) FROM Dual ;
```

Resultado:

```
                    INSTR('PROGRAMA','X',1,1)
                    -------------------------
                                            0
```

```
SELECT NmFunc, INSTR( NmFunc, 'A', 5, 2 )
   FROM Funcionarios WHERE NrMatric = '1001' ;
```

Resultado:

```
NMFUNC                    INSTR(NMFUNC,'A',5,2)
-----------------------   ---------------------
JOAO SAMPAIO                                 10
```

LAST_DAY

Sintaxe:

```
LAST_DAY( <expD> )
```

Descrição:

Retorna uma data indicando qual o último dia do mês correspondente à data fornecida como parâmetro.

Exemplos:

```
SELECT LAST_DAY( SYSDATE ) FROM Dual ;
```

Resultado:

```
LAST_DAY(SYSDATE)
-----------------
31/01/2000
```

Comentários:

No exemplo, estamos supondo que a data do sistema, contida em SYSDATE, seja 1º de Janeiro de 2000. O formato de exibição da data no resultado pode ser diferente do mostrado aqui, devido a configurações do seu ambiente.

LEAST

Sintaxe:

```
LEAST( <exp>, ... )
```

Descrição:

Retorna o menor valor disponível em uma lista de valores fornecida como parâmetro.

Exemplos:

```
SELECT LEAST( 'batata', 825, 'tomate', 'caju' ) FROM Dual ;
```

Resultado:

```
LEAST('BATATA',825,'TOMATE','CAJU')
-----------------------------------
                                825
```

LENGTH

Sintaxe:

```
LENGTH( <expC> )
```

Descrição:

Retorna o número de caracteres contidos em `expC`.

Exemplos:

```
SELECT LENGTH( 'Programa' ) FROM Dual ;
```

Resultado:

```
                        LENGTH('PROGRAMA')
                        ------------------
                                         8
```

```
SELECT NmDepto, LENGTH( NmDepto )
   FROM Deptos WHERE CdDepto = 'D001' ;
```

Resultado:

```
             NMFUNC                LENGTH(NMDEPTO)
             --------------------  ---------------
             ADMINISTRACAO                      13
```

LOWER

Sintaxe:

```
LOWER( <expC> )
```

Descrição:

Retorna `expC` com todas as suas letras em minúsculas.

Exemplos:

```
SELECT LOWER( NmDepto ) FROM Deptos WHERE CdDepto = 'D001';
```

Resultado:

```
                      LOWER(NMDEPTO)
                      --------------------
                      administracao
```

LPAD

Sintaxe:

```
LPAD( <expC1> , <expN> [, <expC2> ] )
```

Descrição:

Retorna o conteúdo de `expC1` preenchido à esquerda com `expN` caracteres de `expC2`. Caso `expC2` não seja informada a função preenche o restante com espaços em branco.

Exemplos:

```
SELECT LPAD( 'abc', 8, '*' ) FROM Dual ;
```

Resultado:

```
LPAD('ABC',8,'*')
-----------------
*****abc
```

```
SELECT LPAD( 'abc', 8, '*x' ) FROM Dual ;
```

Resultado:

```
LPAD('ABC',8,'*X')
------------------
*x*x*abc
```

```
SELECT LPAD( 'abc', 8 ) FROM Dual ;
```

Resultado:

```
LPAD('ABC',8)
-------------
     abc
```

LTRIM

Sintaxe:

```
LTRIM( <expC1> [, <expC2> ] )
```

Descrição:

Elimina todos os caracteres à esquerda de `expC1` que sejam iguais a `expC2`. Caso o parâmetro `expC2` não seja fornecido, os espaços em branco iniciais são eliminados.

Exemplos:

```
SELECT LTRIM( '---Programa---', '-' ) FROM Dual ;
```

Resultado:

```
LTRIM('---PROGRAMA---','-')
---------------------------
Programa---
```

```
SELECT LTRIM( '  -Programa-   ' ) FROM Dual ;
```

Resultado:

```
LTRIM('  -PROGRAMA-   ')
------------------------
-Programa-
```

MAX

Sintaxe:

```
MAX( [ DISTINCT | ALL ] <exp> )
```

Descrição:

Função de agregação que retorna o maior valor de `exp`.

Exemplos:

```
SELECT MAX( VrSalario ) FROM Cargos ;
```

Resultado:

```
MAX(VRSALARIO)
--------------
          2300
```

MIN

Sintaxe:

```
MIN( [ DISTINCT | ALL ] <exp> )
```

Descrição:

Função de agregação que retorna o menor valor de `exp`.

Exemplos:

```
SELECT MIN( VrSalario ) FROM Cargos ;
```

Resultado:

```
MIN(VRSALARIO)
--------------
           350
```

MOD

Sintaxe:

```
MOD( <expN1>, <expN2> )
```

Descrição:

Calcula o resto da divisão de `expN1` por `expN2`

Exemplos:

```
SELECT MOD( 10, 7 ) FROM Dual ;
```

Resultado:

```
                            MOD(10,7)
                            ---------
                                    3
```

```
SELECT MOD( 10, 2 ) FROM Dual ;
```

Resultado:

```
                            MOD(10,2)
                            ---------
                                    0
```

```
SELECT MOD( 2, 5 ) FROM Dual ;
```

Resultado:

```
                             MOD(2,5)
                            ---------
                                    2
```

MONTHS_BETWEEN

Sintaxe:

```
MONTHS_BETWEEN( <expD1>, <expD2> )
```

Descrição:

Determina o número de meses existente entre as datas `expD1` e `expD2`.

Exemplos:

```
SELECT MONTHS_BETWEEN( '15-MAR-00', '01-JAN-00' ) FROM Dual ;
```

Resultado:

```
             MONTHS_BETWEEN('15-MAR-00','01-JAN-00')
             ---------------------------------------
                                          2,4516129
```

NEXT_DAY

Sintaxe:

```
NEXT_DAY( <expD>, <expC> )
```

Descrição:

Retorna a data do primeiro dia de semana correspondente a `expC` que seja igual ou posterior a `expD`.

Exemplos:

```
SELECT NEXT_DAY( '01-JAN-00', 'SABADO' ) FROM Dual ;
```

Resultado:

```
NEXT_DAY('01-JAN-00','SABADO')
------------------------------
08/01/00
```

Comentários:

É retornada a data 8 de Janeiro de 2000, o primeiro sábado depois de 1º de Janeiro de 2000. O valor do segundo parâmetro de `NEXT_DAY` depende do idioma configurado para o SGBD, que no exemplo dado foi assumido como sendo o português. Caso fosse o inglês, por exemplo, deveríamos informar no segundo parâmetro 'SATURDAY' em vez de 'SABADO'. O formato de exibição da data no resultado pode ser diferente do mostrado aqui, devido a configurações do seu ambiente.

NVL

Sintaxe:

```
NVL( <exp1>, <exp2> )
```

Descrição:

Se `exp1` for nulo esta função retorna o valor de `expN2`, senão retorna o próprio valor de `exp1`. O tipo de dado dos dois parâmetros deve ser compatível, ou seja, se, por exemplo, o primeiro parâmetro for numérico, o segundo parâmetro também deverá ser numérico.

Exemplos:

```
SELECT NmFunc, NVL( CdDepto, 'Avulso' ) FROM Funcionarios ;
```

Resultado:

```
NMFUNC                         NVL(CDDEPTO,'AVULSO')
-----------------------------  --------------------
ANA SILVEIRA                   Avulso
JOAO SAMPAIO                   D002
LUCIO TORRES                   D002
PEDRO PINHEIRO                 D001
ROBERTO PEREIRA                D001
JOSE NOGUEIRA                  D001
RUTH DE SOUZA                  D003
MARIA DA SILVA                 D001
LUIZ DE ALMEIDA                D002
PAULO RODRIGUES                D002
```

Comentários:

Observe que a funcionária ‘ANA SILVEIRA’, que não possui departamento (tem um nulo na coluna `CdDepto`) aparece com o texto ‘Avulso’ na segunda coluna do resultado.

POWER

Sintaxe:

```
POWER( <expN1>, <expN2> )
```

Descrição:

Retorna o valor de `expN1` elevado a `expN2`-ésima potência.

Exemplos:

```
SELECT POWER( 2, 3 ) FROM Dual ;
```

Resultado:

```
 POWER(2,3)
 ----------
          8
```

```
SELECT POWER( 2, 1/3 ) FROM Dual ;
```

Resultado:

```
POWER(2,1/3)
------------
  1,25992105
```

```
SELECT POWER( 9, 1/2 ) FROM Dual ;
```

Resultado:

```
POWER(9,1/2)
------------
           3
```

REPLACE

Sintaxe:

```
REPLACE( <expC1>, <expC2> [, <expC3> ] )
```

Descrição:

Retorna o conteúdo de `expC1` com todas suas ocorrências de caracteres de `expC2` substituídas pelos caracteres de `expC3`. Caso o valor do parâmetro `expC3` seja omitido, a função elimina todas a ocorrências de `expC2`.

Exemplos:

```
SELECT REPLACE( 'ROMEU', 'EU', 'ARIA' ) FROM Dual ;
```

Resultado:

```
REPLACE('ROMEU','EU,'ARIA')
---------------------------
ROMARIA
```

```
SELECT REPLACE( 'AxBxCxD', 'x' ) FROM Dual ;
```

Resultado:

```
REPLACE('AXBXCXD','X')
----------------------
ABCD
```

ROUND – para números

Sintaxe:

```
ROUND( <expN1> [, <expN2> ] )
```

Descrição:

Retorna o valor de `expN1` arredondado com `expN2` casas decimais. Se, entretanto, `expN2` for um número negativo, o arredondamento ocorre apenas na parte à esquerda do ponto decimal; se `expN2` for zero ou for omitido, o arredondamento é feito para o valor inteiro mais próximo.

Exemplos:

```
SELECT ROUND( 741.1592, 2 ) FROM Dual ;
```

Resultado:

```
ROUND(741.1592,2)
-----------------
           741,16
```

Comentários:

O número original possui quatro casas depois da vírgula, e no segundo parâmetro foi informado o número 2. O resultado é o número original arredondado com duas casas após a vírgula.

```
SELECT ROUND( 741.1592, -1) FROM Dual ;
```

Resultado:

```
ROUND(741.1592,-1)
------------------
               740
```

Comentários:

No segundo parâmetro foi informado o valor -1, que é um número negativo. A função `ROUND` vai então arredondar o número informado no primeiro parâmetro considerando a dezena mais próxima desse valor.

```
SELECT ROUND( 741.1592, -2) FROM Dual ;
```

Resultado:

```
ROUND(741.1592,-2)
------------------
               700
```

Comentários:

No segundo parâmetro foi informado o valor -2, que é um número negativo. A função `ROUND` vai então arredondar o número informado no primeiro parâmetro considerando a centena mais próxima desse valor.

```
SELECT ROUND( 741.1592 ) FROM Dual ;
```

Resultado:

```
ROUND(741.1592)
---------------
            741
```

Comentários:

Não é informado um valor para o segundo parâmetro e, nesse caso, é assumido o valor zero. A função `ROUND` vai então arredondar o número informado no primeiro parâmetro considerando a unidade mais próxima desse valor. Para o valor utilizado no exemplo, observe que o arredondamento é feito para baixo.

```
SELECT ROUND( 741.6 ) FROM Dual ;
```

Resultado:

```
ROUND(741.6)
------------
         742
```

Comentários:

Não é informado um valor para o segundo parâmetro e, nesse caso, é assumido o valor zero. A função `ROUND` vai então arredondar o número informado no primeiro parâmetro considerando a unidade mais próxima desse valor. Para o valor utilizado no exemplo, observe que o arredondamento é feito para cima.

ROUND – para datas

Sintaxe:

```
ROUND( <expD> [, <expC> ] )
```

Descrição:

Retorna `expD` arredondada como indicado no formato especificado pelo parâmetro `expC`.

Exemplos:

```
SELECT ROUND( TO_DATE( '05-MAR-00' ), 'YEAR' ) FROM Dual ;
```

Resultado:

```
ROUND(TO_DATE('05-MAR-00'),'YEAR')
----------------------------------
01/01/00
```

Comentários:

A chamada da função `ROUND` informa uma data e o argumento `'YEAR'`, indicando que se quer fazer um arredondamento daquela data em nível do ano. É então retornada a data 1º de Janeiro de 2000, que é o primeiro dia do ano referente à data informada como parâmetro que, por ocorrer na primeira metade do ano, leva o arredondamento a ser realizado 'para baixo' ou 'para menos'. O formato de exibição da data no resultado pode ser diferente do mostrado aqui, devido a configurações do seu ambiente.

```
SELECT ROUND( TO_DATE( '05-JUL-00' ), 'YEAR' ) FROM Dual ;
```

Resultado:

```
ROUND(TO_DATE('05-JUL-00'),'YEAR')
----------------------------------
01/01/01
```

Comentários:

A chamada da função `ROUND` informa uma data e o argumento `'YEAR'`, indicando que se quer fazer um arredondamento daquela data em nível do ano. É então retornada a data 1º de Janeiro de 2001, que é o primeiro dia do ano seguinte à data informada como parâmetro que, por ocorrer na segunda metade do ano, leva o arredondamento a ser realizado 'para cima' ou 'para mais'. O formato de exibição da data no resultado pode ser diferente do mostrado aqui, devido a configurações do seu ambiente.

```
SELECT ROUND( TO_DATE( '05-MAR-00' ), 'MONTH' ) FROM Dual ;
```

Resultado:

```
ROUND(TO_DATE('05-MAR-00'),'MONTH')
-----------------------------------
01/03/00
```

Comentários:

A chamada da função `ROUND` informa uma data e o argumento `'MONTH'`, indicando que se quer fazer um arredondamento daquela data em nível do mês. É então retornada a data 1º de Março de 2000, que é o primeiro dia do mês referente à data informada como parâmetro que, por ocorrer na primeira metade do mês, leva o arredondamento a ser realizado 'para baixo' ou 'para menos'. O formato de exibição da data no resultado pode ser diferente do mostrado aqui, devido a configurações do seu ambiente.

```
SELECT ROUND( TO_DATE( '16-MAR-00' ), 'MONTH' ) FROM Dual ;
```

Resultado:

```
ROUND(TO_DATE('05-MAR-00'),'MONTH')
-----------------------------------
01/04/00
```

Comentários:

A chamada da função `ROUND` informa uma data e o argumento `'MONTH'`, indicando que se quer fazer um arredondamento daquela data em nível do mês. É então retornada a data 1º de Abril de 2000, que é o primeiro dia do mês seguinte à data informada como parâmetro que, por ocorrer na segunda metade do mês, leva o arredondamento a ser realizado 'para cima' ou 'para mais'. O formato de exibição da data no resultado pode ser diferente do mostrado aqui, devido a configurações do seu ambiente.

RPAD

Sintaxe:

```
RPAD( <expC1>, <expN> [, <expC2> ] )
```

Descrição:

Retorna o conteúdo de `expC1` preenchido à direita com `expN` caracteres de `expC2`. Caso `expC2` não seja informada a função preenche o restante com espaços em branco.

Exemplos:

```
SELECT RPAD( 'abc', 8, '*' ) FROM Dual ;
```

Resultado:

```
RPAD('ABC',8,'*')
-----------------
abc*****
```

Comentários:

A string original indicada no primeiro parâmetro é convertida para uma string de 8 caracteres, conforme indicado no segundo parâmetro, sendo que os caracteres necessários para completar o comprimento desejado são asteriscos, conforme indicado no terceiro parâmetro.

```
SELECT RPAD( 'abc', 10, '*x' ) FROM Dual ;
```

Resultado:

```
RPAD('ABC',10,'*X')
-------------------
abc*x*x* x*
```

Comentários:

A string original indicada no primeiro parâmetro é convertida para uma string de 10 caracteres, conforme indicado no segundo parâmetro, sendo que os caracteres necessários para completar o comprimento desejado são uma combinação de asterisco seguido de uma letra xis, conforme indicado no terceiro parâmetro.

```
SELECT RPAD( 'abc', 8 ) FROM Dual ;
```

Resultado:

```
RPAD('ABC',8)
-------------
abc
```

Comentários:

A string original indicada no primeiro parâmetro é convertida para uma string de 8 caracteres, conforme indicado no segundo parâmetro, sendo utilizados espaços em branco para completar o comprimento desejado, já que não foi indicado um valor para o terceiro parâmetro da função.

```
SELECT RPAD( 'abcdefghijk', 8 ) FROM Dual ;
```

Resultado:

```
RPAD('ABCDEFGHIJK',8)
---------------------
abcdefgh
```

Comentários:

A string original indicada no primeiro parâmetro é convertida para uma string de 8 caracteres, conforme indicado no segundo parâmetro. Como o comprimento da string original (que é 11) é menor que o comprimento da string desejada (que é 8), os últimos 3 caracteres são eliminados, ou truncados, do resultado.

RTRIM

Sintaxe:

```
RTRIM( <expC1> [, <expC2> ] )
```

Descrição:

Elimina todos os caracteres à direita de `expC1` que sejam iguais a `expC2`. Caso o parâmetro `expC2` não seja fornecido, os espaços em branco finais são eliminados.

Exemplos:

```
SELECT RTRIM( '---Programa---', '-' ) FROM Dual ;
```

Resultado:

```
RTRIM('---PROGRAMA---','-')
---------------------------
---Programa
```

Comentários:

Os hífens indicados no segundo parâmetro são eliminados da parte final da string fornecida no primeiro parâmetro.

```
SELECT RTRIM( '  -Programa-   ' ) FROM Dual ;
```

Resultado:

```
RTRIM('  -PROGRAMA-   ')
------------------------
  -Programa-
```

Comentários:

Os espaços em branco são eliminados da parte final da string fornecida no primeiro parâmetro, já que não foi indicado um valor para o segundo parâmetro.

SIGN

Sintaxe:

```
SIGN( <expN> )
```

Descrição:

Se `expN` for negativo retorna -1, se for positivo retorna 1, se for zero retorna 0.

Exemplos:

```
SELECT SIGN( 5432 ) FROM Dual ;
```

Resultado:

```
SIGN(5432)
----------
         1
```

SOUNDEX

Sintaxe:

```
SOUNDEX( <expC> )
```

Descrição:

Retorna uma representação fonética de cada palavra em `expC`. O uso desta função requer cuidado, pois ela utiliza apenas a fonética da língua inglesa.

Exemplos:

```
SELECT * FROM Deptos
   WHERE SOUNDEX( NmDepto ) = SOUNDEX( 'OFISSINA' ) ;
```

Resultado:

```
CDDEPTO  NMDEPTO               RAMAL
-------  --------------------  -----
D002     OFICINA               310
```

Comentários:

Ao executar a consulta do exemplo, é gerada uma representação fonética para a palavra 'OFISSINA', contendo uma cadeia de caracteres relacionada com a forma como essa palavra é pronunciada na língua inglesa. Então, para cada valor da coluna `NmDepto` da tabela `Deptos` é produzida a correspondente representação fonética, que é comparada com a desejada. O resultado prático é que os departamentos cujo nome é pronunciado de forma semelhante à palavra 'OFISSINA' são listados.

SQRT

Sintaxe:

```
SQRT( <expN> )
```

Descrição:

Retorna a raiz quadrada de `expN`; se o valor de `expN` for negativo, um código de erro é retornado.

Exemplos:

```
SELECT SQRT( 144 ) FROM Dual ;
```

Resultado:

```
 SQRT(144)
----------
        12
```

```
SELECT SQRT( -5 ) FROM Dual ;
```

Resultado:

```
SELECT SQRT( -5 ) FROM Dual
             *
ERRO na linha 1:
ORA-01428: o argumento '-5' está fora da faixa válida
```

STDDEV

Sintaxe:

```
STDDEV( [ DISTINCT | ALL ] <expN> )
```

Descrição:

Função de agregação que retorna o desvio padrão (em inglês, *standard deviation*) de `expN`, desconsiderando-se seus eventuais valores nulos. O desvio padrão é uma medida para expressar a dispersão estatística de valores em torno da média. Os modificadores `DISTINCT` e `ALL` indicam respectivamente se devem ser desprezadas ou não as repetições de valores contidos em `expN`.

Exemplos:

```
SELECT STDDEV( VrSalario ) FROM Cargos ;
```

Resultado:

```
STDDEV(VRSALARIO)
-----------------
       735,357056
```

Comentários:

A consulta produz o valor correspondente ao desvio padrão dos salários cadastrados na tabela de cargos, considerando todos os cargos.

```
SELECT STDDEV( VrSalario )
   FROM Cargos
   WHERE CdCargo IN ( SELECT CdCargo FROM Funcionarios ) ;
```

Resultado:

```
STDDEV(VRSALARIO)
-----------------
       802,651855
```

Comentários:

A consulta produz o valor correspondente ao desvio padrão dos salários dos cargos cadastrados, mas considerando apenas os cargos que possuem funcionários.

```
SELECT STDDEV( VrSalario )
   FROM Funcionarios f
           INNER JOIN Cargos c ON f.CdCargo = c.CdCargo ;
```

Resultado:

```
STDDEV(VRSALARIO)
-----------------
       555,902669
```

Comentários:

A consulta produz o valor correspondente ao desvio padrão dos salários pagos pela empresa, considerando a quantidade de funcionários de cada cargo, já que é feita a junção das tabelas `Cargos` e `Funcionarios`.

```
SELECT STDDEV( DISTINCT VrSalario )
   FROM Funcionarios f
           INNER JOIN Cargos c ON f.CdCargo = c.CdCargo ;
```

Resultado:

```
STDDEV(DISTINCTVRSALARIO)
-------------------------
                802,651855
```

Comentários:

Como é utilizado o modificador `DISTINCT` na função `STDDEV`, a consulta produz o valor correspondente ao desvio padrão dos salários pagos pela empresa, considerando os valores dos salários sem repetições, ou seja, um mesmo salário que é pago a dois funcionários distintos é considerado apenas uma vez no cálculo. Observe que este resultado é igual ao obtido no segundo exemplo, mostrado anteriormente, pois na tabela de cargos não há dois cargos distintos com o mesmo salário.

```
SELECT STDDEV( ALL VrSalario )
   FROM Funcionarios f
           INNER JOIN Cargos c ON f.CdCargo = c.CdCargo ;
```

Resultado:

```
STDDEV(ALLVRSALARIO)
--------------------
          555,902669
```

Comentários:

Como é utilizado o modificador `ALL` na função `STDDEV`, a consulta produz o valor correspondente ao desvio padrão dos salários pagos pela empresa, considerando os valores dos salários preservando repetições, ou seja, um mesmo salário que é pago a dois funcionários distintos é considerado duas vezes no cálculo. Observe que este resultado é igual ao obtido no terceiro exemplo, mostrado anteriormente, pois quando não especificamos os modificadores `ALL` ou `DISTINCT`, `ALL` é assumido como padrão.

SUBSTR

Sintaxe:

```
SUBSTR( <expC> , <expN1> [, <expN2> ] )
```

Descrição:

Retorna a parte de `expC` que começa na posição `expN1` e que se estende por `expN2` caracteres. Caso `expN2` não seja fornecida, a substring retornada será aquela iniciada em `expN1` se estendendo até o final da string original contida em `expC`.

Exemplos:

```
SELECT SUBSTR( 'Programação', 4, 5 ) FROM Dual;
```

Resultado:

```
SUBSTR('PROGRAMAÇÃO',4,5)
-------------------------
grama
```

```
SELECT SUBSTR( 'Programação', 5 ) FROM Dual;
```

Resultado:

```
SUBSTR('PROGRAMAÇÃO',5)
-----------------------
ramação
```

SUM

Sintaxe:

```
SUM( [ DISTINCT | ALL ] <expN> )
```

Descrição:

Função de agregação que retorna a soma dos valores de `expN`, ignorando seus eventuais valores nulos. Os modificadores `DISTINCT` e `ALL` indicam respectivamente se devem ser desprezadas ou não as repetições de valores contidos em `expN`.

Exemplos:

```
SELECT SUM( VrSalario ) FROM Cargos ;
```

Resultado:

```
SUM(VRSALARIO)
--------------
          4950
```

Comentários:

A consulta produz a soma dos salários de todos os cargos cadastrados na tabela `Cargos`.

```
SELECT SUM( VrSalario )
   FROM Cargos
   WHERE CdCargo IN ( SELECT CdCargo FROM Funcionarios ) ;
```

Resultado:

```
SUM(VRSALARIO)
--------------
          4450
```

Comentários:

A consulta produz a soma dos salários dos cargos cadastrados, mas considerando apenas os cargos que possuem funcionários.

```
SELECT SUM( VrSalario )
   FROM Funcionarios f
            INNER JOIN Cargos c ON f.CdCargo = c.CdCargo ;
```

Resultado:

```
SUM(VRSALARIO)
--------------
          7750
```

Comentários:

A consulta produz o valor correspondente ao total de salários pagos pela empresa, considerando a quantidade de funcionários de cada cargo, já que é feita a junção das tabelas `Cargos` e `Funcionarios`.

```
SELECT SUM( DISTINCT VrSalario )
   FROM Funcionarios f
           INNER JOIN Cargos c ON f.CdCargo = c.CdCargo ;
```

Resultado:

```
SUM(DISTINCTVRSALARIO)
----------------------
                  4450
```

Comentários:

Como é utilizado o modificador `DISTINCT` na função `SUM`, a consulta produz o valor correspondente à soma dos salários pagos pela empresa, considerando os valores dos salários sem repetições, ou seja, um mesmo salário que é pago a dois funcionários distintos é considerado apenas uma vez no cálculo. Observe que este resultado é igual ao obtido no segundo exemplo, mostrado anteriormente, pois na tabela de cargos não há dois cargos distintos com o mesmo salário.

```
SELECT SUM( ALL VrSalario )
   FROM Funcionarios f
           INNER JOIN Cargos c ON f.CdCargo = c.CdCargo ;
```

Resultado:

```
SUM(ALLVRSALARIO)
-----------------
             7750
```

Comentários:

Como é utilizado o modificador `ALL` na função `SUM`, a consulta produz o valor correspondente à soma dos salários pagos pela empresa, considerando os valores dos salários preservando repetições, ou seja, um mesmo salário que é pago a dois funcionários distintos é considerado duas vezes no cálculo. Observe que este resultado é igual ao obtido no terceiro exemplo, mostrado anteriormente, pois quando não especificamos os modificadores `ALL` ou `DISTINCT`, `ALL` é assumido como padrão.

TO_CHAR – para números

Sintaxe:

```
TO_CHAR( <expN> [, <expC> ] )
```

Descrição:

Converte o valor numérico contido em `expN` para um valor do tipo de dado caracter no formato especificado em `expC`. Alguns caracteres possuem significado especial como símbolos de formatação e devem ser usados para compor a máscara que determinará no segundo parâmetro o formato desejado para a conversão. O quadro a seguir indica os principais desses símbolos e o seu significado.

Símbolo	***Significado***
`9`	Um dígito ou espaço em branco na posição correspondente do resultado.
`0`	Um dígito ou um zero na posição correspondente do resultado.
`$`	Um cifrão é colocado no início do resultado.
`L`	Um símbolo monetário, que varia conforme a configuração e o idioma do banco, é colocado no início do resultado.
`MI`	Caso o valor numérico seja negativo, um hífen é colocado à direita do resultado.
`PR`	Caso o valor numérico seja negativo, o resultado é colocado entre os delimitadores '<' e '>'.
`G`	É colocado o caracter separador de milhares no resultado final. Esse caracter é dependente da configuração do SGBD.
`D`	É colocado o caracter separador da parte fracionária no resultado final. Esse caracter é dependente da configuração do SGBD.
`.`	Representa o ponto decimal, separador da parte fracionária de um número real.
`RN`	O valor é convertido utilizando a notação de numeração romana.
`E`	O valor é convertido utilizando a notação científica no resultado.

Exemplos:

```
SELECT TO_CHAR( 123, '99999' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(123,'99999')
--------------------
   123
```

Comentários:

É produzida uma string contendo o valor numérico convertido para texto, com espaços em branco sendo utilizados para completar o comprimento desejado para a string. O símbolo '9', quando utilizado em uma máscara significa 'um dígito ou um espaço em branco nesta posição'.

```
SELECT TO_CHAR( 2563, '9999999999' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(2563,'9999999999')
--------------------------
         2563
```

Comentários:

O comprimento da string resultante da conversão é determinado pela quantidade de símbolos de formatação especificados na máscara. No exemplo há apenas 10 símbolos '9' na máscara, e o valor convertido possui 4 caracteres de comprimento ('2563'), sendo necessários portanto 6 caracteres adicionais para completar o comprimento desejado. Nesse caso, como o símbolo '9' foi o único utilizado, são colocados espaços em branco no início da string resultante, até que se complete o tamanho requerido. Observe que, por padrão, um espaço em branco adicional é colocado no início do resultado, o que faz com que nossa string resultante tenha 11 caracteres de comprimento, em vez dos 10 caracteres sugeridos pela máscara.

```
SELECT TO_CHAR( 123, '$99999' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(123,'$99999')
---------------------
    $123
```

Comentários:

O valor é convertido com um cifrão no início.

```
SELECT TO_CHAR( 123, 'L99999' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(123,'L99999')
---------------------
               R$123
```

Comentários:

O valor é convertido com um símbolo de moeda, que é dependente da configuração do ambiente, no início do resultado.

```
SELECT TO_CHAR( 12.34, '99999' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(12.34,'99999')
----------------------
    12
```

Comentários:

É produzida uma string contendo o valor numérico convertido para texto, com espaços em branco sendo utilizados para completar o comprimento desejado para a string. Como no exemplo o valor fornecido para ser convertido possui valores após o ponto decimal e a máscara utilizada não prevê o ponto decimal, há o truncamento do número no resultado.

```
SELECT TO_CHAR( 123, '999.99' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(123,'999.99')
---------------------
 123.00
```

Comentários:

O valor numérico é convertido para texto, com três casas antes do ponto decimal e duas casas depois do ponto decimal. O que faltar para completar as casas na parte inteira é preenchido com espaços em branco; o que faltar para completar as casas na parte decimal é preenchido com zeros. Em vez de se utilizar o símbolo '.' (ponto) pode-se utilizar em seu lugar o símbolo '`D`', que fará com que no resultado seja colocado o símbolo separador de decimais padrão definido para o idioma local.

```
SELECT TO_CHAR( 12.34, '9999.9' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(12.34,'9999.9')
----------------------
   12.3
```

Comentários:

No exemplo, houve o truncamento na parte fracionária do valor convertido, pois o número original possuía dois dígitos significativos após a vírgula e a máscara fornecida só previa uma casa decimal.

```
SELECT TO_CHAR( 123, '00000' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(123,'00000')
--------------------
 00123
```

Comentários:

É produzida uma string contendo o valor numérico convertido para texto, com zeros sendo utilizados para completar à esquerda o comprimento desejado para a string. O símbolo '`0`', quando utilizado em uma máscara significa 'um dígito ou um zero nesta posição'. Observe que, por padrão, a string resultante possui um espaço em branco adicional, posicionado em seu início.

```
SELECT TO_CHAR( 123, 'FM00000' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(123,'FM00000')
----------------------
00123
```

Comentários:

O símbolo '`FM`' (*fill mode*) indica que não desejamos o espaço em branco inicial que é colocado por padrão no resultado da conversão.

```
SELECT TO_CHAR( -123, '9999MI' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(-123,'9999MI')
----------------------
 123-
```

Comentários:

Como o valor a ser convertido é negativo e o símbolo '`MI`' foi especificado na máscara, um hífen (sinal de negativo) é colocado no resultado. Caso o valor fosse positivo e a máscara contivesse o '`MI`' um espaço em branco seria colocado no lugar do hífen.

```
SELECT TO_CHAR( -123, '9999PR' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(-123,'9999PR')
----------------------
 <123>
```

Comentários:

Como o valor a ser convertido é negativo e o símbolo '`PR`' foi especificado na máscara, o resultado aparece entre os delimitadores '`<`' e '`>`'. Caso o valor fosse positivo e a máscara contivesse o '`PR`' um espaço em branco seria colocado no lugar de cada delimitador.

```
SELECT TO_CHAR( 1234567.89, '999G999G999D999' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(1234567.89,'999G999G999D999')
-------------------------------------
   1.234.567,890
```

Comentários:

O exemplo utiliza os símbolos de separadores de milhares e da parte decimal. No caso, é utilizado o ponto para separar os milhares e a vírgula para separar a parte decimal devido à configuração do SGBD para o idioma português do Brasil. O formato do resultado depende da configuração do ambiente.

```
SELECT TO_CHAR( 123, 'RN' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(123,'RN')
-----------------
          CXXIII
```

Comentários:

O valor é convertido para texto, com a numeração no formato romano.

```
SELECT TO_CHAR( 123, '9.9EEEE' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(123,'9.9EEEE')
----------------------
  1.2E+02
```

Comentários:

O valor é convertido adotando-se a formatação científica.

TO_CHAR – para datas e horas

Sintaxe:

```
TO_CHAR( <expD> [, <expC> ] )
```

Descrição:

Converte o valor de data e/ou hora contido em `expD` para um valor do tipo de dado caracter no formato especificado em `expC`. Alguns caracteres possuem significado especial como símbolos de formatação e devem ser usados para compor a máscara que determinará no segundo parâmetro o formato desejado para a conversão. O quadro a seguir indica os principais desses símbolos e o seu significado.

Símbolo	***Significado***
`DD`	Número do dia do mês, pode variar de 1 a 31.
`D`	Número do dia da semana, pode variar de 1 (domingo) a 7 (sábado).
`DDD`	Número do dia do ano, pode varia de 1 a 365 ou 366 (dependendo de o ano ser bissexto ou não).
`DAY`	Nome do dia da semana.
`MM`	Número do mês, pode variar de 1 (Janeiro) a 12 (Dezembro).
`MON`	Iniciais (com 3 letras) do nome do mês.
`MONTH`	Nome do mês por extenso.
`YY`	Ano representado com dois dígitos.
`YYYY`	Ano representado com quatro dígitos.
`HH ou HH12`	Hora do dia variando de 1 (uma hora da manhã ou uma hora da tarde) até 12 (meio-dia ou meia-noite).
`HH24`	Hora do dia variando de 0 a 23.
`MI`	Minutos, variando de 0 a 59.
`SS`	Segundos, variando de 0 a 59.
`SSSSS`	Quantidade de segundos decorridos desde o início do dia, variando de 1 até 86399.

Exemplos:

```
SELECT TO_CHAR( SYSDATE, 'DD' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(SYSDATE,'DD')
---------------------
01
```

Comentários:

Para este exemplo, considerar a data do sistema como sendo 1º de Janeiro de 2000.

```
SELECT TO_CHAR( SYSDATE, 'D' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(SYSDATE,'D')
--------------------
7
```

Comentários:

Para este exemplo, considerar a data do sistema como sendo 1º de Janeiro de 2000, que foi um sábado.

```
SELECT TO_CHAR( SYSDATE, 'DDD' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(SYSDATE,'DDD')
----------------------
001
```

Comentários:

Para este exemplo, considerar a data do sistema como sendo 1º de Janeiro de 2000, que foi o primeiro dia do ano.

```
SELECT TO_CHAR( SYSDATE, 'DAY' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(SYSDATE,'DAY')
----------------------
SÁBADO
```

Comentários:

Para este exemplo, considerar a data do sistema como sendo 1º de Janeiro de 2000, que foi um sábado. O nome do dia da semana vai variar conforme a configuração do ambiente. Em instalações configuradas sob o padrão americano, por exemplo, seria exibido 'SATURDAY' no resultado, em vez de 'SÁBADO'.

```
SELECT TO_CHAR( SYSDATE, 'DD/MM/YYYY' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(SYSDATE,'DD/MM/YYYY')
-----------------------------
01/01/2000
```

Comentários:

Para este exemplo, considerar a data do sistema como sendo 1º de Janeiro de 2000.

```
SELECT TO_CHAR( SYSDATE, 'DD/MON/YYYY' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(SYSDATE,'DD/MON/YYYY')
------------------------------
01/JAN/2000
```

Comentários:

Para este exemplo, considerar a data do sistema como sendo 1º de Janeiro de 2000. As iniciais do nome do mês vão variar conforme a configuração do ambiente. Em instalações configuradas sob o padrão espanhol, por exemplo, seria exibido 'ENE' (de 'Enero', como é chamado o mês de Janeiro nesse idioma) no resultado, em vez de 'JAN'.

```
SELECT TO_CHAR( SYSDATE, 'DD/MONTH/YYYY' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(SYSDATE,'DD/MONTH/YYYY')
--------------------------------
01/JANEIRO  /2000
```

Comentários:

Para este exemplo, considerar a data do sistema como sendo 1º de Janeiro de 2000. O nome do mês pode variar conforme a configuração do ambiente. Em instalações configuradas sob o padrão espanhol, por exemplo, seria exibido 'ENERO' no resultado, em vez de 'JANEIRO'.

```
SELECT TO_CHAR( SYSDATE, 'DD/FMMONTH/YYYY' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(SYSDATE,'DD/FMMONTH/YYYY')
---------------------------------
01/JANEIRO/2000
```

Comentários:

Para este exemplo, considerar a data do sistema como sendo 1º de Janeiro de 2000. Observe que o uso do símbolo de formatação '`FM`' eliminou os espaços em branco finais do nome do mês.

```
SELECT TO_CHAR( SYSDATE, 'FMDAY", "FMDD" de "FMMONTH" de "FMYYYY' )
   FROM Dual ;
```

Resultado:

```
TO_CHAR(SYSDATE,'FMDAY","FMDD"de"FMMONTH"de"FMYYYY')
---------------------------------------------------
SÁBADO, 01 de JANEIRO de 2000
```

Comentários:

Para este exemplo, considerar a data do sistema como sendo 1º de Janeiro de 2000. Observe que é possível utilizar caracteres especiais e palavras na máscara, desde que eles estejam delimitados por aspas duplas.

```
SELECT TO_CHAR( SYSDATE, 'DD/MM/YY HH24:MI:SS' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(SYSDATE,'DD/MM/YYHH24:MI:SS')
-------------------------------------
01/01/00 20:58:30
```

Comentários:

Para este exemplo, considerar a data do sistema como sendo 20 horas, 58 minutos e 30 segundos do dia 1º de Janeiro de 2000.

```
SELECT TO_CHAR( SYSDATE, 'DD/MM/YY HH:MI:SS' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(SYSDATE,'DD/MM/YYHH:MI:SS')
-----------------------------------
01/01/00 08:58:30
```

Comentários:

Para este exemplo, considerar a data do sistema como sendo 20 horas, 58 minutos e 30 segundos do dia 1º de Janeiro de 2000.

```
SELECT TO_CHAR( SYSDATE, 'DD/MM/YY HH:MI:SS PM' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(SYSDATE,'DD/MM/YYHH:MI:SS PM')
--------------------------------------
01/01/00 08:58:30 TARDE
```

Comentários:

Para este exemplo, considerar a data do sistema como sendo 20 horas, 58 minutos e 30 segundos do dia 1º de Janeiro de 2000.

```
SELECT TO_CHAR( SYSDATE, 'SSSSS' ) FROM Dual ;
```

Resultado:

```
TO_CHAR(SYSDATE,'SSSSS')
------------------------
75510
```

Comentários:

Para este exemplo, considerar a data do sistema como sendo 20 horas, 58 minutos e 30 segundos do dia 1º de Janeiro de 2000. O resultado apresentado significa que, nesse momento, terão se passado 75510 segundos desde que o dia começou.

TO_DATE

Sintaxe:

```
TO_DATE( <expC1> [, <expC2> ] )
```

Descrição:

Converte o valor de data e hora indicado em `expC1` e formatado conforme a máscara contida em `expC2`, para o correspondente valor do tipo `DATE`. Para construir a máscara são válidas as mesmas convenções para formato de data e hora utilizadas com a função `TO_CHAR`.

Exemplos:

```
SELECT TO_DATE( '15/JAN/2000', 'DD/MON/YYYY' ) FROM Dual ;
```

Resultado:

```
TO_DATE('15/JAN/2000','DD/MON/YYYY')
------------------------------------
15/01/00
```

Comentários:

A string informada como primeiro parâmetro possui uma informação que deve ser entendida como sendo uma data, cujo formato está descrito na string do segundo parâmetro. O formato que o comando `SELECT` do exemplo utilizará para exibir o resultado depende das configurações do ambiente.

TO_NUMBER

Sintaxe:

```
TO_NUMBER( <expC> )
```

Descrição:

Converte um dado expresso como caracter para o valor correspondente no tipo numérico. Caso o conteúdo a ser convertido possua caracteres que não possuam interpretação numérica, um erro será provocado.

Exemplos:

```
SELECT TO_NUMBER( '1234' ) FROM Dual ;
```

Resultado:

```
                       TO_NUMBER('1234')
                       -----------------
                                    1234
```

```
SELECT TO_NUMBER( '-1234' ) FROM Dual ;
```

Resultado:

```
                      TO_NUMBER('-1234')
                      ------------------
                                   -1234
```

```
SELECT TO_NUMBER( '+1234' ) FROM Dual ;
```

Resultado:

```
                      TO_NUMBER('+1234')
                      ------------------
                                    1234
```

```
SELECT TO_NUMBER( '1,234' ) FROM Dual ;
```

Resultado:

```
                      TO_NUMBER('1,234')
                      ------------------
                                   1,234
```

```
SELECT TO_NUMBER( '12C34' ) FROM Dual ;
```

Resultado:

```
SELECT TO_NUMBER( '12C34' ) FROM Dual
                   *
ERRO na linha 1:
ORA-01722: número inválido
```

TRANSLATE

Sintaxe:

```
TRANSLATE( <expC1>, <expC2>, <expC3> )
```

Descrição:

Retorna o conteúdo de `expC1` com todas as ocorrências de `expC2` substituídas por `expC3`.

Exemplos:

```
SELECT TRANSLATE( 'MARIO', 'O', 'A' ) FROM Dual ;
```

Resultado:

```
TRANSLATE('MARIO','O','A')
--------------------------
MARIA
```

```
SELECT TRANSLATE( 'Banana', 'n', 't' ) FROM Dual ;
```

Resultado:

```
TRANSLATE('Banana','n','t')
---------------------------
Batata
```

TRUNC – para números

Sintaxe:

```
TRUNC( <expN1> [, <expN2> ] )
```

Descrição:

Retorna `expN1` truncado na `expN2`-ésima casa decimal; se o segundo parâmetro for omitido ou for zero, trunca o valor do primeiro parâmetro para o seu valor inteiro; se `expN2` for um valor negativo, zera um ou mais dígitos à esquerda do ponto decimal.

Exemplos:

```
SELECT TRUNC( 741.1592, 2 ) FROM Dual ;
```

Resultado:

```
TRUNC(741.1592,2)
-----------------
           741.15
```

Comentários:

O número original possui quatro casas depois da vírgula, e no segundo parâmetro foi informado o número 2. O resultado é o número original truncado, sem arredondamento portanto, com duas casas após a vírgula.

```
SELECT TRUNC( 741.1592, -1 ) FROM Dual ;
```

Resultado:

```
TRUNC(741.1592,-1)
------------------
               740
```

Comentários:

No segundo parâmetro foi informado o valor -1, que é um número negativo. A função `TRUNC` vai então truncar o número informado no primeiro parâmetro considerando a dezena correspondente a esse valor.

```
SELECT TRUNC( 741.1592, -2 ) FROM Dual ;
```

Resultado:

```
TRUNC(741.1592,-2)
------------------
               700
```

Comentários:

No segundo parâmetro foi informado o valor -2, que é um número negativo. A função `TRUNC` vai então truncar o número informado no primeiro parâmetro considerando a centena correspondente a esse valor.

```
SELECT TRUNC( 741.1592 ) FROM Dual ;
```

Resultado:

```
TRUNC(741.1592)
---------------
            741
```

Comentários:

Não é informado um valor para o segundo parâmetro e, nesse caso, é assumido o valor zero. A função `TRUNC` vai então arredondar o número informado no primeiro parâmetro considerando a unidade correspondente a esse valor, sem arredondamento.

TRUNC – para datas

Sintaxe:

```
TRUNC( <expD> [, <expC> ] )
```

Descrição:

Retorna o conteúdo de `expD` truncado segundo o formato especificado em `expC`. Caso o valor do segundo parâmetro não seja especificado, o valor retornado será a data contida em expD sem com a parte referente ao horário zerado.

Exemplos:

```
SELECT TRUNC( TO_DATE( '05-MAR-00' ), 'YEAR' ) FROM Dual ;
```

Resultado:

```
TRUNC(TO_DATE('05-MAR-00'),'YEAR')
----------------------------------
01/01/00
```

Comentários:

A data foi truncada com base no ano, retornando a data inicial do ano correspondente.

```
SELECT TRUNC( TO_DATE( '05-JUL-00' ), 'YEAR' ) FROM Dual ;
```

Resultado:

```
TRUNC(TO_DATE('05-JUL-00'),'YEAR')
----------------------------------
01/01/00
```

Comentários:

A data foi truncada com base no ano, retornando a data inicial do ano correspondente.

```
SELECT TRUNC( TO_DATE( '05-MAR-00' ), 'MONTH' ) FROM Dual ;
```

Resultado:

```
TRUNC(TO_DATE('05-MAR-00'),'MONTH')
-----------------------------------
01/03/00
```

Comentários:

A data foi truncada com base no mês, retornando a data inicial do mês correspondente.

```
SELECT TRUNC( TO_DATE( '16-MAR-00' ), 'MONTH' ) FROM Dual ;
```

Resultado:

```
TRUNC(TO_DATE('16-MAR-00'),'MONTH')
-----------------------------------
01/03/00
```

Comentários:

A data foi truncada com base no mês, retornando a data inicial do mês correspondente.

```
SELECT TO_CHAR( TRUNC( SYSDATE ), 'DD/MM/YYYY HH24:MI:SS' )
   FROM Dual ;
```

Resultado:

```
TO_CHAR(TRUNC(SYSDATE),'DD/MM/YYYYHH24:MI:SS')
----------------------------------------------
01/01/2000 00:00:00
```

Comentários:

Para este exemplo, considerar a data do sistema como sendo 20 horas, 58 minutos e 30 segundos do dia 1º de Janeiro de 2000. Observe que a parte da informação correspondente ao horário encontra-se zerada.

UPPER

Sintaxe:

```
UPPER( <expC> )
```

Descrição:

Retorna o conteúdo de `expC` com todas as suas letras em maiúsculas.

Exemplos:

```
SELECT UPPER( 'Dia 5 de Abril' ) FROM Dual ;
```

Resultado:

```
UPPER(DIA5DEABRIL)
------------------
DIA 5 DE ABRIL
```

USERENV

Sintaxe:

```
USERENV( <opção> )
```

Descrição:

Retorna informações da opção escolhida para o usuário e/ou a conexão corrente.

Opção	*Significado*
ENTRYID	Identificador de entrada.
SESSIONID	Número da seção de uso.
TERMINAL	Identificação do terminal do usuário.
LANGUAGE	Linguagem em uso.

Exemplos:

```
SELECT USERENV( 'ENTRYID' ) FROM Dual ;
```

Resultado:

```
 USERENV('ENTRYID')
 ------------------
                  0
```

Comentários:

O valor apresentado no resultado pode ser diferente do mostrado aqui.

```
SELECT USERENV( 'SESSIONID' ) FROM Dual ;
```

Resultado:

```
 USERENV('SESSIONID')
 --------------------
                  867
```

Comentários:

O valor apresentado no resultado pode ser diferente do mostrado aqui.

```
SELECT USERENV( 'TERMINAL' ) FROM Dual ;
```

Resultado:

```
USERENV('TERMINAL')
-------------------
COMPUT01
```

Comentários:

O valor apresentado no resultado pode ser diferente do mostrado aqui.

```
SELECT USERENV( 'LANGUAGE' ) FROM Dual ;
```

Resultado:

```
USERENV('LANGUAGE')
----------------------------------------
BRAZILIAN PORTUGUESE_BRAZIL.WE8MSWIN1252
```

Comentários:

O valor apresentado no resultado pode ser diferente do mostrado aqui.

VARIANCE

Sintaxe:

```
VARIANCE( [ DISTINCT | ALL ] <expN> )
```

Descrição:

Função de agregação que retorna a variância dos valores contidos em `expN`, desconsiderando seus eventuais nulos. A variância é uma medida para expressar a dispersão estatística de valores em torno da média. Os modificadores `DISTINCT` e `ALL` indicam respectivamente se devem ser desprezadas ou não as repetições de valores contidos em `expN`.

Exemplos:

```
SELECT VARIANCE( VrSalario ) FROM Cargos ;
```

Resultado:

```
VARIANCE(VRSALARIO)
-------------------
             540750
```

Comentários:

A consulta produz o valor correspondente à variância dos salários cadastrados na tabela de cargos, considerando todos os cargos.

```
SELECT VARIANCE( VrSalario )
   FROM Cargos
   WHERE CdCargo IN ( SELECT CdCargo FROM Funcionarios ) ;
```

Resultado:

```
VARIANCE(VRSALARIO)
-------------------
             644250
```

Comentários:

A consulta produz o valor correspondente à variância dos salários dos cargos cadastrados, mas considerando apenas os cargos que possuem funcionários.

```
SELECT VARIANCE( VrSalario )
   FROM Funcionarios f
           INNER JOIN Cargos c ON f.CdCargo = c.CdCargo ;
```

Resultado:

```
VARIANCE(VRSALARIO)
-------------------
         309027,778
```

Comentários:

A consulta produz o valor correspondente à variância dos salários pagos pela empresa, considerando a quantidade de funcionários de cada cargo, já que é feita a junção das tabelas `Cargos` e `Funcionarios`.

```
SELECT VARIANCE( DISTINCT VrSalario )
   FROM Funcionarios f
           INNER JOIN Cargos c ON f.CdCargo = c.CdCargo ;
```

Resultado:

```
VARIANCE(DISTINCTVRSALARIO)
---------------------------
                     644250
```

Comentários:

Como é utilizado o modificador `DISTINCT` na função `VARIANCE`, a consulta produz o valor correspondente à variância dos salários pagos pela empresa, considerando os valores dos salários sem repetições, ou seja, um mesmo salário que é pago a dois funcionários distintos é considerado apenas uma vez no cálculo. Observe que este resultado é igual ao obtido no segundo exemplo, mostrado anteriormente, pois na tabela de cargos não há dois cargos distintos com o mesmo salário.

```
SELECT VARIANCE( ALL VrSalario )
   FROM Funcionarios f
           INNER JOIN Cargos c ON f.CdCargo = c.CdCargo ;
```

Resultado:

```
VARIANCE(ALLVRSALARIO)
----------------------
           309027,778
```

Comentários:

Como é utilizado o modificador `ALL` na função `VARIANCE`, a consulta produz o valor correspondente à variância dos salários pagos pela empresa, considerando os valores dos salários preservando repetições, ou seja, um mesmo salário que é pago a dois funcionários distintos é considerado duas vezes no cálculo. Observe que este resultado é igual ao obtido no terceiro exemplo, mostrado anteriormente, pois quando não especificamos os modificadores `ALL` ou `DISTINCT`, `ALL` é assumido como padrão.

VSIZE

Sintaxe:

```
VSIZE( <exp> )
```

Descrição:

Retorna o número de bytes usados para armazenar internamente o conteúdo de `exp`.

Exemplos:

```
SELECT NmFunc, VSIZE( NmFunc )
   FROM Funcionarios
   WHERE NrMatric = '1001' ;
```

Resultado:

```
NMFUNC                VSIZE(NMFUNC)
--------------------  -------------
JOAO SAMPAIO                    12
```

```
SELECT NmCargo, VrSalario, VSIZE( VrSalario )
   FROM Cargos ;
```

Resultado:

```
NMCARGO                VRSALARIO  VSIZE(VRSALARIO)
--------------------   ---------  ----------------
COZINHEIRA                   350                 3
AUX. ESCRITORIO              450                 3
VIGIA                        500                 2
MECANICO                     750                 3
GERENTE                     2300                 2
ESCRITURARIO                 600                 2
```

Comandos DDL SQL

Em SQL temos um conjunto de comandos com a finalidade de permitir a criação, alteração e eliminição de objetos da base de dados. Esse conjunto é chamado de Linguagem de Definição de Dados (em inglês, Data Definition Language - DDL) e é composto basicamente pelos comandos CREATE, ALTER e DROP.

Com eles podemos definir objetos como tabelas, visões, índices e seqüências (entre outros), com pequenas variações na sintaxe de cada comando. Veremos neste material o tratamento referente a tabelas e visões, ficando os demais tipos de objetos para serem trabalhados futuramente.

Tipos de dados no Oracle

Os tipos de dados permitidos para a definição das colunas das tabelas que pretendemos criar variam conforme o SGBD a ser utilizado. Por exemplo, MS-Access, Oracle e SQL Server são gerenciadores de bancos de dados que oferecem seus próprios conjuntos de tipos de dados. Assim, uma coluna para armazenar a data de admissão de um funcionário seria do tipo DataHora para o Access, SmallDateTime para o SQL Server e Date para o Oracle. Como vamos utilizar em nossos exemplos o SGBD Oracle 9 ou superior, veremos inicialmente os seus tipos de dados mais importantes, que aparecem descritos no quadro a seguir.

Tipo	***Pode armazenar ...***
`CHAR( n )`	Quaisquer combinações de caracteres alfabéticos, especiais e dígitos numéricos. O tamanho de um dado do tipo CHAR é sempre fixo, informado no parâmetro n.
`VARCHAR2( n )`	O mesmo tipo de dado que um campo CHAR, exceto que o espaço ocupado pela informação é variável, ocupando no máximo n caracteres.
`NUMBER( t, d )`	Números de todos os tipos, incluindo inteiros e reais. Um dado do tipo NUMBER terá um total de t dígitos, sendo d dígitos à direita do ponto decimal.
`DATE`	Data e hora entre 01/Jan/4712 a.C. e 31/Dec/4712 d.C.
`LONG`	Semelhante a dados VARCHAR2, porém com tamanho máximo de 2 GB.

Regras para nomes no Oracle

Os nomes, tanto de tabelas como de colunas, devem ter no máximo 30 caracteres de extensão e na sua composição são permitidos todos os caracteres alfabéticos; os dígitos numéricos (0 a 9) e os caracteres especiais "_" (sobrescrito, ou *underscore*), o "$" (cifrão ou *dolar*) e o "#" (*sharp*). Um nome deve sempre ter como primeiro caracter uma letra do alfabeto e uma tabela ou coluna não pode ter nome igual a qualquer palavra reservada do Oracle. Um usuário não pode ter dois objetos de banco de dados com o mesmo nome, mesmo que de tipos diferentes. É desaconselhável a utilização de $ e # na composição de nomes.

Restrições (Constraints)

As restrições de integridade são recursos proporcionados pelo SGBD para implementar as regras de consistência mais básicas para os dados de uma tabela, de maneira que as informações nela contidas mantenham o banco de dados numa situação aceitável de integridade lógica. Na prática, é através dessas restrições que especificamos para o SGBD qual o domínio de um dado.

Ao criarmos uma tabela, podemos definir restrições em dois níveis:

> **nível de coluna**: limita os valores possíveis em uma coluna a uma faixa de valores. Uma restrição em nível de coluna é especificada imediatamente a seguir à definição do nome e do tipo da coluna a ser afetada;
>
> **nível de tabela**: limita os valores possíveis em uma combinação de colunas a uma faixa de valores. Uma restrição em nível de tabela é especificada após às definições de todas as colunas da tabela.

O quadro a seguir apresenta as cláusulas previstas pelo SGBD Oracle 7 para implementar as restrições permitidas.

Tipo	***Restrição***	***Significado***
Coluna	`[NOT] NULL`	Permite ou não a inclusão de um valor nulo na coluna.
	`UNIQUE`	O valor da coluna deve ser único dentro da tabela.
	`PRIMARY KEY`	A coluna é a chave primária da tabela.
	`CHECK ( condição )`	O valor da coluna deverá estar de acordo com a condição.
	`REFERENCES tabela[(coluna)]`	Indica que o valor da coluna deverá existir na chave primária de outra tabela. Em outras palavras, a coluna é uma chave estrangeira.
Tabela	`UNIQUE (coluna, ...)`	A combinação dos valores das colunas deve ser única dentro da tabela.
	`PRIMARY KEY (coluna, ...)`	A combinação dos valores das colunas é a chave primária da tabela.
	`CHECK (condição)`	O valor deverá estar de acordo com a condição.
	`FOREIGN KEY (coluna, ...) REFERENCES tabela(coluna)`	Indica que o valor da combinação das colunas deverá existir na chave primária de outra tabela. Em outras palavras, a combinação dos valores das colunas é uma chave estrangeira.

Comando CREATE TABLE

Cria uma tabela, especificando suas colunas e restrições. Uma tabela pode conter até 254 colunas. A sintaxe apresentada a seguir está simplificada, omitindo algumas cláusulas opcionais um pouco mais técnicas que não são importantes neste momento.

```
CREATE TABLE <nome_tabela>
      ( <nome_coluna1> <tipo_dado> [<restrições_C1>],
      ...
      <nome_colunaN> <tipo_dado> [<restrições_CN>] [,
      <restrição_T1> [,
      ...
      <restrição_TN> ] ] ) ;
```

Argumento	*Significado*
<nome_tabela>	Especifica o nome da tabela a ser criada, conforme as regras de nomenclatura do Oracle.
<nome_coluna>	Especifica o nome da coluna a ser criada, conforme as regras de nomenclatura do Oracle.
<tipo_dado>	Tipo de dado da coluna, que deverá ser escolhido dentro do conjunto de tipos de dados válidos. Se necessário, especificar também o tamanho do dado.
<restrições_C>	Restrições de coluna a serem aplicadas para o dado.
<restrições_T>	Restrições de tabela a serem aplicadas.

Exemplos:

```
CREATE TABLE Clientes
      (     IdCliente   NUMBER( 5, 0 )    PRIMARY KEY,
            Sobrenome   VARCHAR2( 50 )    NOT NULL,
            Nome        VARCHAR2( 50 )    NOT NULL,
            Endereco    VARCHAR2( 100 ),
            CEP         VARCHAR2( 8 ),
            UF          CHAR( 2 )         CHECK (UF IN('SP','RJ', 'RS')),
            Fone        VARCHAR2( 20 )    NOT NULL,
            UNIQUE (Sobrenome, Nome) ) ;

CREATE TABLE Pedidos
      (     IdPedido    NUMBER( 5, 0 )    CONSTRAINT PK_Ped PRIMARY KEY,
            IdCliente   NUMBER( 5, 0 )    NOT NULL REFERENCES Clientes,
            Status      CHAR( 1 )         CHECK ( Status IN( 'P', 'R' )),
            DtPedido    DATE              NOT NULL,
            DtEnvio     DATE,
            DtPagto     DATE        ) ;
```

Obs.: Para visualizar os nomes e a estrutura das tabelas existentes no banco de dados, utilize o comando DESCRIBE e as instruções SQL que são apresentadas no final desta apostila.

Comando ALTER TABLE

Altera definições de uma tabela, adicionando e modificando colunas ou adicionando e excluindo restrições.

```
ALTER TABLE <nome_tabela>
      [ADD [<nome_coluna><tipo_dado>[<restrição>C] | <restricaoT>]]
      [MODIFY (<nome_coluna>, ...)]
      [DROP CONSTRAINT <restrição>] ;
```

Argumento	*Significado*
`<nome_tabela>`	Nome da tabela a ser modificada.
`ADD ...`	Adiciona colunas ou restrições (em nível de coluna ou tabela) à tabela.
`MODIFY ...`	Modifica as características de uma coluna.
`DROP CONSTRAINT ...`	Exclui restrição (em nível de coluna ou tabela).

Exemplos:

```
ALTER TABLE Clientes ADD ( fax VARCHAR2( 20 ) NOT NULL ) ;
ALTER TABLE Clientes MODIFY ( fax VARCHAR2( 20 ) NULL ) ;
ALTER TABLE Pedidos  DROP CONSTRAINT PK_Ped ;
ALTER TABLE Pedidos  ADD ( PRIMARY KEY ( IdPedido ) ) ;
```

Comando DROP TABLE

Elimina uma tabela do banco de dados, excluindo todos os seus dados, bem como os índices correspondentes.

```
DROP TABLE <nome_tabela> ;
```

Argumento	*Significado*
`<nome_tabela>`	Nome da tabela a ser eliminada.

Exemplos:

```
DROP TABLE Clientes ;
DROP TABLE Pedidos ;
```

Comando CREATE VIEW

Cria ou modifica uma visão. Uma visão é uma espécie de tabela lógica, onde os seus dados são obtidos dinamicamente a partir das tabelas do banco de dados.

```
CREATE [OR REPLACE] VIEW <nome_visão> [ <nome_coluna1>, ...]
      AS <consulta>
      [WITH CHECK OPTION [CONSTRAINT <restrição>]] ;
```

Argumento	*Significado*
`OR REPLACE`	Se especificado, indica que a definição da visão deverá ser apenas atualizada, caso já exista.
`<nome_visão>`	Especifica o nome da visão a ser criada/modificada, conforme as regras de nomenclatura do Oracle.
`<nome_coluna>`	Nome da coluna na visão.
`AS <consulta>`	Consulta SELECT ... FROM ... WHERE ... que fornecerá os dados a serem exibidos pela visão. O comando SELECT não poderá conter cláusulas ORDER BY ou FOR UPDATE.
`WITH CHECK OPTION`	Se especificado, indica que possíveis inclusões ou modificações realizadas na visão não poderão contrariar a condição da cláusula WHERE da consulta.
`CONSTRAINT`	Atribui um nome à restrição.

Exemplos:

```
CREATE VIEW ClientesSP
      AS SELECT *
                  FROM Clientes
                  WHERE UF = 'SP'
      WITH CHECK OPTION ;


CREATE OR REPLACE VIEW ClientesAux
      AS SELECT IdCliente, Nome, Fone
                  FROM Clientes ;
```

```
CREATE VIEW ClientePedido
      AS SELECT Clientes.IdCliente, Sobrenome, Nome, IdPedido,
                  DtEnvio
                  FROM Clientes, Pedido
                  WHERE Clientes.IdCliente = Pedido.IdCliente
                        AND Pedido.Status = 'P'
                        AND DtEnvio IS NULL
      WITH CHECK OPTION ;


CREATE VIEW ClienteAux2 ( CdCliente, NmCliente )
      AS SELECT IdCliente, Nome || Sobrenome
                  FROM Clientes ;
```

Obs.: Para visualizar os nomes e informações adicionais sobre as visões existentes no banco de dados, utilize as instruções SQL apresentadas no final desta apostila.

Comando DROP VIEW

Elimina uma visão do banco de dados.

```
DROP VIEW <nome_visão> ;
```

Argumento	*Significado*
`<nome_visão>`	Nome da visão a ser eliminada.

Exemplos:

```
DROP VIEW ClientesSP ;
DROP VIEW ClientePedido ;
```

Comando CREATE INDEX

Cria um índice para uma tabela do banco de dados.

```
CREATE [UNIQUE] INDEX <nome_índice>
      ON <nome_tabela> ( <col1> [, ... <coln>] )
```

Argumento	*Significado*
`UNIQUE`	Se utilizado, indica que o índice a ser criado não deve permitir que a tabela possua valores duplicados na expressão chave.
`<nome_índice>`	Nome do índice a ser criado.
`ON <nome_tabela>`	Nome da tabela para a qual o índice será criado.
`<coln>`	Nome de uma coluna da tabela que fará parte da chave do índice.

Exemplos:

```
CREATE INDEX IX_DtAdm ON Funcionario ( DtAdm ) ;
CREATE UNIQUE INDEX Ind_NmFunc ON Funcionario ( NmFunc ) ;
CREATE INDEX DpCgNom ON Funcionario ( CdDepto, CdCargo, NmFunc ) ;
```

Obs.: Para visualizar os nomes e informações adicionais sobre os índices existentes no banco de dados, utilize as instruções SQL apresentadas no final desta apostila.

Comando DROP INDEX

Elimina um índice de uma tabela.

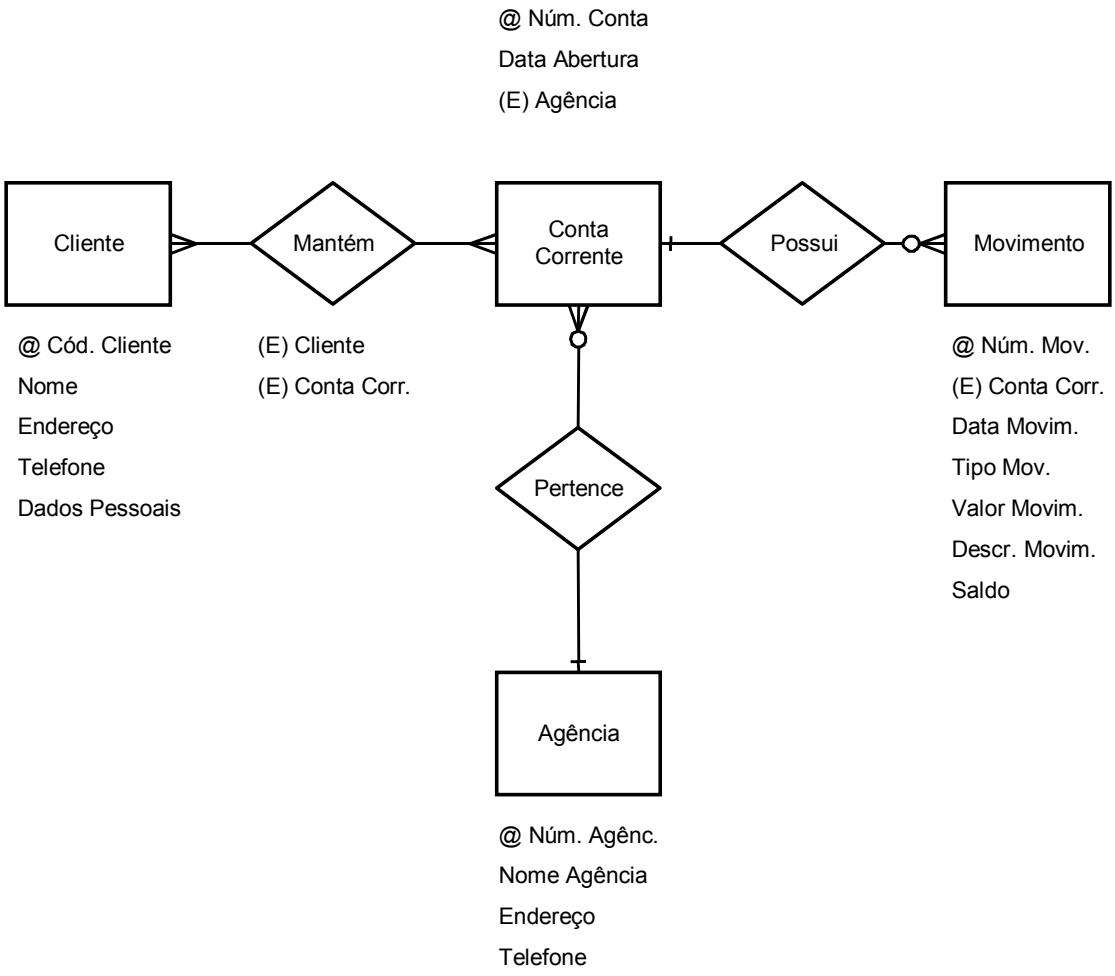
```
DROP INDEX <nome_tabela>.<nome_índice>
```

Argumento	*Significado*
`<nome_tabela>`	Nome da tabela à qual pertence o índice a ser eliminado.
`<nome_índice>`	Nome do índice a ser eliminado.

Exemplos:

```
DROP INDEX Funcionario.IX_DtAdm ;
DROP INDEX Funcionario.Ind_NmFunc ;
DROP INDEX Funcionario.DpCgNom ;
```

Exercícios: Caso "Banco simplificado"



1) Monte um script com os comandos DDL SQL necessários para criar as tabelas correspondentes ao modelo de dados "Banco simplificado" apresentado acima. Para isso, considere as seguintes observações:

- **a)** Atributos marcados com @ representam os identificadores de cada entidade
- **b)** Atributos marcados com (E) representam atributos de relacionamento na entidade onde aparecem
- **c)** Não são informações obrigatórias apenas: Telefone e Dados Pessoais do Cliente e Endereço da Agência
- **d)** O Nome da Agência não pode ser repetido dentro da tabela de agências
- **e)** O Tipo do Movimento deve ser "C" ou "D" apenas
- **f)** O Valor do Movimento deve ser > 0
- **g)** O Nome do Cliente não pode ser repetido dentro da tabela de Clientes
- **h)** Considere as especificações dos outros exercícios ao projetar as características de cada coluna

2) Monte um script com os comandos DML SQL necessários para povoar com dados as tabelas criadas no exercício anterior, obedecendo as seguintes especificações:

- **a)** 10 Clientes (sendo que 1 deles deve se chamar "João da Silva")
- **b)** 3 Agências (com códigos 0040, 0050 e 0060)
- **c)** 12 Contas Corrente (sendo que 2 devem ser contas conjuntas. Deve existir uma conta corrente de número 1027. O cliente "João da Silva" deve possuir 2 contas corrente)
- **d)** 20 Movimentos (cuidando para que o 1º movimento de uma conta corrente seja sempre um crédito. O valor do Saldo deverá ser calculado e fornecido corretamente no comando)

3) Monte um script com os comandos DML SQL necessários para executar as seguintes consultas:

- **a)** Exibir todas os dados de todas as contas corrente, por ordem de Código da Agência
- **b)** Exibir todas as contas da agência 0050, em ordem de Data de Abertura da conta
- **c)** Idem "b", exibindo apenas os campos referentes a Nome da Agência, Número da Conta e Data de Abertura
- **d)** Movimentos da conta número 1027, em ordem crescente de Data do Movimento. Exibir os campos referentes a Data do Movimento, Valor do Movimento, Tipo do Movimento e Número da Conta
- **e)** Todas as movimentações feitas pelo cliente "João da Silva" em todas as suas contas corrente, em ordem de Data do Movimento. Exibir os campos Data do Movimento, Valor do Movimento, Tipo do Movimento e Número da Conta
- **f)** Idem "e", mas exibindo apenas os débitos
- **g)** Exibir os movimentos das contas corrente com saldo negativo
- **h)** Relação total dos depósitos de cada agência. Exibir os campos referentes a Nome da Agência e Total de Depósitos. As agências deverão aparecer por ordem de Total de Depósitos
- **i)** Idem "h", mas exibir dados apenas dos depósitos realizados nos últimos 30 dias
- **j)** O total de depósitos de cada conta, em ordem descendente de volume de depósitos. Deverão aparecer apenas as contas com total de depósitos maior que 5000.

Comandos DCL SQL

A terceira parte dos recursos SQL refere-se ao gerenciamento dos direitos de acesso aos recursos do banco de dados. Esse controle é exercido através da criação de contas de usuários do banco de dados e da concessão de determinados privilégios a esses usuários.

Existem pré-definidos no SGBD Oracle alguns usuários administrativos (SYS, SYSTEM, SYSMAN, DBSNMP) e de exemplo (DEMO, SCOTT). Os primeiros servem para a instalação e gerenciamento do ambiente de banco de dados, fornecendo os poderes básicos para o administrador do banco de dados (DBA) executar suas tarefas, enquanto que os restantes destinam-se a exploração de alguns exemplos fornecidos pelo produto. Tipicamente a tarefa de gerenciar as contas e direitos dos usuários é uma tarefa do DBA.

Comando GRANT

Concede privilégios (direitos) sobre objetos/recursos do banco de dados a um usuário.

```
GRANT <lista de privilégios>
            [ON <objeto>]
            TO <usuário>
            [WITH GRANT OPTION] ;
```

Argumento	*Significado*
`<lista de privilégios>`	Relação dos privilégios a atribuir ao usuário. Existem cerca de 60 privilégios possíveis de serem atribuídos a um usuário.
`<objeto>`	Objeto do banco de dados (tabela, view, etc) sobre o qual os privilégios serão atribuídos.
`<usuário>`	Nome do usuário que receberá os privilégios.
`WITH GRANT OPTION`	Se especificado, indica que o usuário poderá repassar a outros o(s) direito(s) que recebeu.

Exemplos:

```
GRANT CONNECT, RESOURCE
            TO curso01sql ;
```

```
GRANT SELECT, INSERT, UPDATE
            ON Funcionario
            TO curso01sql ;
```

Comando REVOKE

Retira privilégios (direitos) sobre objetos/recursos do banco de dados de um usuário.

```
REVOKE <lista de privilégios>
            [ON <objeto>]
            FROM <usuário> ;
```

Argumento	*Significado*
`<lista de privilégios>`	Relação dos privilégios a retirar do usuário.
`<objeto>`	Objeto do banco de dados (tabela, view, etc) sobre o qual os privilégios serão retirados.
`<usuário>`	Nome do usuário que perderá os privilégios.

Exemplos:

```
REVOKE RESOURCE
            FROM curso01sql ;
```

```
REVOKE INSERT
            ON Funcionario
            FROM curso01sql ;
```

Comando CREATE USER

Cria um usuário para o SGBD.

```
CREATE USER <nome_usuario>
            [IDENTIFIED BY <senha>] ;
```

Argumento	*Significado*
<nome_usuario>	Nome do usuário a ser criado.
<senha>	Senha inicial do usuário.

Exemplo:

```
CREATE USER curso01sql
            IDENTIFIED BY curso01 ;
```

Comando DROP USER

Elimina um usuário do SGBD.

```
DROP USER <nome_usuario> ;
```

Argumento	*Significado*
<nome_usuario>	Nome do usuário a ser criado.

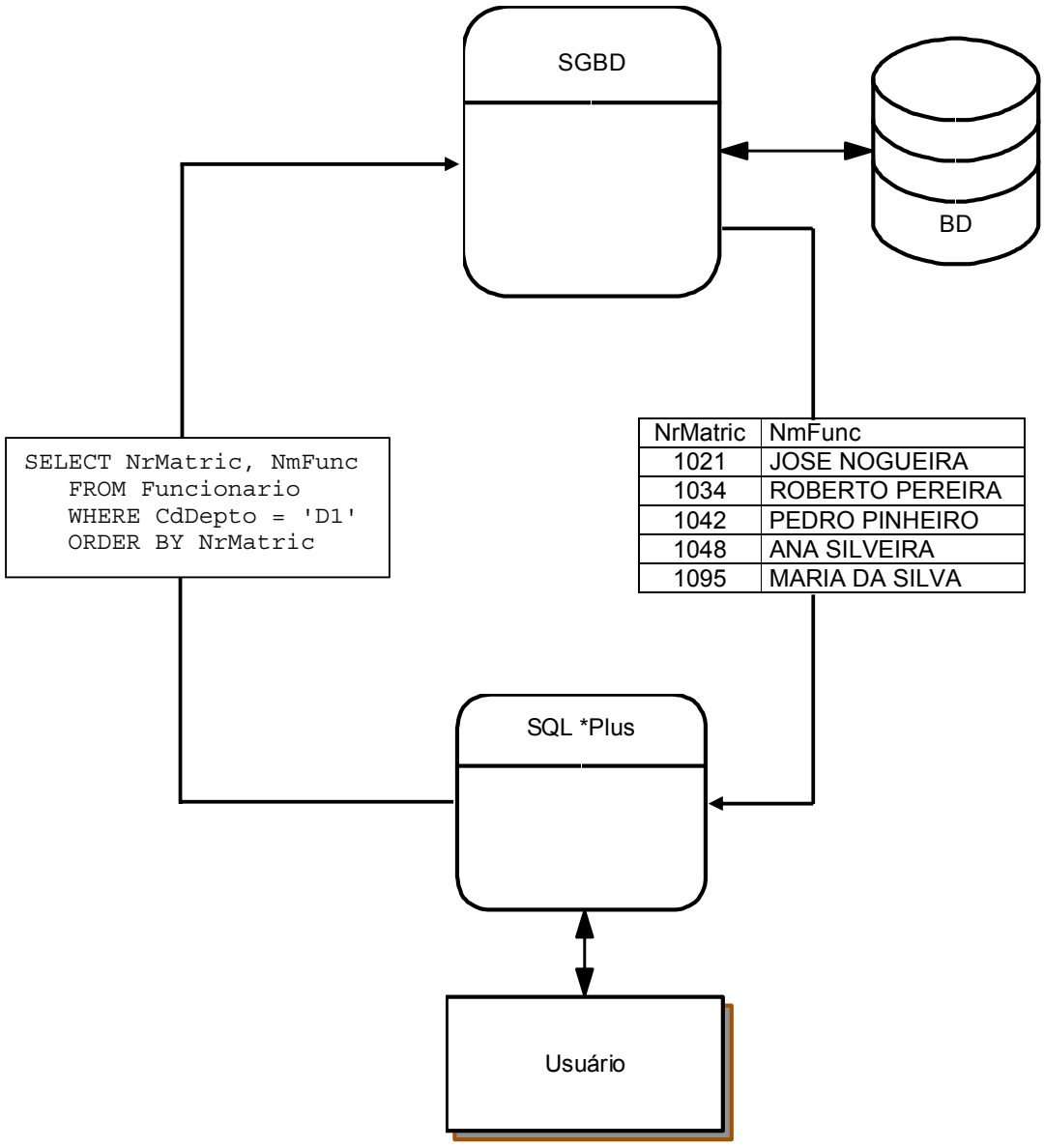
Exemplo:

```
DROP USER curso01sql ;
```

III SQL *Plus

Introdução

O SQL *Plus é uma ferramenta para consultas interativas fornecida pela Oracle Corp. para acesso aos bancos de dados disponíveis no servidor. Instalada no equipamento cliente (no caso de sistemas MS-Windows NT e Novell NetWare), a ferramenta posiciona-se como um intermediário entre o usuário e o SGBD conforme mostra o diagrama a seguir:



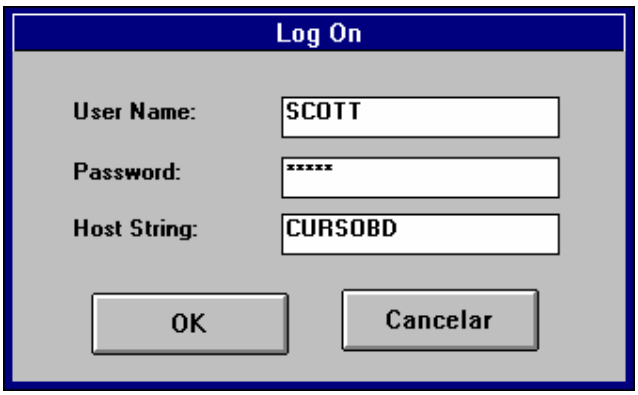
A interface do produto é relativamente simples, consistindo basicamente em uma janela gráfica (na versão Windows) com um prompt para digitação de comandos (tanto SQL como PL/SQL e aqueles específicos do SQL *Plus).

Os comandos digitados pelo usuário são armazenados num buffer (SQL Buffer) mantido na memória pela ferramenta. Apenas o último comando SQL pode ser recuperado. O SQL *Plus

permite a edição do buffer através de comandos que implementam um editor de linhas simplificado.

Abertura de sessão SQL *Plus

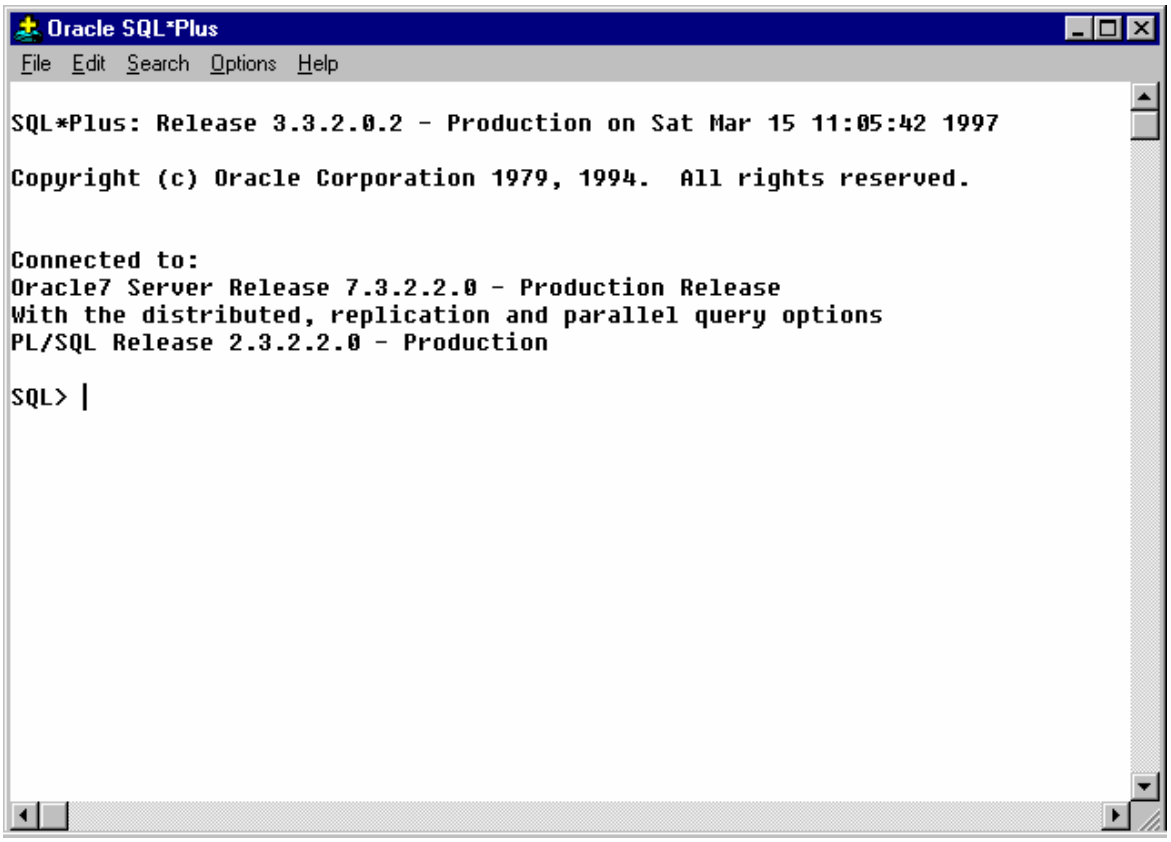
Para utilização do SQL *Plus é necessário que o usuário inicialmente conecte-se com o servidor, através da caixa de diálogo Log On.



Nessa janela deverão ser fornecidas as seguintes informações:

User Name	Nome da conta do usuário no SGBD.
Password	Senha do usuário no SGBD.
Host String	Alias do banco de dados a conectar.

Uma vez que a conexão seja estabelecida com sucesso, você terá na tela a janela do SQL *Plus:



No prompt dessa janela você poderá emitir os comandos que desejar. O resultados serão exibidos nessa mesma tela, imediatamente abaixo do que você digitou.

Um comando pode ser digitado ocupando várias linhas (que são numeradas automaticamente) e, para que seja executado você tem 3 opções:

1) colocar como último caractere do comando um ponto-e-vírgula (;) ;

2) colocar uma barra (/) na última linha; ou ainda,

3) deixar uma linha em branco no final do comando e, retornando ao prompt normal do SQL *Plus, emitir o comando RUN.

*Relação de comandos SQL *Plus*

Os principais comandos específicos do SQL *Plus são listados no quadro a seguir.

Tipo	*Comando*	*Finalidade*
Execução	/	Executa a declaração contida no buffer.
	!	Executa um comando do sistema operacional dentro da sessão SQL *Plus.
	RUN	Lista na tela e executa a declaração contida no buffer.
	EXECUTE	Executa uma declaração PL/SQL.
Edição	LIST	Lista a última declaração SQL (por inteiro ou parcialmente).
	APPEND	Adiciona um texto ao final da linha corrente.
	CHANGE	Modifica uma parte de uma declaração contida no buffer.
	DEL	Apaga linhas do buffer de comandos.
	INPUT	Insere uma ou mais novas linhas no buffer.
	EDIT	Ativa um editor de textos.
Ambiente	SET	Permite alterar variáveis de ambiente do SQL *Plus.
	SHOW	Exibe o conteúdo de uma variável de ambiente SQL *Plus.
Arquivos	SAVE	Salva o conteúdo do buffer em um arquivo em disco.
	START	Permite executar um arquivo de comandos.
	@	Permite executar um arquivo de comandos.
	GET	Carrega o arquivo de comandos para o buffer.
	SPOOL	Imprime ou grava em disco os resultados dos comandos.
Outros	DESCRIBE	Descreve a estrutura de um objeto do banco de dados.
	CONNECT	Connecta um novo usuário ao SGBD sem encerrar a sessão SQL *Plus.
	DISCONNECT	Desconecta o usuário do SGBD, sem encerrar a sessão SQL *Plus.
	EXIT	Desconecta o usuário do SGBD e encerra a sessão do SQL *Plus.
	QUIT	Desconecta o usuário do SGBD e encerra a sessão do SQL *Plus.

A seguir, uma rápida visão dos principais comandos específicos do produto. A sintaxe de alguns deles foi intencionalmente simplificada, de maneira a ilustrar os seus recursos mais importantes. Informações mais completas sobre cada comando você encontra no capítulo 6 do manual *SQL *Plus User's Guide and Reference* que é fornecido com o produto.

Comando LIST

Finalidade: Lista a última declaração SQL (por inteiro ou parcialmente). A última linha listada será considerada a *linha corrente do buffer* e aparecerá na tela precedida por um asterisco.

Sintaxe: **L[IST] [<nº da linha>] [LAST]**

Exemplos:

Para listar todo o conteúdo do buffer:

```
SQL> LIST
```

Aparecerá o último comando digitado, como por exemplo:

```
  1  SELECT NmFunc, CdDepto, VrSalario, CdCargo
  2  FROM Funcionario
  3  WHERE CdDepto = 'D1'
  4* ORDER BY NmFunc
```

Observe que a linha 4 é assinalada com um asterisco, mostrando que ela é a linha corrente do buffer.

Para listar somente a terceira linha, tornando-a corrente:

```
SQL> LIST 3
```

Supondo o comando SELECT anterior, aparecerá:

```
  3* WHERE CdDepto = 'D1'
```

Observe que a linha 3 é agora a linha corrente do buffer.

Comando APPEND

Finalidade: Adiciona um texto ao final da linha corrente.

Sintaxe: **A[PPEND] <texto>**

Exemplo:

Este exemplo considera que o comando armazenado no buffer é o seguinte:

```
1  SELECT NmFunc, CdDepto, VrSalario, CdCargo
2  FROM Funcionario
3  WHERE CdDepto = 'D1'
4* ORDER BY NmFunc
```

Para acrescentar a condicao AND VrSalario > 500 na cláusula WHERE do comando:

Tornando corrente a linha 3 que será modificada:

```
SQL> LIST 3
  3* WHERE CdDepto = 'D1'
```

Escrevendo agora o comando APPEND:

```
SQL> APPEND  AND VrSalario > 500
```

Verificando como ficou o comando inteiro:

```
SQL> LIST
  1  SELECT NmFunc, CdDepto, VrSalario, CdCargo
  2  FROM Funcionario
  3  WHERE CdDepto = 'D1' AND VrSalario > 500
  4* ORDER BY NmFunc
```

Comando CHANGE

Finalidade: Modifica uma parte de uma declaração contida no buffer. Este comando afeta apenas a linha corrente do buffer (aquela que aparece precedida por um asterisco no comando LIST).

Sintaxe: **C[HANGE] /<texto original>/<novo texto>/**

Exemplo:

Este exemplo considera que o comando armazenado no buffer é o seguinte:

```
1  SELECT NmFunc, CdDepto, VrSalario, CdCargo
2  FROM Funcionario
3  WHERE CdDepto = 'D1' AND VrSalario > 500
4* ORDER BY NmFunc
```

Para alterar o departamento a selecionar para 'D3':

Tornando corrente a linha 3 que será modificada:

```
SQL> LIST 3
  3* WHERE CdDepto = 'D1' AND VrSalario > 500
```

Escrevendo agora o comando CHANGE:

```
SQL> CHANGE /D1/D3/
```

Verificando como ficou a linha 3:

```
SQL> LIST 3
  3*  WHERE CdDepto = 'D3' AND VrSalario > 500
```

Para alterar o critério de ordenação dos dados para CdCargo:

Tornando corrente a linha 4 que será modificada:

```
SQL> LIST 4
  4* ORDER BY NmFunc
```

Escrevendo agora o comando CHANGE:

```
SQL> CHANGE /NmFunc/CdCargo/
```

Verificando como ficou a linha 4:

```
SQL> LIST 4
  4* ORDER BY CdCargo
```

Comando DEL

Finalidade: Apaga linhas do buffer de comandos. Se for emitido o comando sem cláusula adicional alguma, a linha corrente do buffer (aquela que aparece precedida por um asterisco no comando LIST) será excluída.

Sintaxe: **DEL [<nº da linha>] [LAST]**

Exemplo:

Este exemplo considera que o comando armazenado no buffer é o seguinte:

```
1  SELECT NmFunc, CdDepto, VrSalario, CdCargo
2  FROM Funcionario
3  WHERE CdDepto = 'D1'
4  AND VrSalario > 500
5* ORDER BY NmFunc
```

Para apagar a linha 4:

Tornando corrente a linha 4:

```
SQL> LIST 4
  4* AND VrSalario > 500
```

Escrevendo agora o comando DEL:

```
SQL> DEL
```

Verificando como ficou o buffer:

```
SQL> LIST
1  SELECT NmFunc, CdDepto, VrSalario, CdCargo
2  FROM Funcionario
3  WHERE CdDepto = 'D1'
4* ORDER BY NmFunc
```

Para apagar a linha 3 sem precisar torná-la corrente antes:

```
SQL> DEL 3
```

Verificando como ficou o buffer:

```
SQL> LIST
  1  SELECT NmFunc, CdDepto, VrSalario, CdCargo
  2  FROM Funcionario
  3* ORDER BY NmFunc
```

Comando DESCRIBE

Finalidade: Descreve a estrutura de um objeto do banco de dados. Se o objeto for uma tabela, view ou sinônimo, o comando listará as definições de suas colunas; se o objeto for uma função ou procedimento do banco de dados, serão exibidas as especificações de seus parâmetros formais (caso existam) e o tipo do valor retornado (se o objeto for uma função).

Sintaxe: **DESC[RIBE] <objeto>**

Exemplo:

Para descrever a estrutura da tabela Funcionario:

```
SQL> DESCRIBE FUNCIONARIO

Name                           Null?    Type
------------------------------ -------- ----
NRMATRIC                       NOT NULL CHAR(4)
NMFUNC                         NOT NULL VARCHAR2(30)
DTADM                          NOT NULL DATE
VRSALARIO                      NOT NULL NUMBER(10,2)
CDCARGO                                 CHAR(2)
CDDEPTO                                 CHAR(2)
```

Para descrever o procedimento AchaFuncionario:

```
SQL> DESCRIBE ACHAFUNCIONARIO

PROCEDURE ACHAFUNCIONARIO
Argument Name                  Type     In/Out   Default?
------------------------------ -------- -------- ---------
MATRICULA                      NUMBER   IN
NOME                           VARCHAR2 OUT
```

Comando EXIT

Finalidade: Encerra a sessão SQL *Plus e retorna o controle ao sistema operacional.
Sintaxe: **EXIT**
Exemplo:

```
SQL> EXIT
```

Comando INPUT

Finalidade: Insere uma ou mais novas linhas no buffer. A(s) nova(s) linha(s) será(ão) inserida(s) após a linha corrente do buffer (aquela que aparece precedida por um asterisco no comando LIST).
Sintaxe: **I[NPUT] [<texto>]**
Exemplo:
Este exemplo considera que o comando armazenado no buffer é o seguinte:

```
1  SELECT NmFunc, CdDepto, VrSalario, CdCargo
2  FROM Funcionario
3  WHERE CdDepto = 'D1'
```

Para acrescentar uma linha com um critério de ordenação ao final do SELECT:
Tornando corrente a última linha do buffer:

```
SQL> LIST
  1  SELECT NmFunc, CdDepto, VrSalario, CdCargo
  2  FROM Funcionario
  3* WHERE CdDepto = 'D1'
```

Escrevendo agora o comando INPUT:

```
SQL> INPUT ORDER BY NmFunc
```

Verificando como ficou o comando SELECT no buffer:

```
SQL> LIST
  1  SELECT NmFunc, CdDepto, VrSalario, CdCargo
  2  FROM Funcionario
  3  WHERE CdDepto = 'D1'
  4* ORDER BY NmFunc
```

Para adicionar uma linha com mais uma condição para a cláusula WHERE do SELECT:

Tornando corrente a linha 3 após a qual será feita a inclusão:

```
SQL> LIST 3
  3* WHERE CdDepto = 'D1'
```

Escrevendo agora o comando INPUT:

```
SQL> INPUT AND VrSalario > 500
```

Verificando como ficou o comando SELECT no buffer:

```
SQL> LIST
  1  SELECT NmFunc, CdDepto, VrSalario, CdDepto
  2  FROM Funcionario
  3  WHERE CdDepto = 'D1'
  4  AND VrSalario > 500
  5* ORDER BY NmFunc
```

Instruções para consultas ao catálogo do SGBD

Um SGBD deve armazenar internamente informações sobre os diversos objetos contidos no banco de dados, tanto para poder compilar e otimizar o processamento das consultas solicitadas pelos usuários como também para autorizar os acessos e facilitar as tarefas de gerenciamento. O conjunto de tabelas e visões do sistema responsáveis pelo armazenamento e organização dessas informações é chamado de Catálogo do SGBD ou Dicionário de Dados do SGBD. Existem no mercado alguns utilitários gráficos que podem ser utilizados para gerenciar o banco de dados e fornecer acesso rápido e intuitivo ao catálogo, tais como o pacote Oracle Enterprise Manager, por exemplo. Entretanto, também é possível realizar esse tipo de consulta por meio de instruções SQL relativamente simples, como as que serão mostradas a seguir, consideradas de grande interesse no dia-a-dia de utilização do SGBD.

Como obter os nomes das tabelas disponíveis no banco de dados

É possível verificar quais as tabelas disponíveis através das seguintes consultas:

```
SELECT * FROM user_tables ;
```

ou

```
SELECT table_name FROM user_tables ;
```

ou

```
SELECT table_name FROM all_tables ;
```

ou

```
SELECT table_name FROM tabs ;
```

Nos dois primeiros casos, visualiza-se apenas as tabelas criadas pelo próprio usuário, nos dois últimos, todas as tabelas às quais ele possui algum tipo de direito no banco de dados.

Como ver a estrutura de uma tabela do banco de dados

Para descobrir os nomes das colunas de uma tabela, bem como o tipo de dados de cada uma e o seu tamanho, existe o comando DESCRIBE do SQL *Plus que já foi apresentado anteriormente, mas também é possível fazer essa consulta via SQL, com a seguinte instrução:

```
SELECT Column_Name, Data_Type, Data_Length,
       Data_Precision, Data_Scale,
       DECODE( Nullable, 'Y', 'NULL', 'N', 'NOT NULL' ) as Nullable
FROM User_Tab_Columns
WHERE Table_Name = <nome_tabela_em_maiúsculas> ;
```

Por exemplo, para exibir a estrutura da tabela chamada Funcionario deve-se digitar:

```
SELECT Column_Name, Data_Type, Data_Length,
       Data_Precision, Data_Scale,
       DECODE( Nullable, 'Y', 'NULL', 'N', 'NOT NULL' ) as Nullable
FROM user_tab_columns
WHERE Table_Name = 'FUNCIONARIO' ;
```

Como ver os índices de uma tabela

Para descobrir os nomes dos índices já criados para uma tabela, bem como suas características principais e expressão chave, pode-se fazer a seguinte consulta:

```
SELECT Index_Name, Column_Name AS Chave,
       Column_Position, Column_Length
   FROM User_Ind_Columns
   WHERE Table_Name = <nome_tabela_em_maiúsculas> ;
```

Por exemplo, para exibir detalhes sobre todos os índices tabela chamada Funcionario deve-se digitar:

```
SELECT Index_Name, Column_Name AS Chave,
       Column_Position, Column_Length
   FROM User_Ind_Columns
   WHERE Table_Name = 'FUNCIONARIO' ;
```

Como obter os nomes das visões disponíveis no banco de dados

É possível verificar quais as visões existentes através das seguintes consultas:

```
SELECT View_Name FROM User_Views ;
```

ou

```
SELECT Owner, View_Name FROM All_Views ;
```

No primeiro caso, visualiza-se apenas os nomes das visões criadas pelo próprio usuário e no segundo, todas as visões às quais ele possui algum tipo de direito no banco de dados.

Como descobrir o SELECT de uma visão já criada no banco de dados

É possível verificar qual o SELECT referente a uma visão através das seguintes consultas:

```
SELECT Text
   FROM User_Views
   WHERE View_Name = <nome_da_visão_em_maiúsculas> ;
```

ou

```
SELECT Text
   FROM All_Views
   WHERE Owner = <nome_do_dono_em_maiúsculas>
         AND View_Name = <nome_da_visão_em_maiúsculas> ;
```

Como modificar a senha de um usuário do banco de dados

A modificação da senha só é permitida para o próprio dono da conta ou para um usuário que possua privilégios administrativos (em geral, o role DBA). A instrução para fazer a modificação é:

```
ALTER USER <nome_conta> IDENTIFIED BY <nova_senha> ;
```

Por exemplo, para alterar a senha do usuário 'VENDAS1' para 'axt027' a instrução seria a seguinte:

```
ALTER USER Vendas1 IDENTIFIED BY axt027 ;
```